

PROGRAMMERS MANUAL FOR TFT FLAT PANEL DETECTORS



Name: Programmers Manual HBI SDK
Number: 4000023446
Revision: A
State: In Work
Classification: Public
Subcategory: Software Programmer's Manual

Revision History

Version	Date	Description	Author
1.0.0.0	13.4.2026	1st version in general DT format	X. Yu M. Yang M. Kurkela

CONTENTS

Revision History.....	2
1. Introduction.....	9
1.1. Purpose	9
1.2. About The SDK.....	9
1.3. Environment	10
1.4. Installation.....	10
1.4.1. Ethernet Interface with Standard UDP	10
1.4.2. Ethernet Interface with Jumbo Packets	12
1.4.3. Optical Fiber Interface (with DMA Data Acquisition Card).....	13
1.4.4. Wireless Interface with Standard UDP Packet	18
2. Communication Overview.....	19
2.1. Data Flow Chart.....	19
2.2. Call Flow Chart	20
2.3. Instance Handles and Panel Device ID	24
2.4. Callback Functions	24
2.5. Trigger Mode	26
2.6. Image Calibration.....	27
2.7. Command	29
2.7.1. Request Command	29
2.7.2. Response Command	30
2.8. Data Structure	32
2.8.1. Connection Parameters	32
2.8.2. Image Properties.....	32
2.8.3. Trigger Mode Classification	33
2.8.4. Continuous Collection and Classification	33
2.8.5. Collection Mode Types	34
2.8.6. Collecting Image Parameters	35
2.8.7. Correction Enable setting parameters	36
2.8.8. Download Template File Parameters	36
2.8.9. Mode Parameter	39
2.8.10. Multiple Mode List	40
2.8.11. Other Parameters	40
2.9. Flat Panel Status	40
2.9.1. Static Flat Panel Detector Status of Ethernet Interface	41
2.9.2. Dynamic Flat Panel Detector Status of Network Port	42
2.9.3. Dynamic Flat Panel Status of Optical Fiber Interface.....	43
2.9.4. Wireless Static Flat Panel Status	45
3. API Function Description	47
3.1. Function Types	47
3.2. Function Classification	48
3.2.1. Initialize and Release Resources	48
3.2.2. Registering Callback Functions	48
3.2.3. Connecting and Disconnecting Panel	49
3.2.4. Trigger Mode and Calibration Enable Settings	49
3.2.5. Single Frame Acquisition and Continuous Acquisition.....	50

3.2.6.	Frame Rate, PGA, and Binning Setting	50
3.2.7.	Single Frame Acquisition Mode	51
3.2.8.	Continuous Acquisition Mode	52
3.2.9.	Quick Template Generation and Template Download	53
3.2.10.	Image Attributes, Software and Firmware Versions	54
3.2.11.	Parameters and Restoring Factory Settings	54
3.2.12.	Zoom (ROI) Area Settings	54
3.2.13.	Calibration Template Path	55
3.2.14.	Image Display and Saving	55
3.2.15.	Packet Interval	55
3.2.16.	Image Rotation/Mirroring	55
3.2.17.	Firmware Template Management Method	56
3.2.18.	API Functions under Multi-template Management Mode	57
3.3.	Interface Functions	58
3.3.1.	HBI_Init	58
3.3.2.	HBI_Destroy	59
3.3.3.	HBI_ConnectDetector	59
3.3.4.	HBI_ConnectDetectorUdp	60
3.3.5.	HBI_ConnectDetectorJumbo	61
3.3.6.	HBI_ConnectDetectorPcie	61
3.3.7.	HBI_ConnectDetectorEx	62
3.3.8.	HBI DisconnectDetector	64
3.3.9.	HBI_RegEventCallBackFun	64
3.3.10.	HBI_ResetDetector	65
3.3.11.	HBI_ResetFirmware	65
3.3.12.	HBI_GetFPDSerialNumber	66
3.3.13.	HBI_GetSDKVersion	66
3.3.14.	HBI_GetFirmwareVerion	67
3.3.15.	HBI_GetError	67
3.3.16.	HBI_GetFpdCfgInfo	68
3.3.17.	HBI_GetImageProperty	69
3.3.18.	HBI_SinglePrepare	69
3.3.19.	HBI_SingleAcquisition	70
3.3.20.	HBI_LiveAcquisition	71
3.3.21.	HBI_StopAcquisition	72
3.3.22.	HBI_SetSinglePrepareTime	73
3.3.23.	HBI_GetSinglePrepareTime	73
3.3.24.	HBI_SetLiveAcquisitionTime	74
3.3.25.	HBI_GetLiveAcquisitionTime	74
3.3.26.	HBI_TriggerAndCorrectDisplay	75
3.3.27.	HBI_UpdateTriggerMode	76
3.3.28.	HBI_UpdateCorrectEnable	77
3.3.29.	HBI_GetCorrectEnable	77
3.3.30.	HBI_SetSoftwareCorrect	78
3.3.31.	HBI_GetSoftwareCorrect	78
3.3.32.	HBI_SetAedIntegrateTm	79
3.3.33.	HBI_GetAedIntegrateTm	79
3.3.34.	HBI_SetAedThreshold	80
3.3.35.	HBI_GetAedThreshold	80

3.3.36.	HBI_SetsaturationValue	81
3.3.37.	HBI_GetsaturationValue	81
3.3.38.	HBI_SetClippingValue	82
3.3.39.	HBI_GetClippingValue	82
3.3.40.	HBI_SetLPFLevel	83
3.3.41.	HBI_GetLPFLevel	84
3.3.42.	HBI_SetPGAlevel	84
3.3.43.	HBI_GetPGAlevel	85
3.3.44.	HBI_SetBinning	86
3.3.45.	HBI_GetBinning	87
3.3.46.	HBI_SetPreviewMode	87
3.3.47.	HBI_GetPreviewMode	88
3.3.48.	HBI_SAecModeApply	89
3.3.49.	HBI_SAecAcq	90
3.3.50.	HBI_RegProgressCallBack	90
3.3.51.	HBI_GenerateTemplate	91
3.3.52.	HBI_DownloadTemplate	92
3.3.53.	HBI_StopDownloadTemplate	93
3.3.54.	HBI_InitDual	93
3.3.55.	HBI_InitEx	94
3.3.56.	HBI_DestroyEx	94
3.3.57.	HBI_GetDetectorHandle	95
3.3.58.	HBI_GetDetectorId	95
3.3.59.	HBI_ActivateDetector	96
3.3.60.	HBI_GetPreviewImageProperty	96
3.3.61.	HBI_PgaBinningAcqTime	97
3.3.62.	HBI_TriggerPgaBinningAcqTime	97
3.3.63.	HBI_TriggerBinningAcqTime	98
3.3.64.	HBI_PgaAcqTime	100
3.3.65.	HBI_SetSelfDumpingTime	100
3.3.66.	HBI_GetSelfDumpingTime	101
3.3.67.	HBI_TriggerPgaBinningAcqTimeCorrect	101
3.3.68.	HBI_OpenTemplateWizard	102
3.3.69.	HBI_SetRoiMode	103
3.3.70.	HBI_SetZoomModeRect	104
3.3.71.	HBI_SetCorrectFileDirectory	105
3.3.72.	HBI_GetCorrectFileDirectory	106
3.3.73.	HBI_UpdateSaveOption	106
3.3.74.	HBI_SetPacketIntervalTime	107
3.3.75.	HBI_AutoWindowWidthWindowLevel	107
3.3.76.	HBI_SetImageRotateAndFlip	108
3.3.77.	HBI_Rotate90AndFlip	109
3.3.78.	HBI_ResetCorrectTemplate	109
3.3.79.	HBI_GetCurZoomRect	110
3.3.80.	HBI_IsExistsOfCurCorrectTemplate	111
3.3.81.	HBI_GetCurrentCorrectFileName	111
3.3.82.	HBI_GetCurTempFileFullDirectory	112
3.3.83.	HBI_GetCurrentCorrectFilePath	112
3.3.84.	HBI_Prepare	113

3.3.85.	HBI_DownloadTemplateByType	114
3.3.86.	HBI_GetCurModelList	114
3.3.87.	HBI_UpdateModelList	115
3.3.88.	HBI_GetCurModeData	115
3.3.89.	HBI_UpdateCurModeData	116
3.3.90.	HBI_UpdateModeData	116
3.3.91.	HBI_GetCurModelID	117
3.3.92.	HBI_SwitchUserMode	117
3.3.93.	HBI_ModeCorrectTempalteEnable	118
3.3.94.	HBI_AllModeCorrectTempalteEnable	118
3.3.95.	HBI_SaveModeList2File	119
3.3.96.	HBI_DownloadModeTemplate	119
3.3.97.	HBI_DownloadModeTemplateByType	120
3.3.98.	HBI_DownloadModeTemplateByldAndType	121
3.3.99.	HBI_GetCurModeTemplateStatus	121
3.3.100.	HBI_FirmwareManagementTempalteType	122
3.3.101.	HBI_SetValidModelItemNumber	122
3.3.102.	HBI_IsMutilMode	123
3.3.103.	HBI_MUpdateTriggerMode	123
3.3.104.	HBI_MSetBinning	124
3.3.105.	HBI_MSetPGAlevel	125
3.3.106.	HBI_MSetRoiMode	126
3.3.107.	HBI_MSetZoomModeRect	127
3.3.108.	HBI_MSetSelfDumpingTime	128
3.3.109.	HBI_MSetLiveAcquisitionTime	128
3.3.110.	HBI_MSetFpsOfTime	129
3.3.111.	HBI_MPgaAcqTime	130
3.3.112.	HBI_PgaBinningAcqTime	131
3.3.113.	HBI_MTriggerAndCorrectDisplay	131
3.3.114.	HBI_MTriggerBinningAcqTime	132
3.3.115.	HBI_MTriggerPgaBinningAcqTime	133
3.3.116.	HBI_MTriggerPgaBinningAcqTimeCorrect	135
3.3.117.	HBI_GetModeData	136
3.3.118.	HBI_GetModeListSize	136
3.3.119.	HBI_ModeExists	137
3.3.120.	HBI_CheckCurtMode	137
3.3.121.	HBI_InitDetectorParams	138
3.3.122.	HBI_SetWindowMode	139
3.3.123.	HBI_GetWindowMode	139
3.3.124.	HBI_SetBinningDivisor	140
3.3.125.	HBI_GetBinningDivisor	140
3.4.	Callback Event	141
3.5.	Return Code	149
4.	Detector calibration	157
4.1.	About Image Calibration	157
4.2.	Quick Template Generation	160
4.2.1.	Offset Template	161
4.2.2.	Gain Template	161

4.2.3.	Defect Template	162
4.2.4.	Template Download	162
4.2.5.	Calibration Enable.....	163
4.3.	Template Wizard Resources	164
4.3.1.	Open Template Wizard.....	164
4.3.2.	Offset Template Generation Page	165
4.3.3.	Gain Template Generation Page	166
4.3.4.	Defect Template Generation Page.....	167
4.3.5.	Configuration Page	168
4.4.	Template Management.....	169
4.4.1.	Software Template Management	169
4.4.2.	Firmware Template Management	169
5.	Secondary Development	171
5.1.	Interface Description.....	171
5.2.	Software deployment.....	172
5.3.	Binning Integration.....	172
5.3.1.	Steps	172
5.3.2.	Offset Template Generation and Download	178
5.3.3.	Gain Template Generation and Download	181
5.3.4.	Defect Template Generation and Download.....	183
5.3.5.	Progress Status Callback Function.....	186
5.3.6.	Update Calibration Setting	189
5.4.	Multi-template Management and Integration	190
5.4.1.	Integration	191
5.4.2.	Multi-template Management	197
5.4.2.1.	Multi-template Setting	198
5.4.2.2.	Updating Mode List.....	199
5.4.2.3.	Switching Mode Items	206
5.4.2.4.	Calibration Template Generation and Download	207
5.4.3.	Offset Template Generation and Download	210
5.4.4.	Gain Template Generation and Download	213
5.4.5.	Defect Template Generation and Download.....	216
5.4.6.	Progress Status Callback Function.....	219
5.4.7.	Status of Calibration Template	222
5.4.8.	Update Calibration.....	224
5.5.	Problems and Suggestions.....	225
6.	Image Overlay.....	226
6.1.	Software Overlay	226
6.2.	Firmware Overlay	227
7.	Rotation and Mirroring.....	229
7.1.	Software Rotation and Mirroring.....	229
7.2.	Firmware Mirroring	230
8.	Preview Mode	231
9.	SAEC Mode	233
10.	IP Configuration	236
11.	Dual Panel Integration.....	238

12. Appendix	240
Appendix I: Open-source code example.....	240
Appendix II: Table of panel types and resolutions.....	240
Appendix III: Configuration Files	242
Appendix IV: Callback Function Parameters and States	243
Appendix V: Switching from old version to the latest version.....	244
Appendix VI: Window width and window level adjustment	249

1. Introduction

1.1. Purpose

The main purpose is to facilitate users to quickly complete secondary development, and document support is provided. Additionally, users can refer to the SDK Demo source code project and its instruction manual.

The target audience is users who are conducting secondary development based on Detection Technology PLC provided flat panel detectors.

1.2. About The SDK

For the convenience of user integration development, we provide a standard dynamic link library (DLL) interface. The user can call the interface program encapsulated in the dynamic link library by importing it to achieve control and image acquisition of the flat panel detector. At present, this version is highly integrated and supports communication methods including wired, wireless, and optical fiber communication. In addition, it supports a "dual (multi) panel" solution.

Main Content: The functions, parameters, calling process, correction template generation/download, and development support of the SDK function library interface. The following mainly introduces "single panel" integration, and we will separately explain the "dual panel" solution in Chapter 12.

The overall structure of the software system and the main functions of the interface layer include image acquisition, image calibration, and system management, as shown in Figure 1:

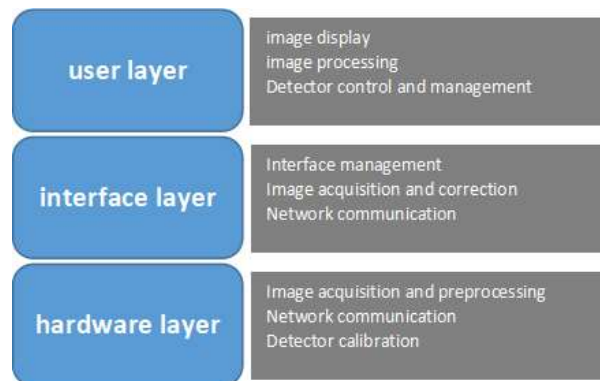


Figure 1

This interface software is implemented with C++. The return type is a pointer type, indicating that a handle or string address or custom data needs to be returned.

This software can run on 32-bit or 64-bit Windows operating systems.

Supporting both Ethernet and optical fiber communication mechanism.

This interface package is based on the C/C++ development language, pay attention to dependency libraries.

Including Header Files:

HbiFpd.h // Interface function file

HbiType.h // parameter, structure data type file

HbiError.h // Error List File

Linking the Library with dynamic library files:

Generate Library: HBISDKApi.lib // Static Link Library

HBISDKApi.DLL // Dynamic Link Library

Dependency library: opencv_World341.dll // Opencv library

1.3. Environment

Operating System: Windows 7 or above, both 32-bit and 64-bit are supported. Linux with Ubuntu is supported.

Memory: >= 4GB

Hard Disk: >= 100G

NIC: 1G/2.5G/10G Link Speed, Jumbo packets need to be supported for dynamic products.

PCIe Card: Optical fiber interface products.

1.4. Installation

At present, this interface package supports four communication methods:

1. Ethernet Interface with standard UDP
2. Ethernet Interface with Jumbo packets
3. Optical fiber interface with DMA data acquisition card
4. Wireless Interface with standard UDP.

You can modify the flat panel IP through the XDiscovery tool.

1.4.1. Ethernet Interface with Standard UDP

For detailed ethernet settings, please refer to the XDiscovery User's Manual (4000023445_Users_Manual_XDiscovery.pdf).

1. Environment Configuration:

Local IP address: 192.168.10.20

Remote Device IP Address: 192.168.10.40

as shown in Figure 2

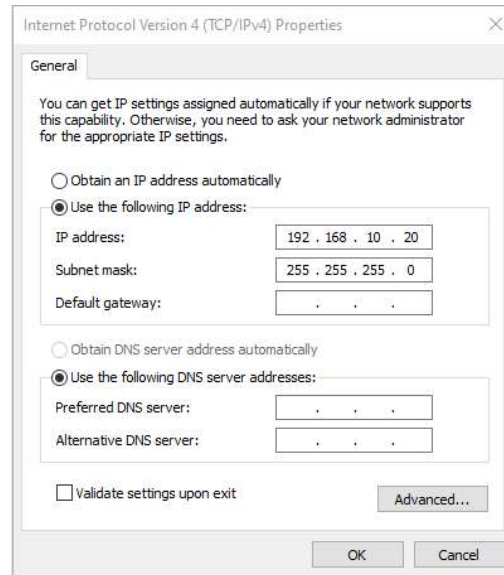


Figure 2

Tips: When opening the tool software for the first time and prompted to disable or allow when connecting, select allow. Otherwise, there may be a failed connection or a missing image. If there is such a situation, pay attention to checking and configuring the firewall. There are two methods Figure 3 and Figure 4:

Method 1: Directly close the firewall;

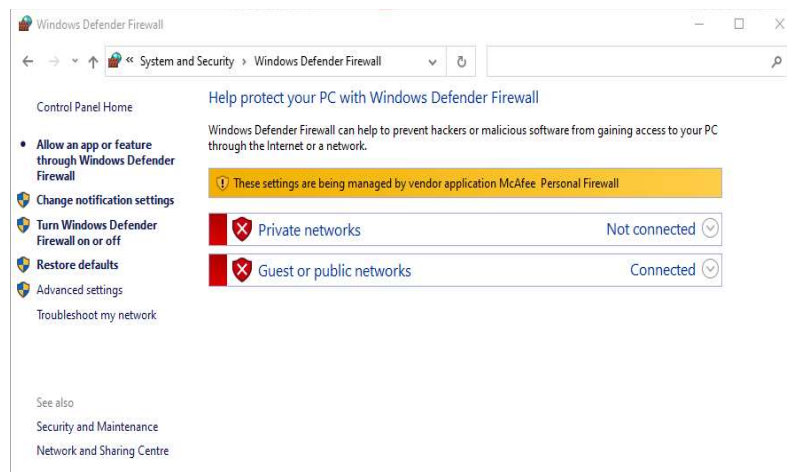


Figure 3

Method 2: Firewall exception setting - "Allow applications to pass through Windows Firewall";

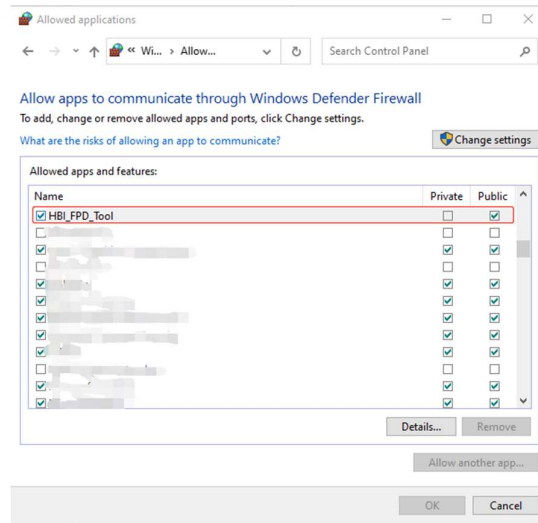


Figure 4

1.4.2. Ethernet Interface with Jumbo Packets

The configuration method for Ethernet Interface with Jumbo packets is the same as '1.4.1', only the default parameters of the network card need to be modified (provided that the network card supports JUMBO settings), as follows:

1. Jumbo Packet Setting

The jumbo packet mechanism is a hardware-based requirement that enables high transmission bandwidth. To achieve optimal performance, the product should be connected via a NIC card that supports the jumbo packet mechanism; such a NIC card is typically included as a standard accessory. Customers are advised to use the provided NIC card for product connection and image acquisition, as alternative NIC cards may not have undergone the necessary reliability validation tests. The supplied NIC card has been thoroughly verified across various operating systems and hardware configurations, thereby minimizing compatibility issues.

(1) Control Panel ->Select "Network Card" ->Right click "Properties" ->Open the "Ethernet Properties" page, as follows Figure 5 and Figure 6:

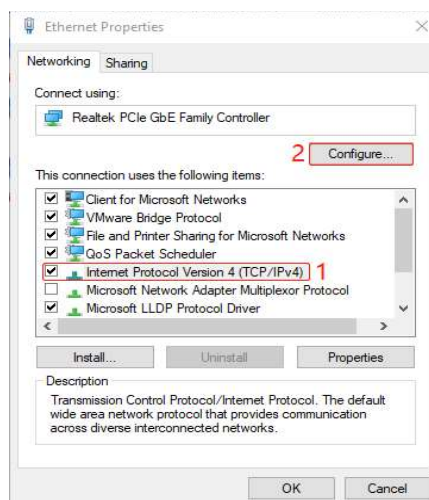


Figure 5

(2) "Advanced" - "Jumbo Packet" - MTU Value "9014 Bytes".

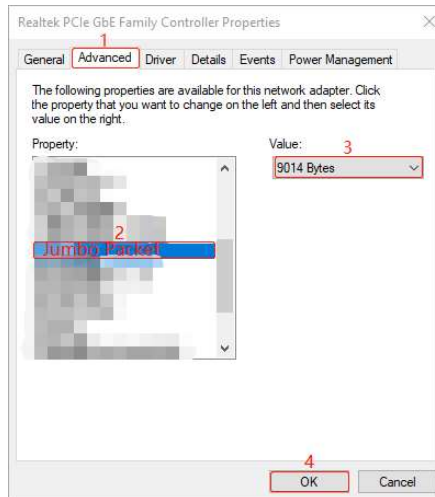


Figure 6

(3) "Advanced" - "Receive Buffers" - "Value" 2048 ".

(4) "Advanced" - "Transmit Buffers" - "Value" 2048" - "OK".

Reference value for all should be the maximum available.

1.4.3. Optical Fiber Interface (with DMA Data Acquisition Card)

1. Environment Configuration:

After installing the PCIe device card, connect the optical fiber cable to the flat panel and PC and connect the power supply.

Open "Device Manager" ->"Other Devices" ->"PCI Serial Port", as shown in the figure below, there will be a yellow exclamation mark if the PCIe driver is not installed correctly.

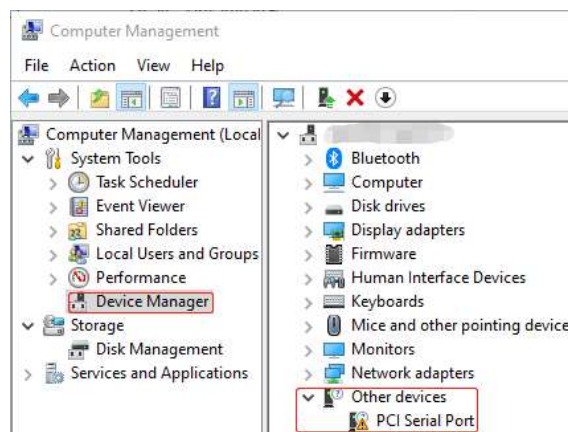


Figure 7

2. Installing Drivers

Right click on "PCI Serial Port" ->Select "Update Driver " ->"Browse my computer for drivers" ->Select "XDA.sys"
->"Install the driver files" ->The installation is completed by default.

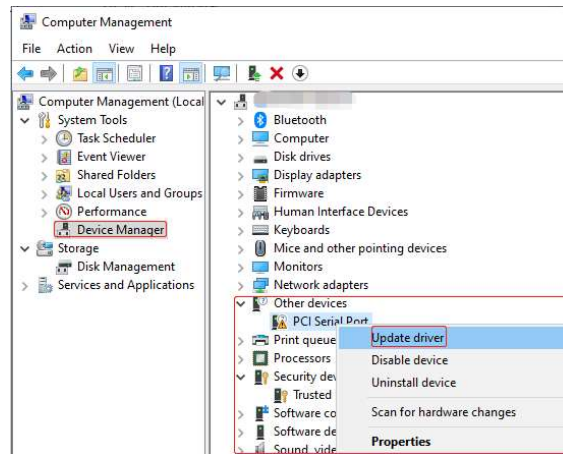


Figure 8

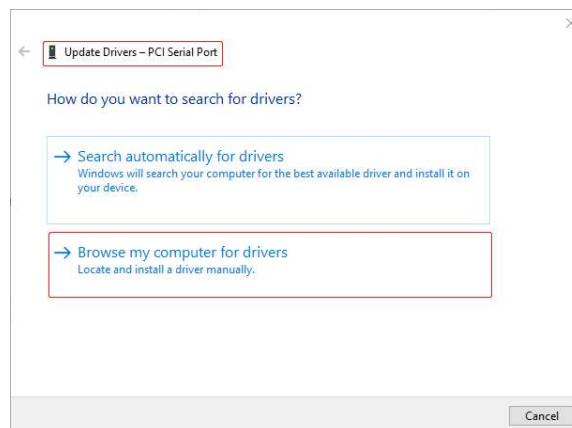


Figure 9

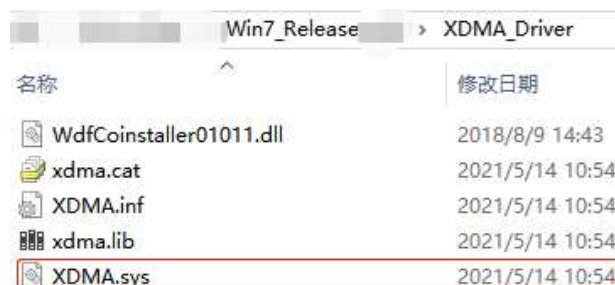


Figure 10

When windows security alert appears, choose to always install this driver software.



Figure 11

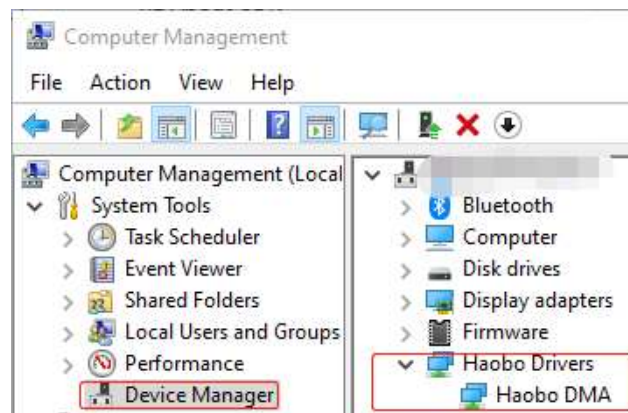


Figure 12

At present, the PCIe driver has been successfully installed.

3. Restart and Verify

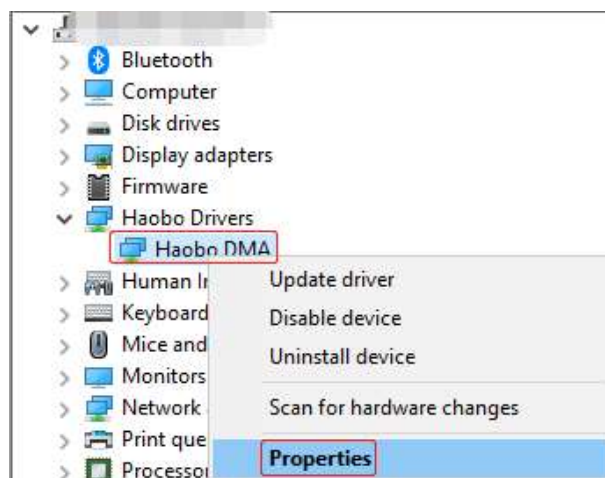


Figure 13

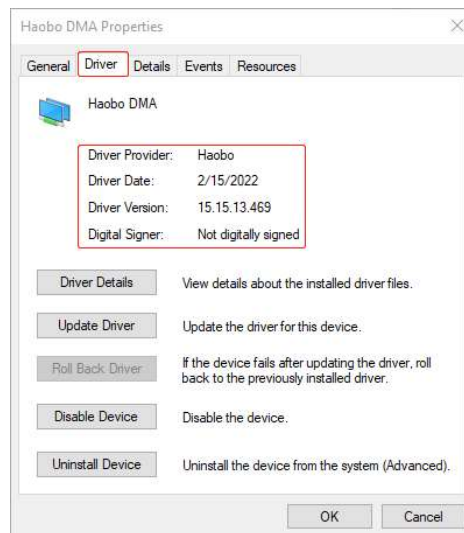


Figure 14

4. Update and Uninstall

To update the PCIe device driver, first uninstall the driver and then reinstall it.



Figure 15

5. Set Startup Options

The purpose of the following settings is to avoid failure of loading PCIe device drivers during startup.

Steps: "Control Panel" -> "All Control Panel Items" -> "Power Options" -> "Select the function of the power button" -> "Change currently unavailable settings" -> "Cancel" Start Quick Start (recommended) -> "Save Changes".

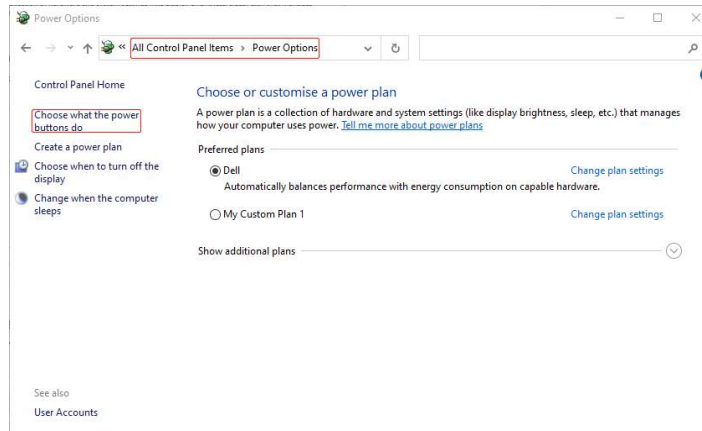


Figure 16

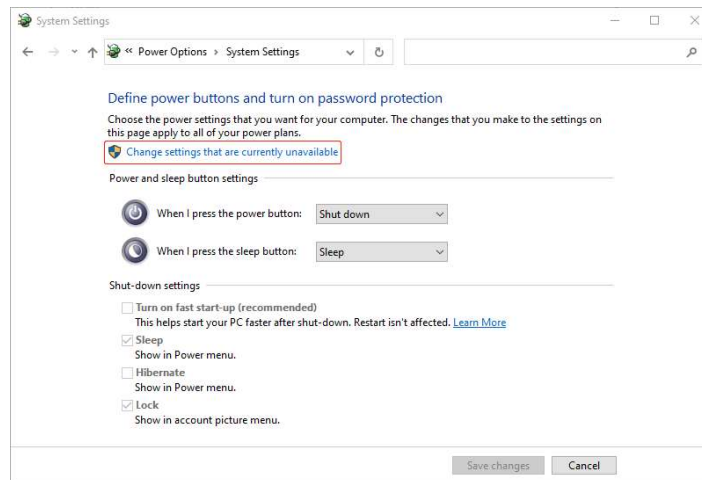


Figure 17

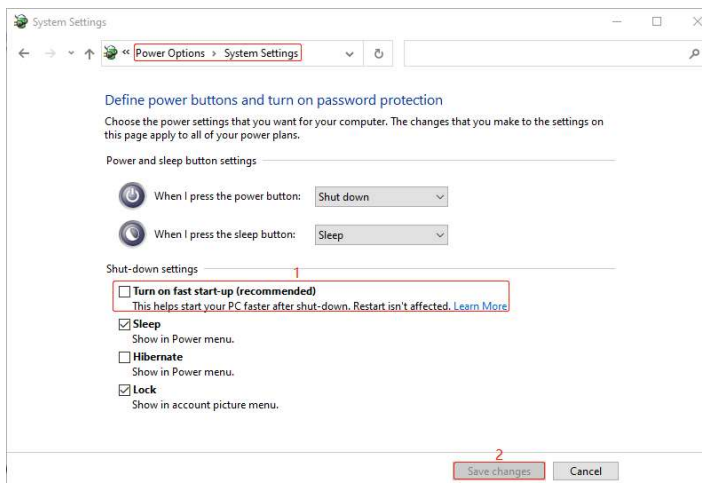


Figure 18

1.4.4. Wireless Interface with Standard UDP Packet

Due to the product not being released, the detailed configuration will be updated later.

2. Communication Overview

2.1. Data Flow Chart

Data communication between this software and detector devices is categorized into two types: command data and image data. Command data comprises all system parameters and command sets, whereas image data consists of pixel signal arrangements derived from the TFT sensor. The data flow is shown in and Figure 20.

1.Command Data Flow Chart:

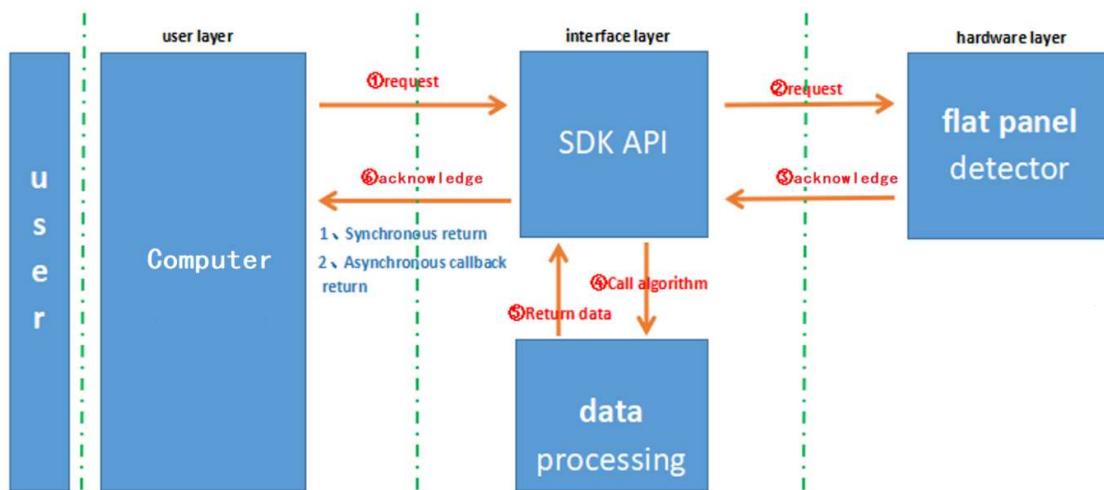


Figure 19

2.Image Data Flow Chart:

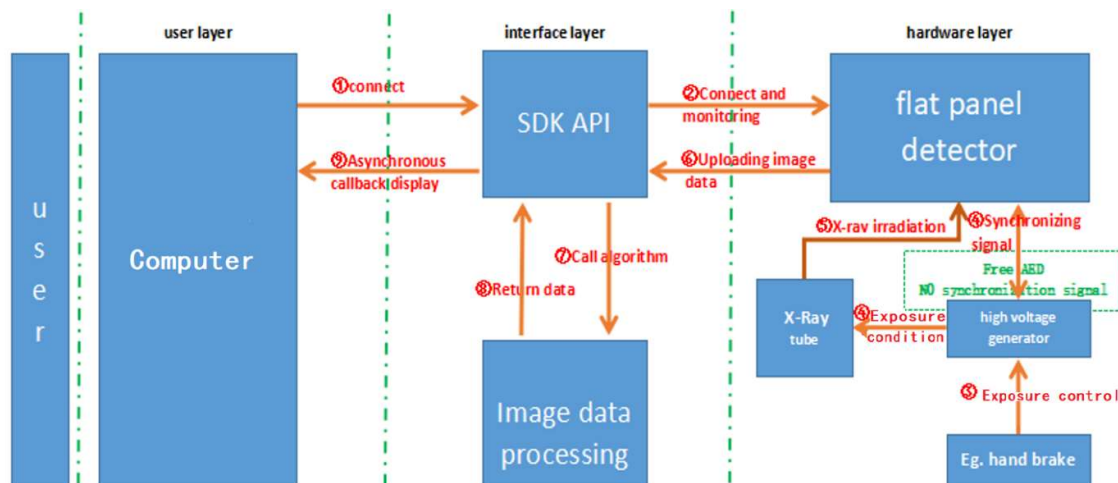


Figure 20

The image acquisition mode can be divided into two types, the static mode image acquisition and dynamic mode image acquisition.

1.Static Mode:

1. Software trigger mode
2. High voltage trigger mode
3. AED trigger mode

2.Dynamic Mode:

1. External trigger mode
2. External trigger / FPD master mode
3. Internal / SW trigger mode (software control).

2.2. Call Flow Chart

Main process flow chart:

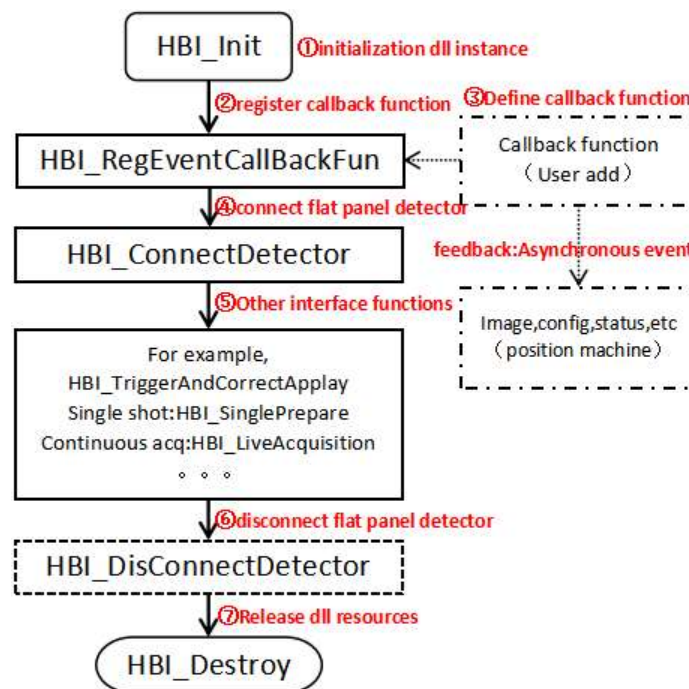


Figure 21

Main Process:

Start

Initialize'HBI_Init'

Register callback function 'HBI_RegEventCallBackFun'

Connect to detector 'HBI_ConnectDetector'

Set Trigger Mode and Enabling Relationship 'HBI_TriggerAndCorrectApply'

Send acquisition command (single frame acquisition 'HBI_SinglePrepare' or continuous acquisition 'HBI_LiveAcquisition')

Disconnect 'HBI_DisConnectDetector'

Release DLL resource 'HBI_Destroy'

End.

Attention: HBI_Destroy consists of two parts (disconnecting and releasing resources).

Users obtain flat panel status, images, configurations, and other data through callback functions.

Other Operational Flowchart:

Parameter settings:

HBI_Set * -> Update Parameters -> HBI_Get * -> Get the latest configuration

Single Frame Acquisition:

Method 1: Prepare delay time = 0;

HBI_Single Prepare -> HBI_SingleAcquisition -> Notification receiving thread -> Receive data single frame processing

Method 2: Prepare delay > 0;

HBI_SinglePrepare -> Notify receiving thread -> Receive data single frame processing

Note: The prepare delay time can be achieved through HBI_SetSinglePrepareTime API function. The prepare delay time will be used for X-ray exposure.

Multi Frame Acquisition:

HBI_LiveAcquisition -> Notify image receiving thread, config system parameters -> Receive data for multi frame processing.

The frame rate (or continuous acquisition interval time) can be achieved through 'HBI_SetLiveAcquisitionTime' modification function.

Note: There is a parameter which can be used for configuring the image acquisition frame number, when it is configured to 0, a stop command 'HBI_StopAcquisition' is needed to stop current continuous image acquisition. If the frame number parameter is configured greater than 0, after the configured number of frames are collected, the panel will automatically stop, no extra stop command is necessary.

Offset Template Generation:

Generate template 'HBI_GenerateTemplate' -> Update Calibration Enable 'HBI_UpdateCorrectEnable'.

Gain Template Generation:

Generate template 'HBI_GenerateTemplate' -> - Download Template 'HBI_DownloadTemplate' -> Update Calibration Enable 'HBI_UpdateCorrectEnable'.

Defect Template Generation:

Generate template 'HBI_GenerateTemplate' -> Download Template 'HBI_DownloadTemplate' -> Update Calibration Enable 'HBI_UpdateCorrectEnable'.

Note:

1. Before the panel connection is successful, any parameters set will not take effect because they cannot be synchronized with the firmware. For example, if a user calls the 'HBI_SetEnableCalibrate' interface before connection, it will not take effect.
2. When setting system parameters, note that the interface is asynchronous. After successfully sending the parameter configuration command, the firmware still needs a certain amount of time to write these parameters to its internal non-volatile memory, and then provides a feedback command to the SDK for acknowledgment. Therefore, it is better if the software waits for each command's acknowledgment or implements a delay for each parameter setting command. Otherwise, previously set parameters may be overwritten by new incoming parameters.
3. For parameters that do not need to be changed frequently, it is recommended to configure them during the initialization period and then use them directly. Generally, only 'HBI_TriggerAndCorrectDisplay' needs to be called to update the trigger mode and calibration enable options.
4. Similarities and differences in setting parameter feedback information

After a successful parameter configuration, there are two kinds of acknowledgments received through callback functions. Some acknowledgment commands will provide system configuration parameters, as seen in the ECALLBACK_TYPE_ROM_UPLOAD event, while other acknowledgment commands will confirm the operation, such as the ECALLBACK_TYPE_SET_CFG_OK event.

5. Communication methods, triggering modes, image calibration, and application differences

Current image calibration is divided into offset calibration, gain calibration, and defect calibration.

Table 1.

Type	Communication	Trigger Mode	Offset Calibration	Gain Calibration	Defect Calibration	Application
Static	00: UDP	01-static: software trigger mode; 03- static: hvg trigger mode;	01: software pre-offset; 02: firmware post-offset;	01: software gain;	01: software defect	Medical, Veterinary

		04- static: AED trigger mode; 08- static: SAEC trigger mode				
	03: UDP(Wireless)	01-static: software trigger mode; 03- static: hvg trigger mode; 04- static: AED trigger mode; 08- static: SAEC trigger mode;	01: software pre-offset; 02: firmware post-offset;	01: software gain;	01: software defect	Medical, Veterinary
Dynamic	01: UDP(Jumbo)	05-dynamic: hvg pulse sync in mode; 06- dynamic: fpd pulse sync out mode; 07- dynamic: free run mode;	03: firmware pre-offset;	02: firmware gain;	02: firmware defect;	Medical, Industry
	02: Optical Fiber (PCIe)	05-dynamic: hvg pulse sync in mode; 06- dynamic: fpd pulse sync out mode; 07- dynamic: free run mode;	03: firmware pre-offset;	02: firmware gain;	02: firmware defect;	Medical, Industry

a. Firmware calibration requires downloading the gain and defect templates before use. The gain and defect templates are always generated by software, and then downloaded to the firmware.

b. Calibration template generation. Currently, we support two methods to generate templates: using the DT tool software or by calling template generation API functions. Please refer to "5. Secondary development" for details.

c. The firmware pre-offset template can be generated by command.

(1) For flat panel detector connection function, set the last parameter of 'HBI_ConnectDetector' to 1. The firmware will automatically generate a pre-offset template during connection.

(2) Generate a template of the type 'OFFSET_TEMPLATE_TYPE' using the 'HBI_GenerateTemplate' function.

(3) For continuous acquisition, use the 'HBI_LiveAcquisition' function. Set acquisition parameters as follows: eAqccmd='LIVE_ACQ_DEFAULT_TYPE', eLivetype='ONLY_PREOFFSET' (only generate pre-offset template) or

eLivetype='PREOFFSET_IMAGE' (continuously capture images after generating pre-offset template). All other parameters are set to default. Static detectors generally use software calibration, while dynamic detectors use firmware calibration.

d. When using the Ethernet interface with jumbo packets, pay attention to the network card settings. (The NIC must support the Jumbo packet mechanism.)

6. Template file and log file directory

The current template and log files are automatically generated based on the dynamic library DLL path. Therefore, they are saved in the same directory as the DLL file, as shown in the picture below:

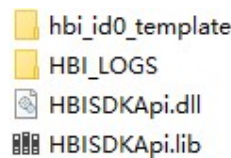


Figure 22

The root directory of the template for "panel A" is "hbi_id0_template", and vice versa, "panel B" is "hbi_id1_template".

2.3. Instance Handles and Panel Device ID

A handle is an identifier used to represent an object or resource, such as a form or file. Note that handles are not constant.

Handle is a term often used in C++ programming. It is not a specific, fixed, or unchanging data type or entity, but rather a broad concept in programming. Generally, a handle refers to a method for accessing another object—a generalized pointer, which could be an integer, an object, or an actual pointer. Its purpose is to establish a unique connection with the object being accessed.

In C++, you can typically use a pointer to access an object. However, in many specific applications, directly using pointers to represent objects is not always the best solution.

A handle is the identifier of an object or instance in the Windows system, which includes modules, application instances, windows, controls, bitmaps, GDI objects, resources, files, and more.

From a data type perspective, it is simply a 16-bit unsigned integer. Applications obtain a handle by calling the Windows API, which can then be used by other Windows functions to manipulate the corresponding kernel object. A handle can also be null, like a pointer, in which case it does not represent any valid kernel object.

"Instance handle and panel device ID" are unsigned pointers and unsigned integer variables that facilitate the management of SDK objects and support applications such as dual panels. They are important parameters for interface functions. For example, for a default single flat panel, handle1, ID 0; for dual flat panels, A panel is handle1, ID 0, and B panel is handle2, ID 1.

2.4. Callback Functions

A callback function is a function that is called through a function pointer. If you pass the pointer (address) of a function as a parameter to another function, and this pointer is used to call the function it points to, it is called

a callback function. The callback function is not called directly by the implementer of the function, but rather by another specific event or condition that occurs.

A callback is any method that is called by another method, with a method as its first parameter. Often, a callback is a method that is invoked when certain events occur.

To prevent UI blocking, some interfaces use asynchronous operations and register callback functions. There are generally two types of registration functions: global functions and static member functions.

```
Static int handleCommandEvent (void * _context, int nfpdId, unsigned char command, void * pvParam1, int nParam2, int nParam3, int nParam4);
```

```
Class CXXX{
```

```
.....
```

```
Public:
```

```
Static int handleCommandEvent (void * _context, int nfpdId, unsigned char command, void * pvParam1, int nParam2, int param3, int param4);
```

```
.....
```

```
};
```

Parameter Description:

```
// @ pContext:      parameter 1, object pointer.
```

```
// @ ufpdId:        parameter 2, for example, FPD ID.
```

```
// @ byteEventId:   parameter 3, event ID, refer to enum eCallbackEventCommType of "HbiType.h".
```

```
// @ pvParam1:      parameter 4, configuration structure pointer or image structure pointer
```

```
// @ nParam2:        parameter 5, for example, data size or status
```

```
// @ nParam3:        parameter 6, for example, frame id
```

```
// @ nParam4:        parameter 7, for example, frame rate or status
```

```
// callback function return value type: the return value is 1 or 0, 1: success, 0: failed
```

If there is no return value or the return value is 0, the SDK will take it as a failed feedback, and the SDK will record this information into the log. Therefore, if the computer receives any callback message from the SDK, please return 1.

For example:

```
int handleCommandEvent (void * _context, unsigned char command, void * pvParam1, int len, int nParam2, int param3, int param4)
```

```
{
.....

// At the end of the function

    Return 1 // Convention: Upon receiving any callback message (including success or failed), return 1,
    indicating that the computer has received it!

}
```

2.5. Trigger Mode

Trigger mode is a concept on panel collecting image, the current supported work modes are listed in the table below.

Table 2.

Type	Name	Remarks
0x01: STATIC_SOFTWARE_TRIGGER_MODE	Static Acquisition - Software trigger Mode	All operations on the panel are controlled by software commands, and there is no need to connect a high-voltage trigger cable between the flat panel detector and high voltage generator, only supporting single image acquisition under software trigger mode
0x02: STATIC_PREP_TRIGGER_MODE	Static acquisition - Clear mode	Unavailable
0x03: STATIC_HVG_TRIGGER_MODE	Static Acquisition - High voltage generator trigger mode	The external high voltage trigger cable needs to be connected between flat panel detector and trigger device (always high voltage generator), the image acquisition is controlled by external trigger pulse instead of software command, both the level or edge can be configured with sensitive trigger.
0x04: STATIC_AED_TRIGGER_MODE	Static Acquisition - AED trigger mode	All operations on the panel can be controlled by software commands, and there is no need to connect a high-voltage trigger cable between the flat panel detector and high voltage generator; The X-ray exposure and image acquisition are independent, as it has been designed a exposure signal detection mechanism inside of detector, once there is an exposure happens, the internal firmware can automatically detect this exposure action and generate a internal trigger to acquire one single image, the X-ray integration time is configurable.
0x05: DYNAMIC_HVG_TRIGGER_MODE	Dynamic Acquisition – External trigger mode	The external high voltage trigger cable needs to be connected between flat panel detector and trigger device (always high voltage generator), the image

		acquisition is triggered by external sync pulse, both the level or edge can be configured with sensitive trigger.
0x06: DYNAMIC_FPD_TRIGGER_MODE	Dynamic Acquisition – External trigger, FPD master mode	The external high voltage trigger cable needs to be connected between flat panel detector and trigger device (always high voltage generator), the image acquisition is controlled by software command, while the detector will output a exposure pulse to exposure device by each frame, both the level or edge can be configured with sensitive trigger.
0x07: DYNAMIC_FPD_Continue_MODE	Dynamic Acquisition – Internal/SW trigger mode	All operations on the panel are controlled by software commands, and there is no need to connect a high-voltage trigger cable between the flat panel detector and high voltage generator, only supporting single image acquisition under software trigger mode.
0x08: DYNAMIC_FPD_SAEAC_MODE	Static Acquisition - SAEAC mode	This SAEAC mode is designed for mammography application, a preview image will be acquired to calculate the next image's exposure condition, the next image will be used for terminal inspection.

2.6. Image Calibration

Calibration type: Currently supports three calibration methods: offset calibration, gain calibration, and defect calibration. These methods are part of image preprocessing.

Table 3.

Calibration type	Remarks
Offset Calibration - Offset	Eliminating physical offset pattern generated by physical hardware and leakage
Gain calibration - Gain	Flatten the image acquired under cone beam X-ray to a flatten image
Bad pixel calibration - Defect	Calibrate physical defects on image sensor

Calibration template :Currently, image calibration is performed by saving the calibration coefficients in the calibration template, then loading them into memory. The calibration algorithm uses these coefficients to calibrate the image.

Table 4.

Calibration type	Template (hbi_id0_template, where id0 represents panel A)
Offset Calibration - Offset	Offset_xx_x_xxxx. raw (frame rate, PGA, binning)
Gain calibration - Gain	Gaina.raw and gainb.raw (KV, PGA, binning, ROI, SID)

Bad pixel calibration - Defect	Defect. raw (Binning,ROI)
--------------------------------	---------------------------

Software or Firmware Calibration

- **Software Calibration:** When you enable software calibration, the SDK on your computer manages the entire image calibration process. This is ideal for situations where you're capturing single frames or working with a low frame rate. The SDK automatically processes the calibration once you enable it—no additional manual steps required.
- **Firmware Calibration:** With firmware calibration, image calibration is performed directly by the detector's firmware. Once firmware calibration is enabled, the firmware scans and processes the sensor data using built-in algorithms. The calibrated image is then sent to the SDK, which means less processing load on your computer and faster throughput. This method is best for high frame rate image acquisition.

Template Generation

- **Offset Template:** Generated from a set of dark (no light) images.
- **Gain Template:** Requires at least one set of bright images (uniform illumination, no occlusions).
- **Defect Template:** Requires at least three sets of bright images taken at different exposure doses (again, uniform images without occlusions).

You can use the DT tool's template wizard for distributed generation or integrate template generation functions into your own software. For more details, see the "Second Development" section.

Firmware Calibration and Template Handling

- **Offset Calibration Template:** For firmware calibration, the offset template (from dark images) is generated directly by the firmware, so there is no need to download it. However, it is recommended to update this template when the system first starts or after a power cycle.
- **Other Templates:** For high frame rate detectors, download the required calibration templates (gain, defect) to the firmware first. This ensures fast calibration because the firmware handles image correction in real time, minimizing delays caused by SDK processing.

Calibration Enable & Order

- **Calibration Enable:** When a valid calibration template is available, enabling calibration means the detector will automatically apply corrections to the images before sending them to your application. If calibration is disabled, you will receive the raw, uncorrected image.
- **Calibration Sequence:** The standard order is: Offset Calibration → Gain Calibration → Defect Calibration. Offset Calibration must be performed before Gain or Defect Calibration. Defect Calibration requires both Offset and Gain Calibration to be completed first (though you can skip Gain if only Offset is needed).

The following is the calibration enable status:

Table 5.

Calibration type	Calibration enable status
Offset calibration	0 - Disable offset calibration; 1 - Software Pre-offset calibration; 2 - Firmware post offset calibration; 3 - Frmware Pre-offset calibration;
Gain calibration	0 - Disable gain calibration; 1 - Software gain calibration; 2 - Firmware gain calibration;
Defect calibration	0 - Disable defect calibration; 1 - Software defect calibration; 2 - Firmware defect calibration;

2.7. Command

2.7.1. Request Command

Enum eRequestCommType

```

{
    EDL_COMMON_TYPE_INVALVE           = 0x00,
    EDL_COMMON_TYPE_GLOBAL_RESET      = 0x01,
    EDL_COMMON_TYPE_PREPARE           = 0x02,
    EDL_COMMON_TYPE_SINGLE_SHORT      = 0x03,
    EDL_COMMON_TYPE_LIVE_ACQ          = 0x04,
    EDL_COMMON_TYPE_STOP_ACQ          = 0x05,
    EDL_COMMON_TYPE_PACKET_MISS       = 0x06,
    EDL_COMMON_TYPE_FRAME_MISS        = 0x07,
    EDL_COMMON_TYPE_DUMMPING          = 0x08,
    EDL_COMMON_TYPE_END_RESPONSE      = 0x0A, // End response packet
    EDL_COMMON_TYPE_CONNECT_FPD       = 0x0B, // connect fpd

```

```

EDL_COMMON_TYPE_DOWNLOAD_IMAGE      = 0x0C, // wireless download image
EDL_COMMON_TYPE_SLEEP_STATE          = 0x0D, // wireless set sleep state
EDL_COMMON_TYPE_WAKE_UP               = 0x0E, // wireless wake up
EDL_COMMON_TYPE_DISCONNECT_FPD        = 0x0F, // disconnect fpd
EDL_COMMON_TYPE_SET_RAM_PARAM_CFG     = 0x10,
EDL_COMMON_TYPE_GET_RAM_PARAM_CFG     = 0x11,
EDL_COMMON_TYPE_SET_ROM_PARAM_CFG     = 0x12,
EDL_COMMON_TYPE_GET_ROM_PARAM_CFG     = 0x13,
EDL_COMMON_TYPE_SET_FACTORY_PARAM_CFG = 0x14,
EDL_COMMON_TYPE_GET_FACTORY_PARAM_CFG = 0x15,
EDL_COMMON_TYPE_RESET_FIRM_PARAM_CFG  = 0x16,
EDL_COMMON_UPLOAD_OFFSET_TEMPLATE     = 0x2E,
EDL_COMMON_UPLOAD_GAIN_TEMPLATE        = 0x2F, // Upload gain template
EDL_COMMON_UPLOAD_DEFECT_TEMPLATE      = 0x30, // Upload defect template
EDL_COMMON_DEFECT_AUTHOR               = 0x31, //
EDL_COMMON_ERASE_FIRMWARE              = 0x4F, // Erase old firmware package request
EDL_COMMON_UPDATE_FIRMWARE            = 0x50, // Update new firmware package request
EDL_COMMON_TYPE_SET_AQC_MODE           = 0xFF

};

```

2.7.2. Response Command

Enum eCallbackEventCommType

```

{
    ECALLBACK_TYPE_INVOLVE      = 0X00,
    ECALLBACK_TYPE_COMM_RIGHT   = 0X01,
    ECALLBACK_TYPE_COMM_WRONG   = 0X02,
    ECALLBACK_TYPE_DUMMPLING    = 0X03,
    ECALLBACK_TYPE_ACQ_END       = 0X04,
    ECALLBACK_TYPE_UPDATE_FIRMWARE = 0X06, // Update old firmware package ack
}

```

```

ECALLBACK_TYPE_ERASE_FIRMWARE      = 0X07, // Update new firmware package ack
ECALLBACK_TYPE_FPD_STATUS           = 0X09, // Status package
ECALLBACK_TYPE_ROM_UPLOAD           = 0X10,
ECALLBACK_TYPE_RAM_UPLOAD           = 0X11,
ECALLBACK_TYPE_FACTORY_UPLOAD       = 0X12,
ECALLBACK_TYPE_SINGLE_IMAGE         = 0X51,
ECALLBACK_TYPE_MULTIPLE_IMAGE       = 0X52,
ECALLBACK_TYPE_PREVIEW_IMAGE        = 0X53,
ECALLBACK_TYPE_PACKET_MISS          = 0X5B,
ECALLBACK_TYPE_OFFSET_TMP           = 0X5C,
ECALLBACK_OVERLAY_NUMBER            = 0X5D,
ECALLBACK_OVERLAY_16BIT_IMAGE       = 0X5E,
ECALLBACK_OVERLAY_32BIT_IMAGE       = 0X5F,
ECALLBACK_TYPE_LIVE_ACQ_OK          = 0XA0,
ECALLBACK_TYPE_PACKET_MISS_MSG      = 0XA4,
ECALLBACK_TYPE_THREAD_EVENT         = 0XA5,
ECALLBACK_TYPE_ACQ_DISCARDED        = 0XA6,
ECALLBACK_TYPE_OFFSET_ERR_MSG       = 0XA7,
ECALLBACK_TYPE_GAIN_ERR_MSG         = 0XA8,
ECALLBACK_TYPE_DEFECT_ERR_MSG       = 0XA9,
ECALLBACK_TYPE_FPD_STATUS           = 0XAA,
ECALLBACK_TYPE_SET_CFG_OK           = 0XAB, //Setting parameters successful
ECALLBACK_TYPE_SAVE_SUCCESS         = 0XAC, //Image saved successfully
ECALLBACK_TYPE_GENERATE_TEMPLATE    = 0XAD, //Generate template
ECALLBACK_TYPE_FILE_NOTEXIST        = 0XAE, //The file does not exist
ECALLBACK_TYPE_WORK_STATUS          = 0XAF, //Working status (production mode or test mode)

};

Refer to <HbiType.h>.

```

2.8. Data Structure

2.8.1. Connection Parameters

Common connection function interfaces.

Typedef struct CommCfgSt

```
{
    CommCfgSt()
    {
        _Type=FPD_COMM_TYPE: PCIE_COMM_TYPE;
        _Remoteip [0]='\ 0';
        _Localip [0]  ='\ 0';
        _LoacalPort  =32896;
        _RemotePort =32897;
        _pciId      = 7024; // default id
    }
    FPD_COMM_TYPE_Type;
    // The following parts should only be set with ethernet interface, and PCIe interface just needs to set
    the type and the device ID of PCIe.
    char _Remoteip [MAX_IP_LEN];
    char _ Localip [MAX_IP_LEN];
    unsigned short _LoacalPort;
    unsigned short _RemotePort;
    unsigned short _pciId; // PCIe device ID
}COMM_CFG;
```

2.8.2. Image Properties

Typedef struct ImageProperty

```
{
    ImageProperty() {
        Nwidth = IMAGE_WIDTH;
```



```

    Nheight = IMAGE_HEIGHT;

    Datatype = 0;

    Ndatabit = 0;

    Nendian = 0;

}

int nwidth;    // Image width.

int nheight;   // Image height.

int datatype;  // Data type: 0-unsigned char, 1-char, 2-unsigned short, 3-float, 4-double.

int ndatabit;  // Data bit: 0-16bit, 1-14bit, 2-12bit, 3-8bit.

int nendian;   // Endian: 0-little endian, 1-big endian.

}IMAGE_PROPERTY;

```

2.8.3. Trigger Mode Classification

Commonly used in setting trigger mode interfaces.

Typedef enum

```

{

    INVALID_TRIGGER_MODE           = 0x00, // Reserved.

    STATIC_SOFTWARE_TRIGGER_MODE   = 0x01, // Static: Software trigger mode.

    STATIC_PREP_TRIGGER_MODE       = 0x02, // Static: Reserved.

    STATIC_HVG_TRIGGER_MODE        = 0x03, // Static: High voltage generator trigger mode.

    STATIC_AED_TRIGGER_MODE        = 0x04, // Static: AED trigger mode.

    DYNAMIC_HVG_TRIGGER_MODE       = 0x05, // Dynamic: High voltage generator pulse sync in
mode.

    DYNAMIC_FPD_TRIGGER_MODE       = 0x06, // Dynamic: Flat panel detector pulse sync out mode.

    DYNAMIC_FPD_CONTINUE_MODE      = 0x07, // Dynamic: Free run mode (Software trigger mode).

    DYNAMIC_FPD_SAEC_MODE          = 0x08 // Static: SAEC Mode

}EnumTRIGGER_MODE;

```

2.8.4. Continuous Collection and Classification

//Notice: Live Acquisition 'properties

Typedef enum

```
{
    PREOFFSET_IMAGE           = 0x01,      // Preoffset template and image.
    ONLY_IMAGE                 = 0x02,      // Only image.
    ONLY_PREOFFSET             = 0x03,      // Only Preoffset template.
    OVERLAY_16BIT_IMG          = 0x04,      // Overlay result is 16bit image.
    OVERLAY_32BIT_IMG          = 0x05      // Overlay result is 32bit image.
}EnumLIVE_ACQUISITION;
```

2.8.5. Collection Mode Types

//Notice: acq mode: static and dynamic

Typedef enum

```
{
    SINGLE_ACQ_DEFAULT_TYPE = 0x00,      // Single acquisition.
    LIVE_ACQ_DEFAULT_TYPE,                // Live acquisition.
    // Multi step generation method
    LIVE_ACQ_OFFSET_IMAGE,                // Generate offset template with dynamic live acquisition.
    SINGLE_ACQ_GAIN_IMAGE,                // Generate gain template with static single acquisition.
    LIVE_ACQ_GAIN_IMAGE,                 // Generate gain template with dynamic live acquisition.
    SINGLE_ACQ_DEFECT_IMAGE,              // Generate defect template with static single acquisition.
    LIVE_ACQ_DEFECT_IMAGE,                // Generate defect template with dynamic live acquisition.
    // One-key quick generation mode
    OFFSET_TEMPLATE_TYPE,                 // Quick generation process of offset template.
    GAIN_TEMPLATE_TYPE,                   // Quick generation process of gain template- group1
    DEFECT_TEMPLATE_GROUP1,               // Quick generation process of defect template - group1.
    DEFECT_TEMPLATE_GROUP2,               // Quick generation process of defect template - group2.
    DEFECT_TEMPLATE_GROUP3,               // Quick generation process of defect template - group3.
    SOFTWARE_OFFSET_TEMPLATE,              // Quickly generate template collection type, continuously
    collect a set of dark field images - SDK generates offset template
    .....
}
```

```
}EnumIMAGE_ACQ_CMD;
```

2.8.6. Collecting Image Parameters

Including single frame acquisition and continuous acquisition, commonly used in single frame or continuous acquisition interfaces.

```
//Notice: fpd aqc mode
```

```
//Notice: fpd aqc mode
```

```
typedef struct fpd_aqc_mode_st {  
    fpd_aqc_mode_st(){  
        eAqctype      = SINGLE_ACQ_DEFAULT_TYPE;// Default single image acquisition.  
        eLivetype      = ONLY_IMAGE;           // Only image.  
        ngroupro       = 0;  
        nAcqnumber     = 0;  
        ndiscard       = 0;  
        nframeid       = 0;  
    };  
    Enum IMAGE_ACQ_CMD eAqccmd;    // Acquisition command.  
    Enum LIVE_ACQUISITION eLivetype; // Live acquisition only,  
                                     // detail refer to LIVE_ACQ_DEFAULT_TYPE.  
                                     // 1-generate the firmware Preoffset template and then  
                                     // continue to acquire image.  
                                     // 2 - image acquisition only.  
                                     // 3 - generate firmware Preoffset template only.  
    int ngroupro;                  // Group number.  
    int nAcqnumber;                // Reserved image number.  
    int ndiscard;                  // Discard image number.  
    int nframeid;                  // Counter.  
}FPD_AQC_MODE;
```

2.8.7. Correction Enable setting parameters

Commonly used in setting correction enable state functions.

//Notice: fpd software calibrate enable info

```
typedef struct image_correct_enable_st {  
    image_correct_enable_st(){  
        bFeedbackCfg          = false;  
        ucOffsetCorrection     = 0;  
        ucGainCorrection       = 0;  
        ucDefectCorrection     = 0;  
        ucDummyCorrection      = 0;  
    };  
    bool bFeedbackCfg;        // true - ECALLBACK_TYPE_ROM_UPLOAD Event.  
                             // false - ECALLBACK_TYPE_SET_CFG_OK Event.  
    unsigned char ucOffsetCorrection;    // 00 - Reserved.  
                                         // 01 - Software Preoffset Correction.  
                                         // 02 - Firmware Postoffset Correction.  
                                         // 03 - Firmware Preoffset Correction.  
    unsigned char ucGainCorrection;      // 00 - Reserved.  
                                         // 01 - Software Gain Correction.  
                                         // 02 - Hardware Gain Correction.  
    unsigned char ucDefectCorrection;    // 00 - Reserved.  
                                         // 01 - Software Defect Correction.  
                                         // 02 - Software Defect Correction.  
    unsigned char ucDummyCorrection;    // 00 - Reserved.  
                                         // 01 - Software Dummy Correction.  
                                         // 02 - Software Dummy Correction.  
}IMAGE_CORRECT_ENABLE;
```

2.8.8. Download Template File Parameters

Commonly seen in template download interface functions.

// Template file type

typedef enum

{

 OFFSET_TMP = 0x00,

 GAIN_TMP = 0x01,

 DEFECT_TMP = 0x02

}emUPLOAD_FILE_TYPE;

// Template file attributes

typedef struct hbi_file_attr_st

{

 hbi_file_attr_st()

 {

 nModelID = 0;

 emfiletype = GAIN_TMP;

 nBinningtype = -1;

 filepath[0] = '\0';

 };

 int nModelID;

 emUPLOAD_FILE_TYPE emfiletype;

 int nBinningtype;

 char filepath[MAX_PATH];

}DOWNLOAD_FILE;

Note: Compared to previous versions, a new parameter nModelID has been added (only effective in multi-mode mode) to indicate the location of firmware template storage.

In order to better compatibility with multi mode download templates, some interfaces and data types have been added as follows:

 Template type:

//Notice: template file type

typedef enum

```
{
    OFFSET_TEMPLATE=0x00,
    GAIN_TEMPLATE    =0x01,
    DEFECT_TEMPLATE=0x02
}EmpTEMPLATE_TYPE;

Download template dependency parameters:

//Template file attributes
typedef struct download_mode_file_st
{
    download_mode_file_st()
    {
        m_byModelId      = 0;
        m_byBinningType   = 1;
        m_byPgaLevel      = 6;
        m_unFpsTime       = 0;
        m_usKvValue       = 0;
        m_sZoomRectLeft   = 0;
        m_sZoomRectTop    = 0;
        m_sZoomRectWidth  = 0;
        m_sZoomRectHeight = 0;
        emtemplatetype    = GAIN_TEMPLATE;
    };
    unsigned char  m_byModelId;    // Mode ID
    unsigned char  m_byBinningType;// Binning type
    unsigned char  m_byPgaLevel;
    unsigned int   m_unFpsTime;
    unsigned short m_usKvValue;
    unsigned short m_sZoomRectLeft;
```

```

unsigned short m_sZoomRectTop;

unsigned short m_sZoomRectWidth;

unsigned short m_sZoomRectHeight;

emTEMPLATE_TYPE emtemplatetype;

}DOWNLOAD_MODE_FILE;

```

2.8.9. Mode Parameter

Each Mode item contains parameters such as binning type, frame rate time, PGA level, KV energy level, zoom area parameters, trigger mode, and template status. Among these, binning type, frame rate time, PGA level, KV energy level, and zoom area parameters are related to template generation and template download.

//Notice: fpd mode data information: 22

```

typedef struct fpd_mode_data_st
{
    // Register_1

    unsigned char m_byBinningType;//01//Binning Type,1-binning1x1,2-binning2x2,3-
    binning3x3,binning4-4x4,

    // Register_2~5

    unsigned int m_unFpsTime; //04 //Fps Time,unit: ms

    // Register_6

    unsigned char m_byPgaLevel; //01 //Pga Level

    // Register_7~Register_8

    unsigned short m_usKvValue; //02 //Dose Kv Value

    // Register_9~Register_10

    unsigned short m_sZoomRectLeft; //02 //Zoom Rect Left

    // Register_11~Register_12

    unsigned short m_sZoomRectTop; //02 //Zoom Rect Top

    // Register_13~Register_14

    unsigned short m_sZoomRectWidth; //02 //Zoom Rect Width

    // Register_15~Register_16

    unsigned short m_sZoomRectHeight; //02 //Zoom Rect Height

```

```

// Register_17

unsigned char m_byFpdTrigger; //01 //Fpd Trigger Mode

// Register_18~21

unsigned int m_unPrepareTime; //04 //Prepare Time,unit: ms

// Register_22

unsigned char m_byTempStatus; //01 //0-Both dark and bright field templates are invalid, 1-Only
dark field templates are valid

// 2-Only the bright field template is valid, 3-Both dark and bright field templates are valid, others-Both
dark and bright field templates are invalid

}FPD_MODE_DATA;

```

2.8.10. Multiple Mode List

The multi mode list contains the current mode ID and an array of mode items, as follows:

```

//User Acquisition Mode Parameter Configuration: 221

typedef struct fpd_mode_list_st
{
    // Register_1

    unsigned char m_byCurModelId; //01 //Current Mode Id,[0~10]

    // Register_2~Register_221

    FPD_MODE_DATA m_stModeArray[MODE_LIST_TOTAL]; //220 //Current Mode DATA Array

}FPD_MODE_LIST;

```

2.8.11. Other Parameters

Please refer to header file 'HbiType.h'.

2.9. Flat Panel Status

According to the panel communication method, it can be divided into static, dynamic, Ethernet and optical fiber interface, as shown in the callback function event ECALLBACK_ TYPE_ FPD_ STATUS.

Callback function: handleCommandEventA (void * pvContext, int nDevId, unsigned char byteEvtid, void * pvParam1, int nParam2, int nParam3, int nParam4)

2.9.1. Static Flat Panel Detector Status of Ethernet Interface

Table 6.

	State type	Remarks
static flat panel detector	0: FPD_DISCONN_STATUS	In the initial state, if there is an abnormal disconnection message, the following situations are considered abnormal and the connection is disconnected. NParam2=0: Network not connected NParam2=-1: parameter exception NParam2=-2: Failed to return the number of prepared descriptors NParam2=-3: Receive timeout NParam2=-4: Receive failed NParam2=-5: Port unreadable NParam2=-6: Abnormal monitoring network card NParam2=-7: Monitoring network card is normal (tested, actual feedback is not provided) NParam2=-8: Firmware update completed, disconnected NParam2=-9: Optical port communication, fiber optic disconnection NParam2=-10: Optical port communication, failed to read DDR address, restarting the PCIe driver can solve the problem. NParam2=-11: The flat panel is a jumbo frame, which means non jumbo frame communication is used for communication, and the receiving area overflows.
	100: FPD_CONN_STATUS	Connection successful
	1: FPD_PREPARE_STATUS	Prepare status, clear
	2: FPD_READY_STATUS	Ready state, flat panel in idle waiting state
	4: FPD_EXPOSE_STATUS	Busy status, currently collecting images
	9: FPD_UPDATE_FIRMARE	Updating firmware
	10: FPD_RETRANS_MISS	Feedback on packet loss information, including the total number of packets
	11: FPD_STATUS_AED	AED status, in the AED acquisition chart

2.9.2. Dynamic Flat Panel Detector Status of Network Port

Table 7.

	State type	Remarks
dynamic flat panel detector	0: FPD_DISCONN_STATUS	In the initial process, if there is an abnormal disconnection message, the following information are provided for reference. NParam2 = 0: Network not connected NParam2 = -1: parameter exception NParam2 = -2: Failed to return the number of prepared descriptors NParam2 = -3: Receive timeout NParam2 = -4: Receive failed NParam2 = -5: Port unreadable NParam2 = -6: Abnormal monitoring network card NParam2 = -7: Monitoring network card is normal (tested, actual feedback is not provided) NParam2 = -8: Firmware update completed, disconnected NParam2 = -9: Optical port communication, fiber optical disconnection NParam2 = -10: Optical port communication, failed to read DDR address, restarting the PCIe driver can solve the problem. NParam2 = -11: The panel is a Jumbo frame, which means non Jumbo frame communication is used for communication, and the receiving area overflows.
	100: FPD_STATUS	Connection successful
	1: FPD_Prepate_STATUS	Prepare status, clear
	2: FPD_READY_STATUS	Ready state, panel in idle waiting state
	3: FPD_DOOFFSET_TEMPLATE	Busy status, currently collecting images
	4: FPD_EXPOSE_STATUS	Updating firmware
	5: FPD_CONTINUTE_READY	Feedback on packet loss information, including the total number of packets
	6: FPD_DWONLOAD_GAIN	AED status, in the AED acquisition process

	7:FPD_DWONLOAD_DEFECT	In the initial process, if there is an abnormal disconnection message, the following information are provided for reference. NParam2 = 0: Network not connected NParam2 = -1: parameter exception NParam2 = -2: Failed to return the number of prepared descriptors NParam2 = -3: Receive timeout NParam2 = -4: Receive failed NParam2 = -5: Port unreadable NParam2 = -6: Abnormal monitoring network card NParam2 = -7: Monitoring network card is normal (tested, actual feedback is not provided) NParam2 = -8: Firmware update completed, disconnected NParam2 = -9: Optical port communication, fiber optical disconnection NParam2 = -10: Optical port communication, failed to read DDR address, restarting the PCIe driver can solve the problem. NParam2 = -11: The panel is a Jumbo frame, which means non Jumbo frame communication is used for communication, and the receiving area overflows.
	8:FPD_DWONLOAD_OFFSET	Connection successful
	9: FPD_UPDATE_FIRMARE	Prepare status, clear
	10: FPD_RETRANS_MISS	Ready state, panel in idle waiting state
	11: FPD_STATUS_AED	Busy status, currently collecting images

2.9.3. Dynamic Flat Panel Status of Optical Fiber Interface

Table 8.

	State type	Remarks
Optical dynamic flat detector	0: FPD_DISCONN_STATUS	In the initial state, if there is an abnormal disconnection message, the following situations are considered abnormal and the connection is disconnected.

		NParam2=0: Network not connected NParam2=-1: parameter exception NParam2=-2: Failed to return the number of prepared descriptors NParam2=-3: Receive timeout NParam2=-4: Receive failed NParam2=-5: Port unreadable NParam2=-6: Abnormal monitoring network card NParam2=-7: Monitoring network card is normal (tested, actual feedback is not provided) NParam2=-8: Firmware update completed, disconnected NParam2=-9: Optical fiber interface communication, fiber optic disconnection NParam2=-10: Optical fiber interface communication, failed to read DDR address, restarting the PCIe driver can solve the problem. NParam2=-11: The panel is a jumbo frame, which means non jumbo frame communication is used for communication, and the receiving area overflows.
	100: FPD_CONN STATUS	Connection successful
	1: FPD_PREPARE_STATUS	Prepare status, clear
	2: FPD_READY_STATUS	Single frame acquisition completed, ready, flat panel in idle waiting state
	3:FPD_DOOFFSET_TEMPLATE	In the firmware generation pre offset template
	4: FPD_EXPOSE_STATUS	Busy status, currently collecting images
	5: FPD_CONTINUTE_READY	Continuous collection completed, ready, flat panel in idle waiting state
	6: FPD_DWONLOAD_GAIN	Download gain template
	7:FPD_DWONLOAD_DEFECT	Download defect template
	8:FPD_DWONLOAD_OFFSET	Download offset template
	9: FPD_UPDATE_FIRMARE	Updating firmware

	10: FPD_RETRANS_MISS	Feedback on packet loss information, including the total number of packets
	11: FPD_STATUS_AED	AED status, in the AED acquisition chart

2.9.4. Wireless Static Flat Panel Status

Table 9.

	State type	Remarks
Wireless static flat detector	0: FPD_DISCONN_STATUS	if there is an abnormal disconnection message, the following information are provided for reference. NParam2 = 0: Network not connected NParam2 = -1: parameter exception NParam2 = -2: Failed to return the number of prepared descriptors NParam2 = -3: Receive timeout NParam2 = -4: Receive failed NParam2 = -5: Port unreadable NParam2 = -6: Abnormal monitoring network card NParam2 = -7: Monitoring network card is normal (tested, actual feedback is not provided) NParam2 = -8: Firmware update completed, disconnected NParam2 = -9: Optical port communication, fiber optic disconnection NParam2 = -10: Optical port communication, failed to read DDR address, restarting the PCIe driver can solve the problem. NParam2 = -11: The panel is a Jumbo frame, which means non Jumbo frame communication is used for communication, and the receiving area overflows.
	100: FPD_CONN_STATUS	Connection successful
	1: FPD_PREPARE_STATUS	Prepare status, clear
	2: FPD_READY_STATUS	Ready state, panel in idle waiting state
	4: FPD_EXPOSE_STATUS	Busy status, currently collecting images

	9: FPD_UPDATE_FIRMWARE	Updating firmware
	10: FPD_RETRANS_MISS	Feedback on packet loss information, including the total number of packets
	11: FPD_STATUS_AED	AED status, in the AED acquisition chart
	12: FPD_STATUS_SLEEP	Wireless: panel sleep state
	13: FPD_STATUS_WAKEUP	Wireless: panel wake-up status
	14: FPD_DOWNLOAD_NO_IMAGE	Download EMMC storage image: no image
	15: FPD_DOWNLOAD_TAIL_IMAGE	Download the last frame of the EMMC image
	16: FPD_EMMC_MAX_NUMBER	Emmc maximum storage frame rate
	18: FPD_WLAN_BATTERY_STATUS	Wireless battery feedback information

3. API Function Description

3.1. Function Types

The basic data types used in the API are as follows:

Table 10.

void	No parameters	nothing
void*	unsigned pointer	4 bytes
int	signed integer	4 bytes
short	signed short integer	2 bytes
WORD	unsigned short	2 bytes
char	character type	1 byte
BOOL	signed integer (int)	4 bytes
float	single precision floating-point type	4 bytes
unsigned int	unsigned int	4 bytes
unsigned short	unsigned short	2 bytes
int *	Integer pointer	4 bytes
char*	character pointer	4 bytes
unsigned char	unsigned character type	1 byte
CodeStringTable	Custom Type: Error Information Table	12 bytes
RegCfgInfo	Custom Type: Firmware Parameter Object	1024 bytes
ImgCaliCfg	Custom type: flat panel image calibration parameters	20 bytes
SysCfgInfo	Custom Type: Flat panel System Parameters	128 bytes
USER_CALLBACK_HANDLE_PROCESS	Custom Type: Callback Function Pointer Load State	4 bytes
USER_CALLBACK_HANDLE_ENVENT	Custom type: callback function pointer	4 bytes
SOFTWARE_CALIBRATE_ENABLE	Custom Type: Software Calibration Enable	4 bytes
IMAGE_PROPERTY	Custom Type: Image Properties	32 bytes
IMAGE_CORRECT_ENABLE	Custom Type: Image Calibration Enable Parameters	5 bytes

FPD_AQC_MODE	Custom Type: Collection Mode Parameters	36 bytes
DOWNLOAD_T_FILE	Custom Type: Download Template Parameters	264 bytes
CALIBRATE_INPUT_PARAM	Custom Type: Calibration Generate Model Parameters	36 bytes
NULL	null pointer	0

3.2. Function Classification

Mainly targeted for fast integration users, this article focuses on introducing several functional interfaces and categorizing them according to their needs (detailed in section 3.3 Functions), as follows:

3.2.1. Initialize and Release Resources

After initialization, the current instance handle can be obtained, which will allocate some resources to the interface. The resource release command will destroy the allocated resources. If not destroyed, it will cause memory leakage and other exceptions.

Single Panel Interface

Table 11.

No001	HBI_Init	Initialize dynamic library
No002	HBI_Destroy	Release dynamic library resources

Dual Panel Interface

Table 12.

No054	HBI_InitDual	Dual panels: initializing dynamic library instances (new)
No055	HBI_InitEx	Dual panels: Initialize dual panel or multi panels' dynamic library instances (new)
No056	HBI_DestroyEx	Dual panels: All DLL resources (new)

3.2.2. Registering Callback Functions

Currently, the SDK interface provides two callback functions: the main callback function and the callback function for download progress, results, and status. The main callback function must be registered, and after initialization, it should be bound to the handle for registration. Once the callback function is registered, you can use the connection function, set parameters, and execute image acquisition commands.

Table 13.

No009	HBI_RegEventCallBackFun	Registering callback functions - image, configuration, status
-------	-------------------------	---

No050	HBI_RegProgressCallBack	Register callback function, download template or firmware, provide progress and results feedback
-------	-------------------------	--

3.2.3. Connecting and Disconnecting Panel

Table 14.

No003	HBI_ConnectDetector	Establish a connection (supports Ethernet and optical ports)
No004	HBI_ConnectDetectorUdp	New: Ethernet UDP connection
No005	HBI_ConnectDetectorJumbo	New: Ethernet UDP Jumbo packet connection
No006	HBI_ConnectDetectorPcie	New: Optical port connection (PCIe card)
No007	HBI_ConnectDetectorEx	New: Connection interface, do not include parameters, require configuration file. See Appendix 2 "hbi_fpd_init.ini"
No008	HBI_DisConnectDetector	Disconnect

3.2.4. Trigger Mode and Calibration Enable Settings

Please refer to the interface function description below for instructions on how to call them. There are some combination function interfaces below that can set multiple parameters.

Table 15.

No026	HBI_TriggerAndCorrectDisplay	Quick initialization parameters (trigger mode and image correction enable)
No027	HBI_UpdateTriggerMode	Set trigger mode
No028	HBI_UpdateCorrectEnable	Set firmware image calibration enable
No029	HBI_GetCorrectEnable	Obtain firmware image correction enable
No030	HBI_SetSoftwareCorrect	Set software image calibration enable
No031	HBI_GetSoftwareCorrect	Obtain software correction enable
No067	HBI_TriggerPgaBinningCorrect	Set trigger mode, PGA, binning mode, and correction enable
No068	HBI_TriggerPgaBinningAcqTimeCorrect	Set trigger mode, PGA gear, binning mode, frame rate (image acquisition time interval), correction enable

3.2.5. Single Frame Acquisition and Continuous Acquisition

Single frame acquisition command (default): SINGLE_ACQ_DEFAULT_TYPE

Continuous acquisition command (default): LIVE_ACQ_DEFAULT_TYPE.

Table 16.

No018	HBI_SinglePrepare	Preparation for single frame acquisition
No019	HBI_SingleAcquisition	Single frame acquisition
No020	HBI_LiveAcquisition	Continuous acquisition
No021	HBI_StoppAcquisition	Stop continuous acquisition
No022	HBI_SetSinglePrepareTime	Set the interval time between preparation and single frame acquisition command
No023	HBI_GetSinglePrepareTime	Get the interval time between preparation and single frame acquisition command
No024	HBI_SetLiveAcquisitionTime	Set continuous acquisition interval time (static or wireless panel)
No025	HBI_GetLiveAcquisitionTime	Get continuous acquisition interval time (static or wireless panel)
No065	HBI_GetSelfDumpingTime	Set the acquisition interval time (Jumbo frame or optical interface dynamic panel)
No066	HBI_SetSelfDumpingTime	Get acquisition interval time (Jumbo frame or optical interface dynamic panel)
No085	HBI_Prepare (new)	Single frame acquisition preparation, optimized from HBI_Single Prepare command

3.2.6. Frame Rate, PGA, and Binning Setting

Frame rate : The frame rate can be configured by setting the continuous acquisition interval time. For static panels, use HBI_SetLiveAcquisitionTime; for dynamic panels, use HBI_SetSelfDumpingTime. The parameter unit is milliseconds.

PGA : The gain sensitivity of the detector can be divided into 1–7 stages for A-Si and IGZO panels, and 8–9 stages for CMOS panels. The lower the gain, the higher the sensitivity, and vice versa. Binning type: supports binning 1x1, 2x2, 3x3, and 4x4.

Table 17.

No024	HBI_SetLiveAcquisitionTime	Set continuous acquisition interval time (static or wireless panel)
No066	HBI_SetSelfDumpingTime	Obtain acquisition interval time (Jumbo frame or optical dynamic panel)
No042	HBI_SetPGAlevel	Set gain mode
No043	HBI_GetPGAlevel	Get gain mode
No044	HBI_SetBinning	Set binning method
No045	HBI_GetBinning	Get binning method
No067	HBI_TriggerPgaBinningCorrect	Set trigger mode, PGA, binning mode, and correction enable
No068	HBI_TriggerPgaBinningAcqTimeCorrect	Set trigger mode, PGA, binning mode, frame rate (image acquisition interval time), correction enable

3.2.7. Single Frame Acquisition Mode

A single frame image can be acquired using soft trigger mode, high voltage trigger mode, or AED trigger mode.

The soft trigger mode is divided into Prepare + Delay mode and Prepare + Single Shot mode.

Prepare + Delay mode: If the prepare delay time is greater than 0, the user only needs to call the "SinglePrepare" function on the detector. The firmware will wait for the specified prepare delay time, which is used for exposure, and then automatically acquire a single image.

Prepare + Single Shot mode: When the prepare delay time is equal to 0, the user needs to call the "SinglePrepare" function first, wait for a specific time for exposure, and then call the "SinglePrepare" function again to acquire a single frame image.

The prepare delay time parameter can be set using the "HBI_SetSinglePrepareTime" function.

The "SingleAcquisition" command parameters is as follows:

Single frame acquisition command:

Single_ACQ_DEFAULT_TYPE=0x00, // default single frame acquisition

Template generation - Single frame acquisition command

Single_ACQ_GAIN_IMAGE, //Create Gain template - Single frame acquisition of bright field image

Single_ACQ_DEFECT_IMAGE, //Create Defect template - Single frame acquisition of bright field image

3.2.8. Continuous Acquisition Mode

The most commonly used function is HBI_LiveAcquisition. It is used for live image acquisition, firmware pre-offset template generation, image acquisition and overlap, as well as template generation image data acquisition (distributed template generation).

Continuous acquisition command (default): LIVE_ACQ_DEFAULT_TYPE

Typedef struct FPD_AQC_MODE_St

{

EnumIMAGE_ACQ_CMD eAqccmd // Continuous acquisition command (default):
LIVE_ACQ_DEFAULT_TYPE

EnumLIVE_ACQUISITION eLivetype // Two images above; 3- Firmware only creates offset templates; 4-
Overlay: Upload a 16 bit average image; 5- Overlay: Upload 32-bit average image;

int ngroupro // Group number

int nAcqnumber // Frame rate

int ndiscard // Discard frames

int nframeid // Counter

}FPD_AQC_MODE;

Here are several common usages:

1. Firmware generates Pre-offset templates and overlay images

FPD_AQC_MODE.eLivetype = PREOFFSET_IMAGE // Update firmware pre-offset template and continuous image acquisition (trigger mode 07-Dynamic: Internal / SW trigger)

FPD_AQC_MODE.eLivetype = ONLY_IMAGE // Continuous image acquisition

FPD_AQC_MODE.eLivetype = ONLY_PREOFFSET // Generate pre-offset template, then stop live image acquisition (trigger mode 07-Dynamic: Internal / SW trigger)

FPD_AQC_MODE.eLivetype = Overlay_16BIT_IMG // Overlay 16-bit image

FPD_AQC_MODE.eLivetype = Overlay_32BIT_IMG // Overlay 32-bit images

2. Specify the frame rate for acquisition and discard frames

For live image acquisition, the number of discarded and valid images is configurable. The user can set how many images will be discarded and how many will be kept. This function greatly enhances terminal usage flexibility.

FPD_AQC_MODE.nAcqnumber // Frame number

FPD_AQC_MODE.niscard // The number of frames discarded before image acquisition

3. Template generation - live acquisition command

LIVE_ACQ_OFFSET_IMAGE // Create Offset Template - continuously acquire dark images

LIVE_ACQ_GAIN_IMAGE // Create Gain template - continuously acquire bright images

LIVE_ACQ_DEFECT_IMAGE // Create Defect template - continuously acquire bright images

The template data files can be generated and saved to the default path.

4. The following commands cannot be used as HBI_Parameters of LiveAcquisition

OFFSET_TEMPLATE_TYPE // Quick template generation process: continuously acquire a set of dark field images and generate offset templates; firmware generates templates

GAIN_TEMPLATE_TYPE // Quick template generation process: continuously acquire a set of bright field images and generate gain templates

DEFECT_TEMPLATE_GROUP1 // Quick template generation process: continuously acquire a set of bright field images - defect group 1

DEFECT_TEMPLATE_GROUP2 // Quick template generation process: continuously acquire a set of bright field images - defect group 2

DEFECT_TEMPLATE_GROUP3 // Quick template generation process: continuously acquire a set of bright field images - defect group 3 and generate template

Software_OFFSET_TEMPLATE // Quick template generation process: continuously acquire a set of dark field images and generate offset template

Note: These command types can only be used as parameters for the interface function 'HBI_GenerateTemplate'. They cannot be used as parameters for the interface function 'HBI_LiveAcquisition'. HBI_GenerateTemplate is used to quickly generate templates, and the interface includes the process of sending commands and generating templates.

3.2.9. Quick Template Generation and Template Download

Template generation and template downloads: only the interfaces are listed here. A dedicated chapter below demonstrates how to use them.

Table 18.

No050	HBI_RegProgressCallBack	Register callback function, download template to firmware, feedback progress and results
No051	HBI_GenerateTemplate	Quickly generate templates, including offset, gain, and defect
No052	HBI_DownloadTemplate	To download the correction template, you need to input the file type, directory, and other parameters
No053	HBI_StopDownloadTemplate	Stop downloading correction templates

No086	HBI_DownloadTemplateByType	Download correction template files by default according to type (parameter automatic matching), it is recommended to replace HBI_DownloadTemplate.
-------	----------------------------	--

3.2.10. Image Attributes, Software and Firmware Versions

Table 19.

No012	HBI_GetFPDSerailNumber	Obtain detector serial number
No013	HBI_GetSDKVersion	Get SDK DLL version number
No014	HBI_GetFirmareVerion	Obtain firmware version number
No015	HBI_GetError	Obtain error information through error codes
No016	HBI_GetFpdCfgInfo	Obtain firmware ROM parameters
No017	HBI_GetImageProperty	Get Image Properties

3.2.11. Parameters and Restoring Factory Settings

Table 20.

No016	HBI_GetFpdCfgInfo	Get firmware ROM parameters
No011	HBI_ResetFirmware	Reset firmware factory settings

3.2.12. Zoom (ROI) Area Settings

Table 21.

No070	HBI_SetRoiMode	The ROI region setting function
No071	HBI_SetZoomModeRect	The CMOS ZOOM Mode region setting function, currently only supports panel types including CMOS-1412-50um and CMOS-0606-50um.
No080	HBI_GetCurZoomRect	Get the current ZOOM region size data, where the left, top, width, and height in Rect are 0, indicating the entire image.

3.2.13. Calibration Template Path

Table 22.

No072	HBI_SetCorrectFileDirectory	Setting calibration files directory, usually using the default path, and user can set another directory as required
No073	HBI_GetCorrectFileDirectory	Get the user calibration file home directory, usually using the default path, and user can set another directory as required
No081	HBI_IsExistsOfCurCorrectTemplate	Identify whether the current template file exists based on the template type (new)
No082	HBI_GetCurrentCorrectFileName	Get the current template file name (new)
	HBI_GetCurTempFileFullDirectory	Get the full directory of the current template file based on the template type (new)
No084	HBI_GetCurrentCorrectFilePath	Get the absolute path of the current template file based on the template type (new)

3.2.14. Image Display and Saving

Table 23.

No074	HBI_UpdateSaveOption	Update saving image options
-------	----------------------	-----------------------------

3.2.15. Packet Interval

Table 24.

No075	HBI_SetPacketIntervalTime	Set image packet interval gap time
No076	HBI_AutoWindowWidthWindowLevel	Automatic window width and window level adjustment, converts 16 bit data into 8-bit unsigned data through window width and window level.

3.2.16. Image Rotation/Mirroring

Table 25.

No077	HBI_SetImageRotateAndFlip	Rotation (90°) or mirror enable setting
No078	HBI_Rotate90AndFlip	Image rotation and mirroring processing (only applicable to 16 bit unsigned image data, default first rotate and then mirror) (new)
No079	HBI_ResetCorrectTemplate	Reset calibration template file - rotate, mirror reverse processing and save, usually completed before downloading to firmware (new)

3.2.17. Firmware Template Management Method

The firmware manages templates in two different ways. The binning-based management and multi-template management.

Table 26.

No180	HBI_FirmwareManagementTempalteType	Firmware template management template methods: Binning storage and multi mode storage. This interface can switch between two methods.
-------	------------------------------------	---

How can these two methods be managed to prevent incompatibility with previous versions of binning? This can be achieved by classifying parameters. Additionally, different interface functions should be used for each method to ensure compatibility with both binning-based management and multi-template management.

The newly added interface functions correspond to the binning-based management interface functions (see table below). In addition to setting parameters, a new function will automatically synchronize the parameters for the current mode item. In summary, when integrating multiple modes, it is recommended to use 'HBI_M *' as much as possible to set parameters for the interface function. **The multi-template management mode is strongly recommended due to its comprehensive functionality.**

Table 27.

Number	Binning version interface function	Multiple Mode Version Interface Functions	Remarks
1	HBI_UpdateTriggerMode	HBI_MUpdateTrigger	Set trigger mode
2	HBI_SetBinning	HBI_MSetBinning	Set image binning mode
3	HBI_SetPGALevel	HBI_MSetPGALevel	Set gain stage
4	HBI_SetRoiMode	HBI_MSetRoiMode	The ROI region setting
5	HBI_SetZoomModeRect	HBI_MSetZoomModeRect	The CMOS ZOOM Mode region setting function currently only supports CMOS-1412-50um, CMOS-0606-50um, and 1917-120um.
6	HBI_SetSelfDumpingTime	HBI_MSetSelfDumpingTime	Set image acquisition interval time (dynamic)
7	HBI_SetLiveAcquisitionTime	HBI_MSetLiveAcquisitionTime	Set continuous acquisition interval time (static)
8	HBI_SetFpsOfTime	HBI_MSetFpsOfTime	Set continuous frame acquisition interval time
9	HBI_PgaAcqTime	HBI_MPgaAcqTime	Set PGA and frame rate (interval time for image acquisition)

10	HBI_PgaBinningAcqTime	HBI_MPgaBinningAcqTime	Set PGA, binning mode, and frame rate (image acquisition interval time)
11	HBI_TriggerAndCorrectDisplay	HBI_MTriggerAndCorrectDisplay	Set trigger mode and enable image calibration
12	HBI_TriggerBinningAcqTime	HBI_MTriggerBinningAcqTime	Set trigger mode, binning, and frame rate (interval time for image acquisition)
13	HBI_TriggerPgaBinningAcqTime	HBI_MTriggerPgaBinningAcqTime	Set trigger mode, PGA, binning mode, and frame rate (image acquisition interval time)
14	HBI_TriggerPgaBinningAcqTimeCorrect	HBI_MTriggerPgaBinningAcqTimeCorrect	Set trigger mode, PGA, binning, and frame rate (image acquisition interval time), Image correction enable.

3.2.18. API Functions under Multi-template Management Mode

The following interfaces are only applicable to editing multi-templates management mode lists, switching mode items, saving mode list information, refining mode list item data, updating templates for specified mode items, and template status information for mode items.

Table 28.

No102	HBI_SetValidModelItemNumber	Set the number of valid mode items
No103	HBI_IsMutilMode	Check if the current firmware supports multiple modes
No.087	HBI_GetCurModeList	Get mode List
No088	HBI_UpdateModelList	Update mode List
No089	HBI_GetCurModeData	Get the current mode Data
No090	HBI_UpdateCurModeData	Update current mode Data
No091	HBI_UpdateModeData	Update mode Item Data through mode ID
No119	HBI_GetModeData	Obtain mode Data through mode ID
No120	HBI_GetModeListSize	Get the number of items in mode
No092	HBI_GetCurModeID	Get the current mode ID
No093	HBI_SwitchUserMode	Switch mode option
No121	HBI_ModeExists	Identify whether the mode item exists

No122	HBI_CheckCurtMode	Identify if the mode is normal
No094	HBI_ModeCorrectTempalteEnable	The calibration template of the current mode effectiveness
No095	HBI_AllModeCorrectTempalteEnable	All calibration templates effectiveness
No096	HBI_SaveModeList2File	Save the current user mode list
No097	HBI_DownloadModeTemplate	Download template based on mode parameter
No098	HBI_DownloadModeTemplateByType	Download correction template files according to template type (default to current mode Id)
No099	HBI_DownloadModeTemplateByIdAndType	Download correction template files according to mode Id and template type
No100	HBI_GetCurModeTemplateStatus	Get the firmware calibration template status for the current mode item

3.3. Interface Functions

Note:

```
#Ifdef_ DLL_ EX_ IM
#define HBI_FPD_DLL__ Declspec (dllexport)
#else
#define HBI_FPD_DLL__ Declspec (dllimport)
#endif
#define_ DLL_ EX_ IM 0//0-import function, 1-export function, default to 0 (user default).
```

3.3.1. HBI_Init

```
/******
* No: No001
* Function Name: HBI_Init
* Function Description: Initialize dynamic dll, nonvolatile parameters and thread message parameters,
establish SDK object pointer.
* Parameter Description:
                int fpdid                // Flat panel detector ID, default is 0.
* Return Value: void *
```

failed: NULL, Success: Non empty, Instance pointer

* Comment:

*****/

HBI_FPD_DLL void * HBI_Init (int fpdid=0);

3.3.2. HBI_Destroy

*****/

* No: No002

* Function Name: HBI_Destroy

* Function Description: Release dynamic library resources

* Parameter Description:

In: void * handle -unsigned handle pointer, dynamic library FPD SDK Object object pointer

Out: None

* Return Value: void

* Comment:

*****/

HBI_FPD_DLL void HBI_Destroy (void * handle);

3.3.3. HBI_ConnectDetector

*****/

* No: No003

* Function Name: HBI_ConnectDetector

* Function Description: Establish connection

* Parameter Description:

In: void *handle // unsigned handle pointer

COMM_CFG commCfg // communication tyep, refer to <HbiType.h>

UDP_COMM_TYPE = 0, //Standard UDP communication method

UDP_JUMBO_COMM_TYPE=1, //Standard UDP communication method, Support for jumbo

PCIE_COMM_TYPE = 2, //Fiber type

WALN_COMM_TYPE = 3, //Wireless type

```
int doOffsetTemp    // 1: don't update pre-offset template when connection is successful
                    // Non-1: update pre-offset template when connection is successful
```

Out: None

* Return Value: int

0- Successful

Non 0- Failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int HBI_ConnectDetector (void * handle, COMM.CFG commCfg, int doOffsetTemp=0);

Below are several new connection functions that users can choose based on their actual needs.

3.3.4. HBI_ConnectDetectorUdp

/*****

* No: No004

* Function Name: HBI_ConnectDetectorUdp

* Function Description: Standard UDP communication method, establish connection (gigabit Ethernet interface with typical UDP packet)

* Parameter Description:

In:	void *handle	// unsigned handle pointer
	char *szDetectorIp	// FPD IP address: 192.168.10.40
	unsigned short usDetectorPort	// FPD port: 32897(0x8081)
	char *szlocalIp	// PC IP address: 192.168.10.20
	unsigned short usLocalPort	// PC port: 32896(0x8080)
	int doOffsetTemp	// 1: don't update pre-offset template when connection is successful
		// Non-1: update pre-offset template when connection is successful

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

```
HBI_FPD_DLL int __stdcall HBI_ConnectDetectorUdp (void * handle, char * szDetectorIp, unsigned short usDetectorPort, char * szlocalIp, unsigned short usLocalPort, int doOffsetTemp=0);
```

3.3.5. HBI_ConnectDetectorJumbo

*****/

* No: No005

* Function Name: HBI_ConnectDetectorJumbo

* Function Description: Establish connection (gigabit Ethernet interface with typical UDP packet, Support for jumbo)

* Parameter Description:

In:	void *handle	// unsigned handle pointer
	char *szDetectorIp	// FPD IP address: 192.168.10.40
	unsigned short usDetectorPort	// FPD port: 32897(0x8081)
	char *szlocalIp	// PC IP address: 192.168.10.20
	unsigned short usLocalPort	// PC port: 32896(0x8080)
	int doOffsetTemp	// 1: don't update pre-offset template when connection is successful // Non-1: update pre-offset template when connection is successful

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

```
HBI_FPD_DLL int __stdcall HBI_ConnectDetectorJumbo (void * handle, char * szDetectorIp, unsigned short usDetectorPort, char * szlocalIp, unsigned short usLocalPort, int doOffsetTemp=0);
```

3.3.6. HBI_ConnectDetectorPcie

*****/

* No: No006

* Function Name: HBI_ConnectDetectorPcie

* Function Description: // Establish connection (optical fiber interface)

* Parameter Description:

In: void *handle // unsigned handle pointer

 int doOffsetTemp // 1: don't update pre-offset template when connection is successful

 // Non-1: update pre-offset template when connection is successful

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_ConnectDetectorPcie (void * handle, int doOffsetTemperature=0);

3.3.7. HBI_ConnectDetectorEx

/*****

* No: No007

* Function Name: HBI_ConnectDetectorEx

* Function Description: Establish connection

* Parameter Description:

In: void * handle - // unsigned handle pointer

Basic communication control parameters can be found in HBI_FPD_CONFIG.ini

The file directory is the same as the DLL.

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:hbi_fpd_init.ini

- * FPD IP Address: FPD_IP = 192.168.10.40
- * FPD Port: FPD_PORT = 32897
- * Local IP Address: LOCAL_IP = 192.168.10.20
- * Local Port: LOCAL_PORT = 32896
- * TRIGGER_MODE: 0-Invalid Trigger;
 - 1: Static: Software Trigger;
 - 2: Clear;
 - 3: Static: Hvg Trigger Mode;
 - 4: Static: AED Trigger Mode;
 - 5: Dynamic: Hvg Sync Mode;
 - 6: Dynamic: Fpd Sync Mode;
 - 7: Dynamic: Free Run Mode;
 - 8: Static: SAEC Mode;
- * OFFSET_ENABEL: 0: No Offset Correction;
 - 1: Software Preoffset Correction;
 - 2: Firmware Postoffset Correction;
 - 3: Firmware Preoffset Correction;
- * GAIN_ENABLE: 0: No Gain Correction;
 - 1: Software Gain Correction;
 - 2: Firmware Gain Correction;
- * DEFECT_ENABLE: 0: No Defect Correction;
 - 1: Software Defect Correction;
 - 2: Firmware Defect Correction;
- * DUMMY_ENABLE: 0: No Dummy Correction;
 - 1: Software Dummy Correction;
 - 2: Firmware Dummy Correction;
- * LIVE_ACQ_TIME: 1000~1000ms
- * SELF_DUMPING_TIME: 500~500ms

* PREPARE_TIME: 1500~1500ms

* DO_OFFSET: 0: No action

1: Do offset template

*****/

HBI_FPD_DLL int __stdcall HBI_ConnectDetectorEx (void * handle);

3.3.8. HBI_DisConnectDetector

/******

* No: No08

* Function Name: HBI_DisConnectDetector

* Function Description: Disconnect detector

* Parameter Description:

In: void * handle - // unsigned handle pointer

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int HBI_DisconnectDetector (void * handle);

3.3.9. HBI_RegEventCallBackFun

/******

* No: No09

* Function Name: HBI_RegEventCallBackFun

* Function Description: Register callback function

* Parameter Description:

In: void * handle - unsigned handle pointer

USER_Callback_HANDLE_ENVENT - Register callback function

Void * pContext - Object pointer, application software transfer object pointer to SDK in order to access static and global variable

Out: None

* Return Value: int

0- Successful

Non 0- failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int HBI_RegEventCallBackFun (void * handle,
USER_CALLBACK_HANDLE_ENVENT callback, void * pContext);

3.3.10. HBI_ResetDetector

*****/

* No: No10

* Function Name: HBI_ResetDetector

* Function Description: Restart the detector

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: None

* Return Value: int

0- Successful

Non 0- failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int HBI_ResetDetector (void * handle);

3.3.11. HBI_ResetFirmware

*****/

* No: No11

* Function Name: HBI_ResetFirmware

* Function Description: Restore factory setting

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: None

* Return Value: int

0- Successful

Non 0- failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_ResetFirmware(void * handle);

3.3.12. HBI_GetFPDSerialNumber

*****/

* No: No12

* Function Name: HBI_GetFPDSerialNumber

* Function Description: Get the version of DLL

* Parameter Description:

In: void * handle - unsigned handle pointer

In/Out: char * szSn, length 14 bits, version information string for DLL

* Return Value: int

0- Successful

Non 0- failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetFPDSerialNumber(void *handle, char *szSn);

3.3.13. HBI_GetSDKVersion

*****/

* No: No13

* Function Name: HBI_GetSDKVersion

* Function Description: Get the version of DLL

* Parameter Description:

In: void * handle - unsigned handle pointer

char * szVer - Version information string

Out: None

* Return Value: int

0- Successful

Non 0- failed ,refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetSDKVerion (char * szVer);

3.3.14. HBI_GetFirmwareVerion

*****/

* No: No14

* Function Name: HBI_GetFirmwareVerion

* Function Description: Get the version of firmware

* Parameter Description:

In: void * handle - unsigned handle pointer

char * szVer - Version information string

Out: None

* Return Value: int

0- Successful

Non 0- failed ,refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetFirmwareVerion (void * handle, char * szFirmwareVer);

3.3.15. HBI_GetError

*****/

* No: No015

* Function Name: HBI_GetError

* Function Description: Get error information, not support yet

* Parameter Description:

Const CodeStringTable * inTable - Error message list

int count -Number of information lists

int recode - Error code

Out: None

* Return Value: const char *

Non NULL - Success, error message

NULL - Failed

* Comment:

*****/

HBI_FPD_DLL const char * __stdcall HBI_GetError (const CodeStringTable * inTable, int count, int recode);

3.3.16. HBI_GetFpdCfgInfo

/*****/

* No: No016

* Function Name: HBI_GetFpdCfgInfo

* Function Description: Get the configuration of firmware

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: RegCfgInfo * pRegCfg, see HbType. h.

* Return Value: int

0- Successful

Non 0- failed ,refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetFpdCfgInfo (void * handle, RegCfgInfo * pRegCfg);

3.3.17. HBI_GetImageProperty

```

/*****
* No: No17
* Function Name: HBI_GetImageProperty
* Function Description: get image property
* Parameter Description:
    In: void * handle - unsigned handle pointer
    IMAGE_PROPERTY *img_pro // Image property, refer to<HbiError.h> IMAGE_PROPERTY
    Out: None
* Return Value: int
    0- Successful
    Non 0- failed ,refer to <HbiError.h>
* Comment:
*****/
HBI_FPD_DLL int __stdcall HBI_GetImageProperty (void * handle, IMAGE_PROPERTY * img_pro);

```

3.3.18. HBI_SinglePrepare

```

/*****
* No: No18
* Function Name: HBI_SinglePrepare
* Function Description: Single image acquisition(combine with "prepare delay time" parameter)
* Parameter Description:
    In: void * handle - unsigned handle pointer
    Out: None
* Return Value: int
    0- Successful
    Non 0- failed ,refer to <HbiError.h>
* Comment:In the application, there will be a problem that the acquisition command type cannot be
synchronized, resulting in the failure of the above figure. Use HBI_Prepere replacement

```

*Prepare delay=0 - indicates soft trigger single frame acquisition first HBI_Prepere after HBI_SingleAcquisition completes single frame acquisition

*Prepare delay > "0 - means soft trigger single frame acquisition, only HBI_Prepere is required can complete single frame acquisition according to the delay time

*****/

HBI_FPD_DLL int HBI_SinglePrepare (void * handle);

3.3.19. HBI_SingleAcquisition

/******

* No: No19

* Function Name: HBI_SingleAcquisition

* Function Description: Single image acquisition

* Parameter Description:

In: void *handle // unsigned handle pointer

FPD_AQC_MODE _mode // Acquisition mode and related parameters

EnumIMAGE_ACQ_MODE // Image acquisition mode

SINGLE_ACQ_DEFAULT_TYPE, // Single frame acquisition

SINGLE_ACQ_GAIN_IMAGE, // Distributed generation correction

template for template validation, creating Gain template - single frame acquisition of bright field images

SINGLE_ACQ_DEFECT_IMAGE, // Distributed generation of correction

templates for template validation and creation of Defect templates - single frame acquisition of bright field images

EnumLIVE_ACQUISITION eLivetype; // Only for ONLY_IMAGE

int ngroupro // Group number

int nframesum // Acquisition image number

int ndiscard // Discard image number

int nframeid // Frame ID number

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int HBI_SingleAcquisition (void * handle, FPD_AQC_MODE _mode);

3.3.20. HBI_LiveAcquisition

/******

* No: No20

* Function Name: HBI_LiveAcquisition

* Function Description: Live acquisition

* Parameter Description:

In: void *handle // unsigned handle pointer

FPD_AQC_MODE _mode // Work mode and related parameter

The structure contains fields:

EnumIMAGE_ACQ_CMD eAqccmd; // Acquisition command, Refer to the document "HbiType.h", the default parameter is LIVE_ACQ_DEFAULT_TYPE, pay attention to single-frame acquisition and continuous acquisition commands

LIVE_ACQ_DEFAULT_TYPE, // Acquisition command

// Distribute and generate correction templates for template validation

LIVE_ACQ_OFFSET_IMAGE, // Generate Offset template - continuous acquisition of dark field image

LIVE_ACQ_GAIN_IMAGE, // Generate Gain template - continuous acquisition of bright field image

LIVE_ACQ_DEFECT_IMAGE, // Generate Defect template - continuous acquisition of bright field image

// Quickly generate correction templates for system integration

OFFSET_TEMPLATE_TYPE, // Quickly template generation, continuously acquire a group of dark field images and generate offset templates, and firmware generates templates

GAIN_TEMPLATE_TYPE, // Quickly template generation, continuously acquire a group of bright field images and generate gain templates

DEFECT_TEMPLATE_GROUP1, // Quickly template generation and continuously acquire a group of bright field images - defect group1

DEFECT_TEMPLATE_GROUP2, // Quickly template generation and continuously acquire a group of bright field images - defect group2

DEFECT_TEMPLATE_GROUP3, // Quickly template generation and continuously acquire a group of bright field images - defect group3 and generate template

SOFTWARE_OFFSET_TEMPLATE // Quickly template generation and continuously acquire a group of dark field images - generates offset template

Enum LIVE_ACQUISITION eLivetype; // Live acquisition only, detail refer to LIVE_ACQ_DEFAULT_TYPE.

// 1 - generate the firmware Preoffset template and then continue to acquire image.

// 2 - image acquisition only.

// 3 - generate firmware Preoffset template only.

// 4 - Overlay: Upload 16-bit Averaged Image.

// 5 - Overlay: Upload 32-bit Averaged Image.

int ngroupno; // Group number.

int nAcqnumber; // Reserved image number.

int ndiscard; // Discard image number.

int nframeid; // Counter.

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int HBI_LiveAcquisition (void * handle, FPD_AQC_MODE _mode);

3.3.21. HBI_StopAcquisition

/*****

* No: No21

* Function Name: StopAcquisition

* Function Description: Stop live acquisition

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: None

* Return Value: int

0- Successful

Non 0- failed, refer to <HbiError.h>

* Note:

*****/

HBI_FPD_DLL int HBI_StopAcquisition (void * handle);

3.3.22. HBI_SetSinglePrepareTime

*****/

* No: No22

* Function Name: HBI_SetSinglePrepareTime

* Function Description: set single frame acquisition delay

* Parameter Description:

In: void * handle - unsigned handle pointer

int * in_ltime - delay Time ,Unit: ms

0: "HBI_Prepare" and "HBI_SingleAcquisition" command are combined to realize single image acquisition

Non-0: FPD will delay "Prepare Delay Time" and automatically acquire one single image.

* Return Value: int

0- Successful

Non 0- failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetSinglePrepareTime (void * handle, int in_ltime);

3.3.23. HBI_GetSinglePrepareTime

*****/

* No: No23

* Function Name: HBI_GetSinglePrepareTime

* Function Description: get single frame acquisition delay

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: int *out_ftime - delay Time ,Unit: ms

0: "HBI_Prepare" and "HBI_SingleAcquisition" command are combined to realize single image acquisition

Non-0: FPD will delay "Prepare Delay Time" and automatically acquire one single image.

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetSinglePrepareTime (void * handle, int * out_ftime);

3.3.24. HBI_SetLiveAcquisitionTime

*****/

* No: No24

* Function Name: HBI_SetLiveAcquisitionTime

* Function Description: Live acquisition span time setting (apply to static and wireless type FPD)

* Parameter Description:

In: void * handle - unsigned handle pointer

int time - Live acquisition span time, unit: ms

Out: None

* Return Value: int

0- Successful

Non 0- failed ,refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetLiveAcquisitionTime (void * handle, int time);

3.3.25. HBI_GetLiveAcquisitionTime

*****/

* No: No25

* Function Name: HBI_GetLiveAcquisitionTime

* Function Description: Get live acquisition span time (apply to static and wireless type FPD)

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: int *out_time // Live acquisition span time, unit: ms

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetLiveAcquisitionTime(void * handle, int * out_time);

3.3.26. HBI_TriggerAndCorrectDisplay

/*****

* No: No26

* Function Name: HBI_TriggerAndCorrectApplay

* Function Description: trigger mode and image calibration setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int _triggerMode

// 1: Static: Software trigger mode

// 3: Static: High voltage generator trigger mode

// 4: Static: AED trigger mode

// 5: Dynamic: External trigger mode;

// 6: Dynamic: External trigger / FPD master mode;

// 7: Dynamic: Internal / SW trigger mode;

// 8-SAEC.

IMAGE_CORRECT_ENABLE* pCorrect // refer to < HbiType.h>

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment: Replace "HBI_WorkStationInitDllCfg" and "HBI_QuickInitDllCfg" API function

*****/

HBI_FPD_DLL int __stdcall HBI_TriggerAndCorrectDisplay (void * handle, int triggerMode, IMAGE_CORRECT_ENABLE * pCorrect);

3.3.27. HBI_UpdateTriggerMode

/*****/

* No: No27

* Function Name: HBI_UpdateTriggerMode

* Function Description: trigger mode setting

In: void *handle // unsigned handle pointer
int _triggerMode,

// 1: Static: Software trigger mode

// 3: Static: High voltage generator trigger mode

// 4: Static: AED trigger mode

// 5: Dynamic: External trigger mode;

// 6: Dynamic: External trigger / FPD master mode;

// 7: Dynamic: Internal / SW trigger mode;

// 8-SAEC.

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_UpdateTriggerMode (void * handle, int triggerMode);

3.3.28. HBI_UpdateCorrectEnable

/******

* No: No28

* Function Name: HBI_UpdateCorrectEnable

* Function Description: update the setting of firmware calibration enable

* Parameter Description:

In: void *handle // unsigned handle pointer

IMAGE_CORRECT_ENABLE* pCorrect // refer to < HbiType.h>

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_UpdateCorrectEnable (void * handle, IMAGE_CORRECT_ENABLE * pCorrect);

3.3.29. HBI_GetCorrectEnable

/******

* No: No29

* Function Name: HBI_GetCorrectEnable

* Function Description: // Get image calibration enable status

* Parameter Description:

In: void *handle // unsigned handle pointer

IMAGE_CORRECT_ENABLE* pCorrect // refer to < HbiType.h>

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetCorrectEnable (void * handle, IMAGE_CORRECT_ENABLE * pCorrect);

3.3.30. HBI_SetSoftwareCorrect

/******

* No: No30

* Function Name: HBI_SetSoftwareCorrect

* Function Description: Software calibration enable setting

* Parameter Description:

In: void *handle // unsigned handle pointer

SOFTWARE_CALIBRATE_ENABLE inEnable // refer to<< HbiType.h>>
SOFTWARE_CALIBRATE_ENABLE structure

Out: None

* Return Value: int

0 successful

Non 0 failed, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetSoftwareCorrect (void * handle,
SOFTWARE_CALIBRATE_ENABLE inEnable);

3.3.31. HBI_GetSoftwareCorrect

/******

* No:No31

* Function Name: HBI_GetSoftwareCorrect

* Function Description: // Get software calibration enable status

* Parameter Description:

In: void *handle // unsigned handle pointer

SOFTWARE_CALIBRATE_ENABLE inEnable // refer to<HbiError.h>

SOFTWARE_CALIBRATE_ENABLE struct

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment: new interface

*****/

HBI_FPD_DLL int __stdcall HBI_GetSoftwareCorrect (void * handle,
SOFTWARE_CALIBRATE_ENABLE * inEnable);

3.3.32. HBI_SetAedIntegrateTm

/*****

* No: No32

* Function Name: HBI_SetAedIntegrateTm

* Function Description: AED integration time setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int time // AED integration time, unit: ms, default = 500ms

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetAedIntegrateTm (void * handle, int time);

3.3.33. HBI_GetAedIntegrateTm

/*****

* No: No33

* Function Name: HBI_GetAedIntegrateTm

* Function Description: Get AED integration time

* Parameter Description:

In: void *handle // unsigned handle pointer

Out: int *out_time // AED integration time, unit: ms

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetAedIntegrateTm (void * handle, int * out_time);

3.3.34. HBI_SetAedThreshold

*****/

* No: No34

* Function Name: HBI_SetAedThreshold

* Function Description: AED threshold value setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int out_ivalue // AED threshold (10,000~1,000,000)

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetAedThreshold (void * handle, int in_ivalue);

3.3.35. HBI_GetAedThreshold

*****/

* No: No35

* Function Name: HBI_GetAedThreshold

* Function Description: Obtain AED threshold

* Parameter Description:

In: void * handle -unsigned handle pointer (unsigned pointer)

Out: int * out_ lvalue - threshold [10000~1000000]

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetAedThreshold (void * handle, int * out_ lvalue);

3.3.36. HBI_SetsaturationValue

*****/

* No: No36

* Function Name: HBI_SetSaturationValue

* Function Description: Saturation value setting

* Parameter Description:

In: void *handle // unsigned handle pointer

Out:int *out_ lvalue // Saturation value (0 ~ 65,535)

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetsaturationValue (void * handle, int in_ lvalue);

3.3.37. HBI_GetsaturationValue

*****/

* No: No37

* Function Name: HBI_GetSaturationValue

* Function Description: Get image saturation value

* Parameter Description:

In: void * handle -unsigned handle pointer (

Out: int * out_ivalue - saturation value [0~65535]

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetsaturationValue (void * handle, int * out_ivalue);

3.3.38. HBI_SetClippingValue

*****/

* No: No38

* Function Name: HBI_SetClippingValue

* Function Description: Clipping value setting

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: int * out_ivalue - clipping value [0~65535]

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetClippingValue (void * handle, int in_ivalue);

3.3.39. HBI_GetClippingValue

*****/

* No: No39

* Function Name: HBI_GetClippingValue

* Function Description: Get clipping value

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: int * out_ivalue - clipping value [0~65535]

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetClippingValue (void * handle, int * out_ivalue);

3.3.40. HBI_SetLPFLevel

*****/

* No: No40

* Function Name: HBI_SetLPFLevel

* Function Description: Low pass filter setting

* Parameter Description:

In: void * handle -unsigned handle pointer (unsigned pointer)

int nLPFLevel - Gear

[n] - Invalid

[0] -210kHz, default to 210kHz

[1] -105khz

[2] -52khz

[3] -26khz

[4] -13khz

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetLPFLevel (void * handle, int nLPFLevel);

3.3.41. HBI_GetLPFLevel

/******

* No: No41

* Function Name: HBI_GetLPFLevel

* Function Description: Get pass filter

* Parameter Description:

In: void * handle -unsigned handle pointer (

Out: None

* Return Value: int

[0] -210kHz, default to 210kHz

[1] -105khz

[2] -52khz

[3] -26khz

[4] -13khz

[n] - Invalid

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetLPFLevel (void * handle);

3.3.42. HBI_SetPGAlevel

/******

* No: No042

* Function Name: HBI_SetPGAlevel

* Function Description: PGA gain stage setting

* Parameter Description:

In: void * handle - unsigned handle pointer

int mode - Mode

[n] - Invalid

[1] -0.6pC

[2] -1.2pC

[3] -2.4pC

[4] -3.6pC

[5] -4.8pC

[6] -7.2pC, default to 7.2pC

[8] - COMS flat panel, LFW

[9] - COMS flat panel, HFW

[10] -0.3pC

[11] -0.15pC

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetPGAlevel (void * handle, int nGainLevel);

3.3.43. HBI_GetPGAlevel

/*****

* No: No43

* Function Name: HBI_GetPGAlevel

* Function Description: Get PGA gain stage setting value

* Parameter Description:

In: void * handle - unsigned handle pointer

Out: None

* Return Value: int

- [n] - Invalid
- [1] -0.6pC
- [2] -1.2pC
- [3] -2.4pC
- [4] -3.6pC
- [5] -4.8pC
- [6] -7.2pC
- [7] -9.6pC
- [8] - COMS flat panel, LFW
- [9] - COMS flat panel, HFW
- [10] -0.3pC
- [11] -0.15pC

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetPGAlevel (void * handle);

3.3.44. HBI_SetBinning

* No: No44

* Function Name: HBI_SetBinning

* Function Description: Binning type setting

* Parameter Description:

In:

Void * handle -unsigned handle pointer

unsigned int nbin // Binning type

// 1: 1x1

// 2: 2x2

// 3: 3x3

// 4: 4x4

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetBinning(void * handle, unsigned int nbin);

3.3.45. HBI_SetBinning

/******

* No: No45

* Function Name: HBI_GetBinning

* Function Description: Get binning type

* Parameter Description:

In: void * handle -unsigned handle pointer

unsigned int *nbin // Binning mode

// 1: 1x1

// 2: 2x2

// 3: 3x3

// 4: 4x4

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetPGALevel (void * handle);

3.3.46. HBI_SetPreviewMode

/******

* No: No46

* Function Name: HBI_GetPreviewMode

* Function Description: Preview mode setting

* Parameter Description:

In: void *handle - // unsigned handle pointer

int inTriggMode

1-Soft trigger

3-High voltage trigger

4-AED

5-Dynamic: External trigger

6-Dynamic: External trigger / FPD master

7-Dynamic: Internal / SW trigger

8-SAEC

Other not supported.

int inPreviewMode // Image acquire and display mode

// 0: normal image,

// 1: preview image

// 2: preview + normal image

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetPreviewMode (void * handle, int inTriggMode, int inPreviewMode) ;

3.3.47. HBI_GetPreviewMode

* No: No047

* Function Name: HBI_GetPreviewMode

* Function Description: Get preview mode setting value

* Parameter Description:

```

In: void * handle -unsigned handle pointer (unsigned pointer)

Out:  int *outmode          // Image acquire and display mode [0~2]
                                     // 0: normal image,
                                     // 1: preview image
                                     // 2: preview + normal image

```

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetPreviewMode (void * handle, int * outMode);

3.3.48. HBI_SAecModeApply

/*****

* No: No48

* Function Name: HBI_SAecModeApply

* Function Description:SAEC mode setting

* Parameter Description:

```

In: void * handle -unsigned handle pointer

    int inPreviewMode
        0-normal image
        1-preview image

    int inPerTime - fpd pre ready delay time

    int inPostTime - fpd post ready delay time

Out: None

```

* Return Value: int

0- Successful

Non 0- failed

* Comment:

1> Enable: inPreviewMode=1 (preview image), inPerTime, and inPostTime (customer specified)

2> Cancel: inPreviewMode=0 (normal image), inPerTime and inPostTime can default to 0

*****/

HBI_FPD_DLL int __stdcall HBI_SAecModeApply (void * handle, int inPreviewMode, int inPerTime=0, int inPostTime=0);

3.3.49. HBI_SAecAcq

/******

* No: No49

* Function Name: HBI_SAecAcq

* Function Description: SAEC image acquisition

* Parameter Description:

In: void * handle -unsigned handle pointer

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SAecAcq (void * handle);

3.3.50. HBI_RegProgressCallBack

/******

* No: No50

* Function Name: HBI_RegProgressCallBack

* Function Description: Register callback function, feedback template or firmware downloading progress status

* Parameter Description:

In: void * handle -unsigned handle pointer

USER_CALLBACK_HANDLE_PROCESS handleStatusfun-Register callback function

Void * pObject - Object pointer. Generally, the software passes the object pointer into the SDK, and the callback function provides feedback to the software for static or global functions to access other objects. It can also be a null pointer.

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_RegProgressCallBack(void *handle, USER_CALLBACK_HANDLE_PROCESS handleStatusfun, void* pObject);

3.3.51. HBI_GenerateTemplate

/*****

* No: No51

* Function Name: HBI_GenerateTemplate

* Function Description: // Quick generate offset, gain, defect template

* Parameter Description:

In:	void *handle	// unsigned handle pointer
	EnumIMAGE_ACQ_MODE _mode	// Type of template
	OFFSET_TEMPLATE_MODE	// Offset template
	GAIN_TEMPLATE_MODE	// Gain template
	DEFECT_ACQ_GROUP1,	// Defect template with group1 image
	DEFECT_ACQ_GROUP2,	// Defect template with group2 image
	DEFECT_ACQ_AND_TEMPLATE	// Defect template with group3 image and generate

template

int bpreview // Whether to generate preview template

 // 1: yes

 // 0: no (default)

Out: Null

* Return Value:

```
int                                     // 0: Success

                                     // Non-0: Failure
```

* Comment:

* 1.OFFSET_TEMPLATE: Generate offset template, note that the X-ray should be turned off during dark image acquisition, and the firmware acquires a set of dark field images and generates an Offset template

* 2.GAIN_TEMPLATE: To generate gain template, it is necessary to open the X-ray, adjust the dose level, and start the bright field images acquisition and generate gain template

* 3.DEFECT_TEMPLATE_ GROUP1: To generate defect template, it is necessary to open the X-ray, adjust the dose level, and start acquiring the first group of bright field images

* 4.DEFECT_TEMPLATE_ GROUP2: To generate defect template, it is necessary to open the X-ray, adjust the dose level, and start acquiring the second group of bright field images

* 5.DEFECT_TEMPLATE_ GROUP3: To generate defect template, it is necessary to open the X-ray, adjust the dose level, and start acquiring the third group of bright field images and generate a bad template

* 6.SOFTWARE_ OFFSET_TEMPLATE: Generate dark field template, note that the X-ray should be turned off during dark image acquisition. The SDK acquires a set of dark field images and generates an Offset template

*****/

HBI_FPD_DLL int __stdcall HBI_GenerateTemplate (void * handle, EnumIMAGE-ACQ-MODE_mode, int bprev=0);

3.3.52. HBI_DownloadTemplate

*****/

* No: No52

* Function Name: HBI_DownloadTemplate

* Function Description: Download calibration template

* Parameter Description:

In: void * handle -unsigned handle pointer

DOWNLOAD_FILE * downloadfile - Download template file object pointer

int nModelID:Mode ID,when managing based on multiple Mode templates, it is neccessary to specify the Mode ID.

emUPLOAD_FILE_TYPE emfiletype-Template type(OFFSET_TMP-offset template.GAIN_TMP-gain template,DEFECT_TMP-defect template)

binning4*4) int nBinningtype-template binning type (1-binning1*1,2-binning2*2,3-binning3*3,4-

char filepath-Template absolute path

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:Suggest replacing with HBI_DownloadTemplateByType

*****/

HBI_FPD_DLL int __stdcall HBI_DownloadTemplate (void * handle, DOWNLOAD_FILE * downloadfile);

3.3.53. HBI_StopDownloadTemplate

*****/

* No: No53

* Function Name: HBI_StopDownloadTemplate

* Function Description: Stop download calibration template

* Parameter Description:

In: void * handle -unsigned handle pointer

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_StopDownloadTemplate (void * handle);

3.3.54. HBI_InitDual

*****/

* No: No54

* Function Name: HBI_InitDual

* Function Description: Initialize dual flat detector

* Parameter Description: The return parameter contains two handles

In/Out: void * * handle1- unsigned object pointer for instance of panel A

In/Out: void * * handle1- unsigned object pointer for instance of panel B

* Return Value:

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_InitDual (void * * handle1, void * * handle2);

3.3.55. HBI_InitEx

*****/

* No: No55

* Function Name: HBI_InitEx

* Function Description: Initialize double or multi detector dynamic library instance

* Parameter Description: Callback parameter, FPD instance and ID number

In/Out: HBI_HANDLE *pHandArr // Array object, refer to <HbiError.h>

In: int nArrSize // Array number, equal to DETECTOR_MAX_NUMBER

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_InitEx (HBI_HANDLE * pHandArr, int nArrSize);

3.3.56. HBI_DestroyEx

*****/

* No: No56

* Function Name: HBI_DestroyEx

* Function Description: Destroy all dll resources

* Parameter Description: none

* Return Value: none

* Comment:

*****/

HBI_FPD_DLL void__stdcall HBI_DestroyEx();

3.3.57. HBI_GetDetectorHandle

/*****

* No: No57

* Function Name: HBI_GetDetectorHandle

* Function Description: Get handle through FPD ID number

* Parameter Description:

In: int fpdid - flat panel id, 0 "panel id"

* Return Value:

Void * - unsigned object pointer

* Comment:

*****/

HBI_FPD_DLL void*__stdcall HBI_GetDetectorHandle (int fpdid=0);

3.3.58. HBI_GetDetectorId

/*****

* No: No58

* Function Name: HBI_GetDetectorId

* Function Description: Get FPD ID number through handle

* Parameter Description:

In: void * handle - handle pointer

* Return Value:

int – The ID of detector

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetDetectorId (void * handle);

3.3.59. HBI_ActivateDetector

/*****

* No: No59

* Function Name: HBI_ActivateDetector

* Function Description: Activate detector, current SDK version supports multi FPD

* Parameter Description:

In: void * handle - Flat panel handle

* Return Value:

0- Successful, callback function feedback firmware parameters

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_ActivateDetector (void * handle);

3.3.60. HBI_GetPreviewImageProperty

/*****

* No: No60

* Function Name: HBI_GetPreviewImageProperty

* Function Description: Get preview image property

* Parameter Description:

In: void * handle -unsigned handle pointer

IMAGE_PROPERTY *img_pro//Image property, refer to<HbiError.h> IMAGE_PROPERTY

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetPreviewImageProperty (void * handle, IMAGE_PROPERTY * img_pro);

3.3.61. HBI_PgaBinningAcqTime

/*****

* No: No61

* Function Name: HBI_PgaBinningAcqTime

* Function Description: PGA gain stage, binning, frame speed setting

* Parameter Description:

In: void * handle -unsigned handle pointer

int nPgaLevel - PGA gear, [n] - Invalid, [1] -0.6pC, [2] -1.2pC, [3] -2.4pC, [4] -3.6pC, [5] -4.8pC, [6] -7.2pC, default 7.2pC, [7] -9.6pC, [10] -0.3pC, [11] -0.15pC

unsigned char binning -1:1x1,2:2x2,3:3x3,4:4x4, not supported for others

int acqtime - interval time, in milliseconds, greater than 0

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_PDF_DLL int __stdcall HBI_PgaBinningAcqTime (void * handle, int nPgaLevel, unsigned char binning, int acqtime, int nfeedback=0);

3.3.62. HBI_TriggerPgaBinningAcqTime

/*****

* No: No62

* Function Name: HBI_TriggerPgaBinningAcqTime

* Function Description: // trigger mode, PGA gain stage, binning type, frame speed setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int triggerMode // trigger mode

```

// Static:

// 01: Software trigger mode;

// 02: Clear;

// 03: High voltage trigger mode;

// 04: Auto exposure detection mode.

// Dynamic:

// 05: External trigger mode;

// 06: External trigger / FPD master Mode;

// 07: Internal / SW trigger mode.

int nPgaLevel // PGA gain stage,[n]-Invalid, [1]-0.6pC, [2]-1.2pC, [3]-2.4pC, [4]-3.6pC, [5]-4.8pC,
[6]-7.2pC, [7]-9.6pC, [8]-LFW (COMS), [9]-HFW (COMS), [10]-0.3pC, [11]-0.15pC, Default: 7.2 pC.

unsigned char binning

// 1: 1x1,

// 2: 2x2,

// 3: 3x3,

// 4: 4x4,

int acqtime // Live acquisition span time, unit: ms

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return Value:

int // 0: Success

// Non-0: Failure

* Comment:

*****/

HBI_PDF_DLL int __stdcall HBI_TriggerPgaBinningAcqTime (void * handle, int triggerMode, int nPgaLevel,
unsigned char binning, int acqtime, int nfeedback=0);

```

3.3.63. HBI_TriggerBinningAcqTime

*****/

* No: No63

* Function Name: HBL_TriggerBinningAcqTime

* Function Description: // trigger mode, binning, frame speed setting

* Parameter Description:

In:	void *handle	// unsigned handle pointer
	int triggerMode	// trigger mode
		// Static:
		// 01: Software trigger mode;
		// 02: Clear;
		// 03: High voltage trigger mode;
		// 04: Auto exposure detection mode.
		// Dynamic:
		// 05: External trigger mode;
		// 06: External trigger / FPD master Mode;
		// 07: Internal / SW trigger mode.
	unsigned char binning	
		// 1: 1x1,
		// 2: 2x2,
		// 3: 3x3,
		// 4: 4x4,
	int time	// Live acquisition span time, unit: ms
		// For example: 1000ms (1fps), 500ms (2fps), 250ms
	(4fps),125ms (8fps)	
	Out: Null	
* Return Value:		
	int	// 0: Success
		// Non-0: Failure

* Comment:

*****/

HBI_PDF_DLL int __stdcall HBI_TriggerBinningAcqTime (void * handle, int triggerMode, unsigned char binning, int time);

3.3.64. HBI_PgaAcqTime

/******

* No: No64

* Function Name: HBI_PgaAcqTime

* Function Description: PGA gain stage, frame speed setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int nPgaLevel //PGA gain stage,[n]-Invalid,[1]-0.6pC,[2]-1.2pC,[3]-2.4pC,[4]-3.6pC,[5]-4.8pC,[6]-7.2pC,[7]-9.6pC,[8]-LFW (COMS),[9]-HFW (COMS),[10]-0.3pC,[11]-0.15pC,Default: 7.2 pC.

int acqtime // Live acquisition span time, unit: ms

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_PDF_DLL int __stdcall HBI_PgaAcqTime (void * handle, int nPgaLevel, int acqtime, int nfeedback=0);

3.3.65. HBI_SetSelfDumpingTime

/******

* No: No65

* Function Name: HBI_SetSelfDumpingTime

* Function Description: Self dumping span time setting (set fps) (Dynamic flat panel detector)

* Parameter Description:

In: void * handle -unsigned handle pointer

int time - interval time, in milliseconds,>=1000ms

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetSelfDumpingTime (void * handle, int time);

3.3.66. HBI_GetSelfDumpingTime

/*****

* No: No66

* Function Name: HBI_GetSelfDumpingTime

* Function Description: Get self dumping span time(Dynamic flat panel detector)

* Parameter Description:

In: void * handle -unsigned handle pointer

Time - Time, in milliseconds and milliseconds

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetSelfDumpingTime (void * handle, int * out_time);

3.3.67. HBI_TriggerPgaBinningAcqTimeCorrect

/*****

* No: No067

* Function Name: HBI_TriggerPgaBinningAcqTimeCorrect

* Function description: trigger mode, PGA gain stage, binning type, frame speed, correction enabling setting

* Parameter Description:

In: void *handle - Handle (unsigned pointer)

int triggerMode // Tigger mode

```

// Static:

// 01: Software trigger mode;

// 02: Clear;

// 03: High voltage generator(hvg) trigger mode;

// 04: Auto exposure detection mode.

// Dynamic:

// 05: External trigger mode;

// 06: External trigger / FPD master Mode;

// 07: Internal / SW trigger mode.

int nPgaLevel // PGA gain stage,[n]-Invalid,[1]-0.6pC,[2]-1.2pC,[3]-2.4pC,[4]-3.6pC,[5]-
4.8pC,[6]-7.2pC,[7]-9.6pC,[8]-LFW (COMS),[9]-HFW (COMS),[10]-0.3pC,[11]-0.15pC,Default: 7.2 pC.

unsigned char binning

// 1: 1x1,

// 2: 2x2,

// 3: 3x3,

// 4: 4x4,

int acqtime // Live acquisition span time, unit: ms

IMAGE_CORRECT_ENABLE* pCorrect,refer to <HbiError.h>

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return value: int

0 - Success

Non 0 - Failed

* Remarks:

*****/

HBI_FPD_DLL int __stdcall HBI_TriggerPgaBinningAcqTimeCorrect(void* handle, int triggerMode, int nPgaLevel,
unsigned char binning, int acqtime, IMAGE_CORRECT_ENABLE* pCorrect);

```

3.3.68. HBI_OpenTemplateWizard

```

/*****

```

* No: No068

* Function Name: HBI_OpenTemplateWizard

* Function Description: Open the correction template generation tool

* Parameter Description:

In: void * handle -unsigned handle pointer

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_OpenTemplateWizard (void * handle);

3.3.69. HBI_SetRoiMode

/*****

* No: No069

* Function Name: HBI_SetRoiMode

* Function Description: A-silicon ROI region setting function.

* Currently only 1613-125um.3030-140um.4343-100um.2530-100um are supported.

* Parameter Description:

In: void *handle - unsigned handle pointer

int nMode - ROI mode

for example,1613-125um: 0-Normal Image, w*h(1248*1024), Rect(0,0,1247,1023);

1-w*h(32*1024), Rect(608,0,639,1023);

2-w*h(1248*32), Rect(0,496,1247,527);

3-w*h(1248*64), Rect(0,480,1247,543);

4-w*h(1248*128),Rect(0,448,1247,575);

Others-Normal Image.

3030-140um: 0-Normal Image, w*h(2048*2048), Rect(0,0,2047,2047);

1-w*h(1024*2048),Rect(512,0,1535,2047);

Others-Normal Image。

4343-100um: 0-Normal Image, w*h(4288*4288), Rect(0,0,4287,4287);

1-w*h(4288*800),Rect(0,1743,4287,2544);

Others-Normal Image。

2530-100um: 0-Normal Image, w*h(2432*3072), Rect(0,0,2431,3071);

1-w*h(1808*1800),Rect(316,636,2115,2435);

2-w*h(1312*1300),Rect(566,886,1865,2185);

Others-Normal Image。

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment:only 1613-125um.3030-140um.4343-100um.2530-100um are supported。

*****/

HBI_FPD_DLL int __stdcall HBI_SetRoiMode (void * handle, int nMode, int nfeedback=1);

3.3.70. HBI_SetZoomModeRect

/*****

* No: No070

* Function Name: HBI_SetZoomModeRect

* Function Description: The ZOOM Mode region setting function for CMOS flat panels

* currently supports only CMOS - 1412-50um and CMOS - 0606-50um ,1917-120umflat panel types.

* Parameter Description:

In: void *handle - unsigned handle pointer

CMOS_ZOOM_RECT stZoomRect - Zoom region

The region is represented by left, top, right, and bottom.

for example, when left, top, right and bottom are illegal values, the image will display the full image by default,

1>When left=top=right=bottom=0, the whole picture is displayed;

2>When left<0 or left>width, the whole picture will be displayed, and others are the same.

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment: currently supports only CMOS - 1412-50um and CMOS - 0606-50um 1917-120um flat panel types.

*****/

HBI_FPD_DLL int __stdcall HBI_SetZoomModeRect (void * handle, CMOS_ZOOM_RECT stZoomRect, int nfeedback=1);

3.3.71. HBI_SetCorrectFileDirectory

/*****

* No: No071

* Function Name: HBI_SetCorrectFileDirectory

* Function Description: User set correction file home directory

* If this interface is not called, the calibration file defaults to the DLL directory

* Setting failed. The default is the current path*Setting failed, defaults to current path

* Parameter Description:

In: void *handle - unsigned handle pointer

char * pUserDir - User defined calibration file directory, for example: E:******\

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetCorrectFileDirectory (void * handle, char * pUserDir);

3.3.72. HBI_GetCorrectFileDirectory

/******

* No: No072

* Function Name: HBI_GetCorrectFileDirectory

* Function Description: Obtain the home directory of user correction files

* Parameter Description:

In: void *handle - unsigned handle pointer

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL char * __stdcall HBI_GetCorrectFileDirectory (void * handle);

3.3.73. HBI_UpdateSaveOption

/******

* No: No073

* Function Name: HBI_UpdateSaveOption

* Function Description: Update Save Image Options

* Parameter Description:

In: void *handle - unsigned handle pointer

SAVE_IMAGE saveimg - Structure parameters

and save image; unsigned char m_bySaveType; // 0-show image only,1-save image only,2-show

unsigned char m_bySaveFileType; // 0-image,1-video;

unsigned short m_bySaveNumber; // Number of images included in the video

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_UpdateSaveOption(void* handle, SAVE_IMAGE saveimg);

3.3.74. HBI_SetPacketIntervalTime

/******

* No: No074

* Function Name: HBI_SetPacketIntervalTime

* Function Description: Set Package Interval

* Parameter Description:

In: void *handle - unsigned handle pointer

int *in_itime - TIME, UNIT:um

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetPacketIntervalTime (void * handle, int in_itime);

3.3.75. HBI_AutoWindowWidthWindowLevel

/******

* No: No075

* Function Name: HBI_AutoWindowWidthWindowLevel (新增)

* Function Description: The automatic window width and window level algorithm converts 16bits data into 8-bit unsigned data through window width and window level processing, and users can directly display 8-bit data.

* Parameter Description:

In: void *handle - unsigned handle pointer

In/Out: unsigned char *imgbuff, Source data (16 bit unsigned small end data), destination data (8-bit unsigned data)

In/Out: int *nbufflen, Buffer data length (halve the data size after conversion)

In: int nwidth - Image data width

In: int nheight - Image data high

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_AutoWindowWidthWindowLevel (void * handle, unsigned char * imgbuff, int * nbufflen, int nwidth, int height);

3.3.76. HBI_SetImageRotateAndFlip

/*****

* No: No076

* Function Name: HBI_SetImageRotateAndFlip

* Function Description: Rotate 90 ° or enable mirroring (SDK processing, applicable to low frame rate or small resolution flat panels)

* Parameter Description:

In: void *handle - unsigned handle pointer

In: int nRotate - 0-Do nothing;1-Left Rotate 90°;02-"Right Rotate 90°;

In: int nFlip - 0-Do nothing;1-Horizontal mirror;2-Mirror vertically;

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment:image processing, By default, rotate before mirro

*****/

HBI_FPD_DLL int __stdcall HBI_SetImageRotateAndFlip (void * handle, int nRotate, int nFlip);

3.3.77. HBI_Rotate90AndFlip

/*****

* No: No077

* Function Name: HBI_Rotate90AndFlip (newly increased)

* Function Description: Image rotation and image processing (only applicable to 16-bit unsigned image data, which is rotated first and then mirrored by default)

* Parameter Description:

In: void *handle - unsigned handle pointer

In/Out: unsigned char *imgbuff - Pointer type input and output parameters, source data (16-bit unsigned small-end data), destination data (16-bit unsigned small-end data)

In: int nbufflen - Buffer data length

In/Out: int *nwidth - Pointer type input and output parameters, pointer address storage image data width

In/Out: int *nheight - Pointer type input and output parameters, pointer address storage image data high

In: int nClockwise - Rotate 90 °, 1 - right rotation, 2 - left rotation, other - no processing

In: int nVertical - Mirror processing, 1-vertical mirror, 2-horizontal mirror, other - no processing

* Return Value: int

0: Success

Non-0: Failure

* Comment:By default, rotate before mirror

*****/

HBI_FPD_DLL int __stdcall HBI_Rotate90AndFlip (void * handle, unsigned char * imgbuff, int nbufflen, int * pnwidth, int * pnheight, int nClockwise, int nVertical);

3.3.78. HBI_ResetCorrectTemplate

/*****

* No: No078

* Function Name: HBI_ResetCorrectTemplate (newly increased)

* Function Description: Reset the calibration template file - rotate, mirror, reverse process and save, generally complete the process before downloading to the firmware

- * 1.Find the generated calibration template file according to the current rotation and mirroring parameters (the template file has been rotated and mirrored);
- * 2.Reverse process the calibration template file (mirror first and then reverse rotation);
- * 3.Automatically store the rotated calibration template file according to multi-template management rules;

* Parameter Description:

In: void *handle - unsigned handle pointer

In: int nTemplateType - 0-offset template(16-bits);1-gain template(32-bits);2-defect template(16-bits);

* Return Value: int

0: Success

Non-0: Failure

* Comment:The purpose is to solve the problem that the firmware calibration calibration file is not synchronized. The firmware uses the generated calibration template file that has not been rotated and mirrored by the dynamic library

*****/

HBI_FPD_DLL int __stdcall HBI_ResetCorrectTemplate (void * handle, int nTemplateType);

3.3.79. HBI_GetCurZoomRect

*****/

* No: No079

* Function Name: HBI_GetCurZoomRect (newly increased)

* Function Description: Get the current ZOOM area size data

* Parameter Description:

In: void *handle - unsigned handle pointer

Out: unsigned int *pZoomLeft - Zoom Rect Left

unsigned int *pZoomTop - Zoom Rect Top

unsigned int *pZoomWidth - Zoom Rect Width

unsigned int *pZoomHeight - Zoom Rect Height

* Return Value: int

0: Success

Non-0: Failure

* Comment: Rect data, where all are 0, indicating the full picture.

*****/

```
HBI_FPD_DLL int __stdcall HBI_GetCurZoomRect (void * handle, unsigned int * pZoomLeft, unsigned int * pZoomTop, unsigned int * pZoomWidth, unsigned int * pZoomHeight);
```

3.3.80. HBI_IsExistsOfCurCorrectTemplate

*****/

* No: No080

* Function Name: HBI_IsExistsOfCurCorrectTemplate (newly increased)

* Function Description: Judge whether the current template file exists according to the template type

* Parameter Description:

In: void *handle - unsigned handle pointer

In: int tempType -Template type (default is 0), 0-offset template;1-gain template;2-defect template;3-defect map template;4-gainb template;

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

```
HBI_FPD_DLL int __stdcall HBI_IsExistsOfCurCorrectTemplate (void * handle, int tempType=0);
```

3.3.81. HBI_GetCurrentCorrectFileName

*****/

* No: No081

* Function Name: HBI_GetCurrentCorrectFileName (newly increased)

* Function Description: Get the current template file name

* Parameter Description:

In: void *handle - unsigned handle pointer

In: int tempType -Template type (default is 0), 0-offset template;1-gain template;2-defect template;3-defect map template;4-gainb template;

* Return Value: char*

NULL - Failure

char* - Return file name

* Comment:

*****/

HBI_FPD_DLL char *__stdcall HBI_GetCurrentCorrectFileName (void * handle, int tempType=0);

3.3.82. HBI_GetCurTempFileFullDirectory

*****/

* No: No082

* Function Name: HBI_GetCurTempFileFullDirectory

* Function Description: Get the full directory of the current template file according to the template type

* Parameter Description:

In: void *handle - unsigned handle pointer

In: int tempType -Template type (default is 0), 0-offset template;1-gain template;2-defect template;

Out: Null

* Return Value: char*

NULL - Failure

char* - Return directory path

* Comment:

*****/

HBI_FPD_DLL char *__stdcall HBI_GetCurTempFileFullDirectory (void * handle, int temptype=0);

3.3.83. HBI_GetCurrentCorrectFilePath

*****/

* No: No083

* Function Name: HBI_GetCurrentCorrectFilePath

* Function Description: Obtain the absolute path of the current template file according to the template type

* Parameter Description:

In: void *handle - unsigned handle pointer

In: int tempType -Template type (default is 0), 0-offset template;1-gain template;2-defect template;3-defect map template;4-gainb template;

Out: Null

* Return Value: char*

NULL - Failure

char* - Return absolute path

* Comment:

*****/

HBI_FPD_DLL char *__stdcall HBI_GetCurrentCorrectFilePath (void * handle, int tempType=0);

3.3.84. HBI_Prepare

* No: No084

* Function Name: HBI_Prepare (Add interface)

* Function Description: Single image acquisition(combine with "prepare delay time" parameter)

* Parameter Description:

In: void *handle // unsigned handle pointer

EnumIMAGE_ACQ_CMD //Current acquisition command type (image above or saved template image data)

eg:

SINGLE_ACQ_DEFAULT_TYPE // Single acquisition.

SINGLE_ACQ_GAIN_IMAGE // Generate gain template with static single acquisition.

SINGLE_ACQ_DEFECT_IMAGE // Generate defect template with static single acquisition.

SINGLE_GENERATE_OFFSET, // Quickly generate template collection types, collect a group of dark field images in a single frame - SDK generates offset template.

SINGLE_GENERATE_GAIN, // Quickly generate template collection types, and collect N groups of single frame (at least 3 groups by default) bright field images - SDK generates gain templates

SINGLE_GENERATE_DEFECT, // Quickly generate template collection types, and collect N groups of single frame (at least 3 groups by default) bright field images - SDK generates defect templates

SINGLE_GENERATE_GAIN_DEFECT // Quickly generate template collection types, and collect N groups of single frame (at least 3 groups by default) bright field images - SDK generates gain and defect templates

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*Prepare delay=0 - indicates soft trigger single frame acquisition first HBI_Prepare after HBI_SingleAcquisition completes single frame acquisition

*Prepare delay > "0 - means soft trigger single frame acquisition, only HBI_Prepare is required can complete single frame acquisition according to the delay time

*****/

HBI_FPD_DLL int __stdcall HBI_Prepare (void * handle, EnumIMAGE_ACQ_CMD
acqcmd=EnumIMAGE_ACQ_CMD:: SINGLE_ACQ_DEFAULT_TYPE);

3.3.85. HBI_DownloadTemplateByType

/******

* No: No085

* Function Name: HBI_DownloadTemplateByType

* Function Description: Download calibration template by template type

* Parameter Description:

In: void * handle -unsigned handle pointer (unsigned pointer)

int infiletype - Download file types 0-gain template, 1-defect template, 2-offset
template, others - Not supported

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_DownloadTemplateByType (void * handle, int infiletype);

3.3.86. HBI_GetCurModeList

/******

* No: No086

* Function Name: HBI_GetCurModeList

* Function Description: Get Mode List

* Parameter Description:

In: void *handle - unsigned handle pointer

* Return Value: FPD_MODE_LIST*

NULL - Failure

Non-NULL - Success, The param of User Acquisition Mode Parameter Configuration

* Comment:

*****/

HBI_FPD_DLL FPD_MODE_LIST *__stdcall HBI_GetCurModeList (void * handle);

3.3.87. HBI_UpdateModeList

/******

* No: No087

* Function Name: HBI_UpdateModeList

* Function Description: Update Mode List

* Parameter Description:

In: void *handle - unsigned handle pointer

FPD_MODE_LIST* modelist -User Acquisition Mode Parameter Configuration

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_UpdateModeList (void * handle, FPD_MODE_LIST * modellist);

3.3.88. HBI_GetCurModeData

/******

* No: No088

* Function Name: HBI_GetCurModeData

* Function Description: Obtain the current Mode Data

* Parameter Description:

In: void *handle - unsigned handle pointer

* Return Value: FPD_MODE_DATA*

NULL - Failure

Non-NULL - Success, The param of fpd mode data information

* Comment:

*****/

HBI_FPD_DLL FPD_MODE_DATA * __stdcall HBI_GetCurModeData (void * handle);

3.3.89. HBI_UpdateCurModeData

*****/

* No: No089

* Function Name: HBI_UpdateCurModeData

* Function Description: Update current Mode Data

* Parameter Description:

In: void *handle - unsigned handle pointer

FPD_MODE_DATA* mode - fpd mode data information

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_UpdateCurModeData (void * handle, FPD_MODE_DATA * mode);

3.3.90. HBI_UpdateModeData

*****/

* No: No090

* Function Name: HBI_UpdateModeData

* Function Description: Update Mode Item Data through Mode ID

* Parameter Description:

In: void *handle - unsigned handle pointer

FPD_MODE_DATA* mode -fpd mode data information

unsigned int id - mode id

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_UpdateModeData (void * handle, FPD_MODE_DATA * mode, unsigned int id=0);

3.3.91. HBI_GetCurModelID

*****/

* No: No091

* Function Name: HBI_GetCurModelID

* Function Description: Obtain the current Mode ID

* Parameter Description:

In: void *handle - unsigned handle pointer

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetCurModelID (void * handle);

3.3.92. HBI_SwitchUserMode

*****/

* No: No092

* Function Name: HBI_SwitchUserMode

* Function Description: Switch Mode item

* Parameter Description:

In: void *handle - unsigned handle pointer

unsigned int modeid - mode id

FPD_MODE_DATA *PMODEDATA - fpd mode data information

* Return Value: int

0: Success

Non-0: Failure

* Comment:

```

/*****/

FPD_DLL int __stdcall HBI_SwitchUserMode (void * handle, unsigned int modeid=0, FPD_MODE_DATA *
PMODEDATA=NULL);

```

3.3.93. HBI_ModeCorrectTempalteEnable

```

/*****/

```

* No: No093

* Function Name: HBI_ModeCorrectTempalteEnable

* Function Description: The calibration template of Mode is effective or not effective

* Parameter Description:

In: void *handle - unsigned handle pointer

unsigned int id - mode id

bool bEnable - 1 - vailid, 0-invaild

* Return Value: int

0: Success

Non-0: Failure

* Comment:

```

/*****/

HBI_FPD_DLL int __stdcall HBI_ModeCorrectTempalteEnable (void * handle, unsigned int id, bool
bEnable=true);

```

3.3.94. HBI_AllModeCorrectTempalteEnable

```

/*****/

```

* No: No094

* Function Name: HBI_AllModeCorrectTempalteEnable

* Function Description: All calibration templates for Mode items are valid or not valid

* Parameter Description:

In: void *handle - unsigned handle pointer

bool bEnable - 1 - valid, 0-invalid

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_AllModeCorrectTempalteEnable (void * handle, bool bEnable=true);

3.3.95. HBI_SaveModeList2File

*****/

* No: No095

* Function Description: Save the current user mode list

* Parameter Description:

In: void *handle - unsigned handle pointer

char* szPath - file path

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SaveModeList2File (void * handle, char * szPath=NULL);

3.3.96. HBI_DownloadModeTemplate

*****/

* No:No0976

* Function Name: HBI_DownloadModeTemplate (new)

* Function description: Download template based on Mode parameter

* Parameter Description:

In: void * handle - Handle (unsigned pointer)

Int intemptype - Download file types 0-gain template, 1-defect template, 2-offset template, others - Not supported

Out: None

* Return value: int

0- Success

Non 0- Failed

* Note: Download template add by mhyang 20230606

*****/

HBI_FPD_DLL int __stdcall HBI_DownloadModeTemplate (void * handle, DOWNLOAD-MODE-FILE downloadfile);

3.3.97. HBI_DownloadModeTemplateByType

/*****

* No:No097

* Function name: HBI_DownloadModeTemplateByType (new)

* Function description: Download correction template files by template type (default to current Mode Id)

* Parameter Description:

In: void * handle - Handle (unsigned pointer)

Int intemptype - Download file types 0-gain template, 1-defect template, 2-offset template, others - Not supported

Out: None

* Return value: int

0- Success

Non 0- Failed

* Note: Download template add by mhyang 20230606

*****/

HBI_FPD_DLL int __stdcall HBI_DownloadModeTemplateByType (void * handle, int intemptype);

3.3.98. HBI_DownloadModeTemplateByIdAndType

/******

* No:No098

* Function name: HBI_DownloadModeTemplateByIdAndType (new)

* Function description: Download correction template files according to Mode Id and template type

* Parameter Description:

In: void * handle - Handle (unsigned pointer)

Unsigned int modeid - Mode Id [0,9]

Int intemptype - Download file types 0-gain template, 1-defect template, 2-offset template, others - Not supported

Out: None

* Return value: int

0- Success

Non 0- Failed

* Note: Download template add by mhyang 20230605

*****/

HBI_FPD_DLL int __stdcall HBI_DownloadModeTemplateByIdAndType (void * handle, unsigned int modeid, int intemptype);

3.3.99. HBI_GetCurModeTemplateStatus

/******

* No:No99

* Function Name: HBI_GetCurModeTemplateStatus (new)

* Function Description: Obtain the firmware calibration template status for current Mode item

* Parameter Description:

In: void *handle - unsigned handle pointer

int * nTempStatus - Template status, 0- Dark field bright field invalid, 1- Dark field valid bright field invalid, 2- Dark field invalid bright field valid, 3- Dark field bright field valid, other - Dark field bright field invalid

Out: None

*Return Value: int

0: Successful

Non 0: Failed

*Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetCurModeTemplateStatus (void * handle, int * nTempStatus);

3.3.100. HBI_FirmwareManagementTempalteType

/*****

*No: No100

* Function name: HBI_FirmwareManagementTempalteType (new)

* Function description: Firmware template management template method

* Parameter Description:

In: void * handle - Handle (unsigned pointer)

unsigned int uType - Firmware template management template method

0-by binning type,

1-by Muti-Mode type

others-by binning type

Out: None

* Return value: int

0 - Success

Non 0 - Failed

* Remarks:

*****/

HBI_FPD_DLL int __stdcall HBI_FirmwareManagementTempalteType (void * handle, unsigned int uType);

3.3.101. HBI_SetValidModelItemNumber

/*****

*No: No101

* Function Name: HBI_SetValidModelItemNumber (new)

* Function Description: Set the number of valid Mode items

* Parameter Description:

In: void * handle -unsigned handle pointer

unsigned int uNumber - Number of Mode items, uNumber: [0,10]

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetValidModelItemNumber (void * handle, unsigned int uNumber);

3.3.102. HBI_IsMutilMode

*****/

* No: No102

* Function name: HBI_IsMutilMode (new)

* Function description: Check if the current firmware supports multiple modes

* Parameter Description:

In: void *handle - Handle (unsigned pointer)

* Return value: bool

true - support

false - Not Supported

* Remarks:The essence of Mode mode is that the firmware manages templates differently

*****/

HBI_FPD_DLL bool __stdcall HBI_IsMutilMode (void * handle);

3.3.103. HBI_MUUpdateTriggerMode

*****/

* No: No103

* Function name: HBI_MUUpdateTriggerMode (new)

* Function description: trigger mode setting

* Parameter Description:

In: void *handle - Handle (unsigned pointer)

```

int _triggerMode-      trigger mode

// 1: Software trigger mode

// 3: High voltage generator trigger mode

// 4: AED trigger mode

// 5-Dynamic:Hvg Sync Mode

// 6-Dynamic:Fpd Sync Mode

// 7-Dynamic:Fpd Continue

// 8-SAEC。

// others-invaild

```

Out:None

* Return value: int

0 - Success

Non 0 - Failed

* Remarks:

*****/

HBI_FPD_DLL int __stdcall HBI_MUpdateTriggerMode(void * handle, int_triggerMode);

3.3.104. HBI_MSetBinning

/*****

* No: No104

* Function name: HBI_MSetBinning (new)

* Function description: Binning mode setting

* Parameter Description:

In:

```

Void * handle -unsigned handle pointer

unsigned int ubining -1:1x1,2:2x2,3:3x3,4:4x4

```

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MSetBinning (void * handle, unsigned int ubining);

3.3.105. HBI_MSetPGAlevel

/******

* No: No105

* Function Name: HBI_MSetPGAlevel (new)

* Function Description: PGA gain stage setting

* Parameter Description:

In: void * handle -unsigned handle pointer (unsigned pointer)

int mode - Mode

[n] - Invalid

[1] -0.6pC

[2] -1.2pC

[3] -2.4pC

[4] -3.6pC

[5] -4.8pC

[6] -7.2pC, default to 7.2pC

[7] -9.6pC

[8] - LFW (COMS)

[9] - HFW (COMS)

[10] -0.3pC

[11] -0.15pC

Out: None

* Return Value: int

0- Successful

Non 0- failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MSetPGAlevel (void * handle, int nGainLevel);

3.3.106. HBI_MSetRoiMode

/*****

* No: No106

* Function name: HBI_MSetRoiMode (new)

* Function Description: A-silicon ROI region setting function.

* Currently only 1613-125um.3030-140um.4343-100um.2530-100um are supported.

* Parameter Description:

In: void *handle - unsigned handle pointer

int nMode - ROI mode

for example,1613-125um: 0-Normal Image, w*h(1248*1024), Rect(0,0,1247,1023);

1-w*h(32*1024), Rect(608,0,639,1023);

2-w*h(1248*32), Rect(0,496,1247,527);

3-w*h(1248*64), Rect(0,480,1247,543);

4-w*h(1248*128),Rect(0,448,1247,575);

Others-Normal Image。

3030-140um: 0-Normal Image, w*h(2048*2048), Rect(0,0,2047,2047);

1-w*h(1024*2048),Rect(512,0,1535,2047);

Others-Normal Image。

4343-100um: 0-Normal Image, w*h(4288*4288), Rect(0,0,4287,4287);

1-w*h(4288*800),Rect(0,1743,4287,2544);

Others-Normal Image。

2530-100um: 0-Normal Image, w*h(2432*3072), Rect(0,0,2431,3071);

1-w*h(1808*1800),Rect(316,636,2115,2435);

2-w*h(1312*1300),Rect(566,886,1865,2185);

Others-Normal Image.

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment:only 1613-125um.3030-140um.4343-100um.2530-100um are supported.

*****/

HBI_FPD_DLL int __stdcall HBI_MSetRoiMode (void * handle, int nMode, int nfeedback=1);

3.3.107. HBI_MSetZoomModeRect

/*****/

* No: No107

* Function name: HBI_MSetZoomModeRect (new)

* Function Description: The ZOOM Mode region setting function for CMOS flat panels

* currently supports only CMOS - 1412-50um and CMOS - 0606-50um ,1917-120umflat panel types.

* Parameter Description:

In: void *handle - unsigned handle pointer

CMOS_ZOOM_RECT stZoomRect - Zoom region

The region is represented by left, top, right, and bottom.

for example, when left, top, right and bottom are illegal values, the image will display the full image by default,

1>When left=top=right=bottom=0, the whole picture is displayed;

2>When eft<0 or left>width, the whole picture will be displayed, and others are the same.

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

Out: Null

* Return Value: int

0: Success

Non-0: Failure

* Comment: currently supports only CMOS - 1412-50um and CMOS - 0606-50um 1917-120um flat panel types.

*****/

HBI_FPD_DLL int __stdcall HBI_MSetZoomModeRect (void * handle, CMOS_ZOOM-RECT stZoomRect, int nfeedback=1);

3.3.108. HBI_MSetSelfDumpingTime

/*****

* No: No108

* Function name: HBI_MSetSelfDumpingTime (new)

* Function Description: // Self dumping span time setting

* Parameter Description:

In:	void *handle // unsigned handle pointer
	int time // Self dumping span time, unit: ms,>0ms

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MSetSelfDumpingTime (void * handle, int time);

3.3.109. HBI_MSetLiveAcquisitionTime

/*****

* No: No109

* Function name: HBI_MSetLiveAcquisitionTime (new)

* Function Description: Live acquisition span time setting (apply to static and wireless type FPD)

* Parameter Description:

In: void *handle // unsigned handle pointer

int time // Live acquisition span time, unit: ms,>0ms

Out: Null

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MSetLiveAcquisitionTime (void * handle, int time);

/******

* No: No110

* Function name: HBI_MSetSinglePrepareTime (new)

* Function Description: // Prepare delay time setting, which is also called exposure window

* Parameter Description:

In: void *handle // unsigned handle pointer

Out: int *out_itime // Delay Time, unit: ms

// 0: "HBI_Prepare" and "HBI_SingleAcquisition" command are combined to realize single image acquisition

// Non-0: FPD will delay "Prepare Delay Time" and automatically acquire one single image when the detector receives "HBI_Prepare" command.

* Return Value: int

// 0: Success

// Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MSetSinglePrepareTime (void * handle, int in_itime);

3.3.110. HBI_MSetFpsOfTime

/******

* No: No111

* Function name: HBI_MSetFpsOfTime (new)

* Function description: Set frame rate control time, support subtle

* Parameter Description:

In: void *handle - Handle (unsigned pointer)

unsigned int untimems - time interval,Unit is milliseconds, > 0ms

unsigned short ustimeus - time interval,The unit is microseconds, >= 0ms.[0,999]

Out: None

* Return value: int

0 - Success

Non 0 - Failed

* Remarks:eg,fps=16, ntimems=62ms, ntimeus=8um

*****/

HBI_FPD_DLL int __stdcall HBI_MSetFpsOfTime (void * handle, unsigned int untimes, unsigned short ustimeus=0);

3.3.111. HBI_MPgaAcqTime

/*****/

* No: No112

* Function Description: // PGA gain stage, frame speed setting

* Parameter Description:

In: void *handle // unsigned handle pointer

int nPgaLevel // PGA gain stage,[n]-Invalid,[1]-0.6pC,[2]-1.2pC,[3]-2.4pC,[4]-3.6pC,[5]-4.8pC,[6]-7.2pC,[7]-9.6pC,[8]-LFW (COMS),[9]-HFW (COMS),[10]-0.3pC,[11]-0.15pC,Default: 7.2 pC.

int acqtime // Live acquisition span time, unit: ms

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return Value: int

// 0: Success

// Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MPgaAcqTime (void * handle, int nPgaLevel, int acqtime, int nfeedback=0);

3.3.112. HBI_PgaBinningAcqTime

*****/

* No: No113

* Function name: HBI_MPgaBinningAcqTime (new)

* Function Description: // PGA gain stage, binning, frame speed setting

* Parameter Description:

In: void *handle // unsigned handle pointer

 int nPgaLevel // PGA gain stage,[n]-Invalid,[1]-0.6pC,[2]-1.2pC,[3]-2.4pC,[4]-3.6pC,[5]-4.8pC,[6]-7.2pC,[7]-9.6pC,[8]-LFW (COMS),[9]-HFW (COMS),[10]-0.3pC,[11]-0.15pC,Default: 7.2 pC.

 unsigned char binning

 // 1: 1x1,

 // 2: 2x2,

 // 3: 3x3,

 // 4: 4x4,

 int acqtime // Live acquisition span time, unit: ms

 int nfeedback // 1: Automatic feedback config parameters if setting is successful

 // Non-1: Feedback setting success status

* Return Value:

 int // 0: Success

 // Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MPgaBinningAcqTime (void * handle, int nPgaLevel, unsigned char binning, int acqtime, int nfeedback=0);

3.3.113. HBI_MTriggerAndCorrectDisplay

*****/

* No: No114

* Function name: HBI_MTriggerAndCorrectApplay (new)

* Function Description: // Tigger mode and image calibration setting

* Parameter Description:

In:	void *handle	// unsigned handle pointer
	int _triggerMode	
		// 1: Software trigger mode
		// 3: High voltage generator trigger mode
		// 4: AED trigger mode
		// 5-Dynamic:Hvg Sync Mode
		// 6-Dynamic:Fpd Sync Mode
		// 7-Dynamic:Fpd Continue
		// 8-SAEC。

IMAGE_CORRECT_ENABLE* pCorrect // refer to<< HbiType.h>>

Out: Null

* Return Value: int

// 0: Success
 // Non-0: Failure, refer to <HbiError.h>

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MTriggerAndCorrectApplay(void* handle, int _triggerMode, IMAGE_CORRECT_ENABLE* pCorrect);

3.3.114. HBI_MTriggerBinningAcqTime

/*****

* No: No115

* Function name: HBI_MTriggerBinningAcqTime (new)

* Function Description: // Tigger mode, binning, frame speed setting

* Parameter Description:

In:	void *handle	// unsigned handle pointer
	int triggerMode	// Tigger mode

```

// Static:

// 01: Software trigger mode;

// 02: Clear;

// 03: High voltage generator(hvg) trigger mode;

// 04: Auto exposure detection mode.

// Dynamic:

// 05: Hvg pulse sync in trigger mode;

// 06: Fpd pulse sync out trigger Mode;

// 07: Free run mode.

unsigned char binning

// 1: 1x1,

// 2: 2x2,

// 3: 3x3,

// 4: 4x4,

int time // Live acquisition span time, unit: ms

// For example: 1000ms (1fps), 500ms (2fps), 250ms

(4fps),125ms (8fps)

```

Out: Null

* Return Value:

```

int // 0: Success

// Non-0: Failure

```

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MTriggerBinningAcqTime (void * handle, int triggerMode, unsigned char binning, int acqtime, int nfeedback=0);

3.3.115. HBI_MTriggerPgaBinningAcqTime

* No: No116

* Function name: HBI_MTriggerPgaBinningAcqTime (new)

* Function description: trigger mode, PGA gain stage, binning type, frame speed setting

* Parameter Description:

In: void *handle - Handle (unsigned pointer)

int triggerMode // Tigger mode

// Static:

// 01: Software trigger mode;

// 02: Clear;

// 03: High voltage generator(hvg) trigger mode;

// 04: Auto exposure detection mode.

// Dynamic:

// 05: Hvg pulse sync in trigger mode;

// 06: Fpd pulse sync out trigger Mode;

// 07: Free run mode.

int nPgaLevel // PGA gain stage,[n]-Invalid,[1]-0.6pC,[2]-1.2pC,[3]-2.4pC,[4]-3.6pC,[5]-4.8pC,[6]-7.2pC,[7]-9.6pC,[8]-LFW (COMS),[9]-HFW (COMS),[10]-0.3pC,[11]-0.15pC,Default: 7.2 pC.

unsigned char binning

// 1: 1x1,

// 2: 2x2,

// 3: 3x3,

// 4: 4x4,

int acqtime // Live acquisition span time, unit: ms

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return value: int

0 - Success

Non 0 - Failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MTriggerPgaBinningAcqTime (void * handle, int triggerMode, int nPgaLevel, unsigned char binning, int acqtime, int nfeedback=0);

3.3.116. HBI_MTriggerPgaBinningAcqTimeCorrect

/*****

* No: No117

* Function name: HBI_MTriggerPgaBinningAcqTimeCorrect (new)

* Function description: trigger mode, PGA gain stage, binning type, frame speed, correction enabling setting

* Parameter Description:

In: void *handle - Handle (unsigned pointer)

int triggerMode // Tigger mode

// Static:

// 01: Software trigger mode;

// 02: Clear;

// 03: High voltage generator(hvg) trigger mode;

// 04: Auto exposure detection mode.

// Dynamic:

// 05: Hvg pulse sync in trigger mode;

// 06: Fpd pulse sync out trigger Mode;

// 07: Free run mode.

int nPgaLevel // PGA gain stage

4.8pC,[6]-7.2pC,[7]-9.6pC // [n]-Invalid,[1]-0.6pC,[2]-1.2pC,[3]-2.4pC,[4]-3.6pC,[5]-

0.15pC,Default: 7.2 pC. // [8]-LFW (COMS),[9]-HFW (COMS),[10]-0.3pC,[11]-

unsigned char binning

// 1: 1x1,

// 2: 2x2,

// 3: 3x3,

// 4: 4x4,

int acqtime // Live acquisition span time, unit: ms

IMAGE_CORRECT_ENABLE* pCorrect, refer to <HbiError.h>

int nfeedback // 1: Automatic feedback config parameters if setting is successful

// Non-1: Feedback setting success status

* Return value: int

0 - Success

Non 0 - Failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_MTriggerPgaBinningAcqTimeCorrect (void * handle, int triggerMode, int nPgaLevel, unsigned char binning, int acqtime, IMAGE_CORRECT_ENABLE * pCorrect);

3.3.117. HBI_GetModeData

/*****/

* No: No118

* Function Name: HBI_GetModeData

* Function Name: HBI_GetModeData

* Function Description: Obtain Mode Data through Mode ID

* Parameter Description:

In: void *handle - unsigned handle pointer

unsigned int id - mode id

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL FPD_MODE_DATA * __stdcall HBI_GetModeData (void * handle, unsigned int id=0);

3.3.118. HBI_GetModeListSize

/*****/

* No: No119

* Function Name: HBI_GetModeListSize

* Function Description: Obtain the number of items for Mode

* Parameter Description:

In: void *handle - unsigned handle pointer

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL unsigned int __stdcall HBI_GetModeListSize (void * handle);

3.3.119. HBI_ModeExists

*****/

* No: No120

* Function Name: HBI_ModeExists

* Function Description: Determine whether the mode exists

* Parameter Description:

In: void *handle - unsigned handle pointer

FPD_MODE_DATA mode - fpd mode data information

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_ModeExists (void * handle, FPD_MODE_DATA mode);

3.3.120. HBI_CheckCurtMode

*****/

* No: No121

* Function Name: HBI_CheckCurtMode

* Function Description: Judge whether the mode is normal

* Parameter Description:

In: void *handle - unsigned handle pointer

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL bool __stdcall HBI_CheckCurtMode (void * handle);

3.3.121. HBI_InitDetectorParams

/*****/

* No: No122

* Function name: HBI_InitDetectorParams (new)

* Function description: initialize the basic parameters of the flat panel detector

* Parameter Description:

In: void *handle - Handle (unsigned pointer)

DETECTOR_INIT_PARAMS param -refer to 《HbiType.h》

param._ucmodeid; //mode id [0,9];

param._uctriggerType; // 1-soft trigger, 3-hvg trigger, 4-FreeAED, 5-Dynamic: Hvg Sync Mode, 6-Dynamic:Fpd Sync Mode, 7-Dynamic:Fpd Continue, 8-SAEC;

//Only vaild in Binning mode,invaild in multi-mode mode,purpose compatible with diffrent version.

param._ucOffsetCorrection; //0-No Offset Correction;1-Software PreOffset Correction; 2-Firmware PostOffset Correction;3-Firmware PreOffset Correction;

param._ucGainCorrection; //0-No Gain Correction; 1-Software Gain Correction; 2-Firmware Gain Correction;

param._ucDefectCorrection; //0-No Defect Correction; 1-Software Defect Correction; 2-Firmware Defect Correction;

param._ucDummyCorrection; //0-No Dummy Correction; 1-Software Dummy Correction; 2-Firmware Dummy Correction;

Out: None

* Return value: int

0 - Success

Non 0 - Failed

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_InitDetectorParams (void * handle, DETECTOR_INIT_PARAMS param);

3.3.122. HBI_SetWindowMode

/******

* No: No123

* Function Name: HBI_SetWindowMode

* Function Description:

* Parameter Description:

In: void *handle - unsigned handle pointer

bool isWindowMode - true:enable,false-disable

Out:None

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetWindowMode(void* handle, bool isWindowMode);

3.3.123. HBI_GetWindowMode

/******

* No: No124

* Function Name: HBI_GetWindowMode

* Function Description:

* Parameter Description:

In: void *handle - unsigned handle pointer

In/Out bool *isWindowMode - true:enable,false-disable

Out:None

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetWindowMode(void* handle, bool* isWindowMode);

3.3.124. HBI_SetBinningDivisor

/*****

* No: No125

* Function Name: HBI_SetBinningDivisor

* Function Description:

* Parameter Description:

In: void *handle - unsigned handle pointer

unsigned char binDiv, [0,1,2,4,8,16]

Out:None

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_SetBinningDivisor(void* handle, unsigned char binDiv);

3.3.125. HBI_GetBinningDivisor

/*****

* No: No126

* Function Name: HBI_GetBinningDivisor

* Function Description:

* Parameter Description:

In: void *handle - unsigned handle pointer

In/Out unsigned char* binDiv, [0, 1, 2, 4, 8,16]

Out:None

* Return Value: int

0: Success

Non-0: Failure

* Comment:

*****/

HBI_FPD_DLL int __stdcall HBI_GetBinningDivisor(void* handle, unsigned char* binDiv);

3.4. Callback Event

The communication mechanism between the dynamic library and firmware is asynchronous. The panel status, parameters, image data and monitoring information can be got through callback functions. The software must register a main callback function and pass the address of the callback function to the dynamic library through the interface, the callback function includes the main callback function and the progress callback function.

Main Callback Function

The status of the panel, parameters, image data, stacking number, main thread status, etc. are feedback to software through the main callback function. Before using it, the main callback function must be registered.

Callback function type definition:

//Notice: call back function

//@ USER_CALLBACK_HANDLE_ENVENT

//@ byteEventId: enum eEventCallbackCommType

//@ ufpdId: Panel device ID

//@ PVEventParam1: fpd config or image data buffer addr

//@ nEventParam2: data size or status

//@ nEventParam3: frame rate or status

//@ nEventParam4: PCIe event ID or reserved extension

```
typedef int (* USER_CALLBACK_HANDLE_ENVENT)(void *pContext, int ufpdId, unsigned char byteEventId, void
* PVEventParam1, int nEventParam2, int nEventParam3, int nEventParam4);
```

Function Description: Callback function definition, handling configuration, images, and panel return status.

Parameter Description:

PContext: Parameter 1, software object pointer, can be null

UfpdId: Parameter 2, panel ID

ByteEventId: Parameter 3, Event ID, refer to enum eCallbackEventCommType in HbiType. h

PvParam1: Parameter 4, configure structure pointer or image structure pointer

NParam2: Parameter 5, data size (image data size is the length of the image buffer)

Param3: Parameter 6, frame number, frame ID

Param4: Parameter 7, frame rate

Download Progress and Result Callback Function

Callback function type definition:

//Notice: call back function

//@ USER_CALLBACK_HANDLE_PROCESS

//@ cmd: enum eAnswerCallbackCommType

//@ buff

//@ len

Typedef int (* USER-CALLBACK HANDLE-PROCESS) (unsigned char cmd, int retcode, void * inContext);

Function description: Callback function definition, handling firmware update message feedback.

Parameter Description:

cmd: command message returned from SDK.

retcode: return code

inContext: object pointer

Callback Event Type

Below are the main types of callback events and their information:

Table 29.

Event ID	Value	Remarks
ECALLBACK_TYPE_FPD_STATUS	0X09	Panel status
ECALLBACK_TYPE_ROM_UPLOAD	0X10	Plate ROM parameters

ECALLBACK_TYPE_SET_CFG_OK	0XAB	Setting parameters successfully, (usually returns ROM parameters) can also return a success message.
ECALLBACK_TYPE_SINGLE_IMAGE	0X51	Single frame image acquisition
ECALLBACK_TYPE_MULTIPLE_IMAGE	0X52	Continuous image acquisition
ECALLBACK_TYPE_OFFSET_TMP	0X5C	Firmware completed, Pre-offset template data upload
ECALLBACK_TYPE_PREVIEW_IMAGE	0X53	Upload preview images in Preview mode
ECALLBACK_OVERLAY_NUMBER	0X5D	Image overlay
ECALLBACK_OVERLAY_16BIT_IMAGE	0X5E	The 16-bit image data format overlay
ECALLBACK_OVERLAY_32BIT_IMAGE	0X5F	The 32-bit image data format overlay
ECALLBACK_TYPE_PACKET_MISS	0X5B	Packet loss / frame loss feedback information
ECALLBACK_TYPE_LIVE_ACQ_OK	0XA0	Feedback information on saving image files: including directory, average grayscale value
ECALLBACK_TYPE_THREAD_EVENT	0XA5	Monitoring thread or application thread monitoring information
ECALLBACK_TYPE_PACKET_MISS_MSG	0XA4	Feedback on packet loss information during image acquisition
ECALLBACK_TYPE_ACQ_DISCARDED	0XA6	Discard image information feedback (user can set how many image can be discarded before acquisition)
ECALLBACK_TYPE_OFFSET_ERR_MSG	0XA7	Generate offset template results with stepping process
ECALLBACK_TYPE_GAIN_ERR_MSG	0XA8	Generate gain template results with stepping process
ECALLBACK_TYPE_DEFECT_ERR_MSG	0XA9	Generate defect template results with stepping process
ECALLBACK_TYPE_UPDATE_FIRMWARE	0X06	Firmware updates: progress and results
ECALLBACK_TYPE_ERASE_FIRMWARE	0X07	Firmware update: erasing firmware
ECALLBACK_TYPE_WORK_STATUS	0XAF	Working mode switching: Feedback on switching between production mode and debugging mode
ECALLBACK_TYPE_SAVE_SUCCESS	0XAC	Saving image feedback during template generation process
ECALLBACK_TYPE_GENERATE_TEMPLATE	0XAD	Quick template generation information feedback
ECALLBACK_TYPE_FILE_NOTEXIST	0XAE	File unavailable feedback

Example: The status of flat panel detector (ECALLBACK_TYPE_FPD_STATUS)

Table 30.

state	value	Remarks
FPD_DISCONN_STATUS	0	Callback received -11 "nEventParam2" and 0 are all considered communication exceptions. Please refer to the SDK Demo project for details
FPD_PREPARE_STATUS	1	Product preparation status for exposure action, stop self-clearing
FPD_READY_STATUS	2	Static product idle status
FPD_DOOFFSET_TEMPLATE	3	Firmware generates Pre-offset template process
FPD_EXPOSE_STATUS	4	Product in busy / expose status
FPD_CONTINUE_READY	5	Dynamic product idle status
FPD_DWONLOAD_GAIN	6	Download gain template
FPD_DWONLOAD_DEFECT	7	Download defect template
FPD_DWONLOAD_OFFSET	8	Download offset template
FPD_UPDATE_FIRMARE	9	Updating firmware
FPD_RETRANS_MISS	10	Retransmitting data
FPD_STATUS_AED	11	Product in AED status
FPD_STATUS_SLEEP	12	Wireless product: sleep mode
FPD_STATUS_WAKEUP	13	Wireless product: wake-up
FPD_DOWNLOAD_NO_IMAGE	14	Wireless product: download images: no image available for download
FPD_DOWNLOAD_TAIL_IMAGE	15	Wireless product: download image: download the last frame image
FPD_EMMC_MAX_NUMBER	16	Emmc storage capacity alarm
FPD_ENDTIME_WARNNING	17	Firmware authorization expired
FPD_CONN_SUCCESS	100	Connection successful

The callback function returns:

```

case ECALLBACK_TYPE_ROM_UPLOAD://Update configuration
    .....

case ECALLBACK_TYPE_SINGLE_IMAGE://Single frame capture image
    .....
  
```


case ECALLBACK_TYPE_MULTIPLE_IMAGE://Continuous collection of images

The callback function returns:

case ECALLBACK_TYPE_ROM_UPLOAD: // Update configuration

.....

case ECALLBACK_TYPE_SINGLE_IMAGE: // Single frame image acquisition

.....

case ECALLBACK_TYPE_MULTIPLE_IMAGE: // Continuous image acquisition

.....

case ECALLBACK_TYPE_FPD_STATUS: // Panel status: connected / disconnected / ready / busy

```
printf("ECALLBACK TYPE FPD STATUS, recode=% d \ n", nlength);
```

```
if (TheDemo!=NULL)
```

```
{
```

```
    CString strMsg=_T ();
```

```
    if (nlength<=0&& nlength>=-11)
```

```
    {
```

```
        If (nlength==0)
```

```
            printf("ECALLBACK_TYPE_FPD_STATUS, Err: Network not connected! \ n");
```

```
        else if (nlength=-1)
```

```
            printf("ECALLBACK_TYPE_FPD_STATUS, Err: parameter exception! \ n");
```

```
        else if (nlength=-2)
```

```
            printf("ECALLBACK TYPE FPD_STATUS, Err: Failed to return number of prepared descriptors! \ n");
```

```
        else if (nlength=-3)
```

```
            printf("ECALLBACK TYPE FPD STATUS, Err: Receive timeout! \ n");
```

```
        else if (nlength=-4)
```

```
            printf("ECALLBACK TYPE FPD STATUS, Err: Receive failed! \ n");
```

```
        else if (nlength=-5)
```

```
            printf("ECALLBACK TYPE FPD STATUS, Err: Port unreadable! \ n");
```

```
        else if (nlength=-6)
```

```

        printf("ECALLBACK TYPE FPD STATUS, network card fail! \n");
    else if (nlength==7)
        printf("ECALLBACK TYPE FPD STATUS, network card ok! \n");
    else if (nlength==8)
        printf("ECALLBACK TYPE FPD STATUS: update Firmware end! \n");
    else if (nlength==9)
        printf("ECALLBACK TYPE FPD STATUS: Fiber optic disconnected!! \n");
    else if (nlength==10)
        printf("ECALLBACK TYPE FPD STATUS: read DDR failed, try restarting the PCIe
driver!");

    else/* if (nlength==11)*/
printf("ECALLBACK TYPE FPD STATUS: ECALLBACK TYPE FPD STATUS: is not jump! \n");
        Status=(int) FPD_DISCONN_STATUS;
    }
    else if (nlength==FPD_CONN_SUCCESS) { // connect
        printf("ECALLBACK TYPE FPD STATUS, start listening! \n");
        Status=(int) FPD_CONN_SUCCESS;
    }
    else if (nlength==FPD_REPARE_STATUS) { // ready
        printf("ECALLBACK TYPE FPD STATUS, ready! \n");
        Status=(int) FPD_PREPARE_STATUS;
    }
    else if (nlength==FPD_READY-STATUS) { // busy
        printf("ECALLBACK TYPE FPD STATUS, busy! \n");
        Status=(int) FPD_READY_STATUS;
    }
    else if (nlength==FPP_DOFFSET-TEMPLATE) { // prepare
        printf("ECALLBACK TYPE FPD STATUS, prepare! \n");
        Status=(int) FPD_DOOFFSET_TEMPLATE;

```

```
}  
  
else if (nlength==FPD_EXPOSE_STATUS) { // busy expose  
    printf("ECALLBACK TYPE FPD STATUS: Exposure! \n");  
    Status=FPD_EXPOSE_STATUS;  
}  
  
else if (nlength==FPD_CONTINUE_READY) { // continue ready  
    printf("ECALLBACK TYPE FPD STATUS: Continue ready! \n");  
    Status=FPD_CONTINUE_READY;  
}  
  
else if (nlength==FPD_DWONLOAD_GAIN) { // download gain template  
    printf("ECALLBACK TYPE FPD STATUS: Download gain template ack! \n");  
    Status=FPD_DWONLOAD_GAIN;  
}  
  
else if (nlength==FPD_DWONLOAD-DEFECT) { // download defect template  
    printf("ECALLBACK TYPE FPD STATUS: Download defect template ack! \n");  
    Status=FPD_DWONLOAD-DEFECT;  
}  
  
else if (nlength==FPD_DWONLOAD_Offset) { // download offset template  
    printf("ECALLBACK TYPE FPD STATUS: Download offset template ack! \n");  
    Status=FPD_DWONLOAD_Offset;  
}  
  
else if (nlength==FPD_UPDATE_FIRMARE) { // update firmware  
    printf("ECALLBACK TYPE FPD STATUS: Update firmware! \n");  
    Status=FPD_UPDATE_FIRMARE;  
}  
  
else if (nlength==FPD_RETRANS_MISS) { // update firmware  
    printf("ECALLBACK TYPE FPD STATUS: Retransmission! \n");  
    Status=FPD_RETRANS_MISS;
```

```

    }
    else
        printf("ECALLBACK TYPE FPD STATUS, Err: Other error=% d \ n", nlength);
    if (status!=-1)
    {
        // update icon
        TheDemo ->PostMessage (WM_DETECTORA_CONNECT_STATUS, (WPARAM) status,
(LPARAM) 0);

        // disconnect
        if (nlength<=0&& nlength>=-10)
        {
            CFPD_ BASE_ CFG * pBaseCfg=TheDemo ->get_Base_Cfg_Ptr (0);
            if (pBaseCfg!=NULL)
            {
                if (pBaseCfg ->m_pFpdHand!=NULL)
                {
                    HBI_DisconnectDetector (TheDemo ->m_pFPD);
                    TheDemo ->UpdateUI();
                }
            }
        }
    }
}
break;

```

// If there is no return value or return value is 0, the SDK will consider feedback failed. Therefore, if the software receives a callback message from the SDK, it will return 1.

Return 1;

Table 31.

No	Callback Events	constant	information
1	ECALLBACK_TYPE_FPD_STATUS	0x09	Product status (nParam2), including normal status (>0) and abnormal status (<=0).
2	ECALLBACK_TYPE_ROM_UPLOAD	0x10	0: Network interface disconnected (such as disconnection command);
3	ECALLBACK_TYPE_FACTORY_UPLOAD	0x12	-1: Received abnormal data, resulting in disconnection
4	ECALLBACK_TYPE_SET_CFG_OK	0xab	-2: Failed to return the number of ready descriptors
5	ECALLBACK_TYPE_SINGLE_IMAGE	0x51	-3: Receive wait timeout
6	ECALLBACK_TYPE_MULTIPLE_IMAGE	0x52	-4: Abnormal reception
7	ECALLBACK_TYPE_PREVIEW_IMAGE	0x53	-5: Port unreadable
8	ECALLBACK_OVERLAY_NUMBER	0x5d	-6: Network card abnormality
9	ECALLBACK_OVERLAY_16BIT_IMAGE	0x5e	-7: Not used
10	ECALLBACK_OVERLAY_32BIT_IMAGE	0x5f	-8: Not used
11	ECALLBACK_TYPE_PACKET_MISS	0x5b	-9: Product disconnection
12	ECALLBACK_TYPE_THREAD_EVENT	0xa5	-10: Failed to read DDR address (optical fiber interface communication)
13	ECALLBACK_TYPE_ACQ_DISCARDED	0xa6	-11: Non Jumbo frame (communication data packet format mismatch)
14	ECALLBACK_TYPE_UPDATE_FIRMWARE	0x06	1: Prepare status, acknowledge of prepare command
15	ECALLBACK_TYPE_ERASE_FIRMWARE	0x07	2: Ready status, single frame acquisition completion or panel idle status
16	ECALLBACK_TYPE_FILE_NOTEXIST	0xae	3: Firmware Pre-offset template generation
17	ECALLBACK_TYPE_WLAN_BATTERY	0x13	4: Busy status, image acquisition process

3.5. Return Code

Please refer to the attachment "HbiError. h" for details on returning the code table.

Table 32.

No	Definition	constant	information
1	HBI_ERR_INVALID_PARAMS	8000	initialization failed

2	HBI_ERR_OPEN_DETECTOR_FAILED	8001	Failed to open detector
3	HBI_ERR_INVALID_PARAMS	8002	Invalid parameter
4	HBI_ERR_CONNECT_FAILED	8003	Connection failed
5	HBI_ERR_MALLOC_FAILED	8004	Memory allocation failed
6	HBI_ERR_RELIMGMEM_FAILED	8005	Release image resources
7	HBI_ERR_RETIMGMEM_FAILED	8006	Failed to return image buffer address
8	HBI_ERR_NODEVICE	8007	Device not found
9	HBI_ERR_DISCONNECT	8008	Unconnected
10	HBI_ERR_DEVICE_BUSY	8009	In the detector acquisition map
11	HBI_ERR_SENDDATA_FAILED	8010	Data sending failed
12	HBI_ERR_RECEIVE_DATA_FAILED	8011	Receiving data failed
13	HBI_ERR_COMMAND_DISMATCH	8012	Command mismatch
14	HBI_ERR_NO_IMAGE_RAW	8013	No image data
15	HBI_ERR_PTHREAD_ACTIVE_FAILED	8014	Failed to start listening thread
16	HBI_ERR_STOP_ACQUISITION	8015	Stop collection failed
17	HBI_ERR_INSERT_FAILED	8016	Insert data failed
18	HBI_ERR_GET_CFG_FAILED	8017	Failed to obtain flat panel parameters
19	HBI_NOT_SUPPORT	8018	Not currently supported
20	HBI_REGISTER_CALLBACK_FAILED	8019	Registration callback function failed
21	HBI_SEND_MESSAGE_FAILD	8021	Sending message failed
22	HBI_ERR_WORKMODE	8022	Switching test mode failed
23	HBI_FAILED	8023	fail
24	HBI_FILE_NOT_EXISTS	8024	file does not exist
25	HBI_COMM_TYPE_ERR	8025	Command type error
26	HBI_TYPE_IS_NOT_EXISTS	8026	This type does not exist
27	HBI_SAVE_FILE_FAILED	8027	Save image failed
28	HBI_INIT_PARAM_FAILED	8028	Failed to initialize parameters

29	HBI_INIT_FILE_NOT_EXIST	8029	The initialization file does not exist
30	HBI_INVALID_FLAT_PANEL	8030	Invalid flat panel type
31	HBI_INVALID_DLL_HANDLE	8031	Invalid handle object
32	HBI_FPD_IS_DISCONNECT	8032	Flat panel not connected
33	HBI_ERR_DETECTOR_NUMBER	8033	The ID of the detector is incorrect
34	HBI_ERR_DATA_CHECK_FAILED	8034	Data verification failed
35	HBI_ERR_CFG_IEMPTY	8035	The configuration parameter object is empty
36	HBI_ERR_OPEN_WIZARD_FAILED	8036	Failed to open template wizard
37	HBI_ERR_WIZARD_ALREADY_EXIST	8037	Template Wizard already exists
38	HBI_DIRECTORY_NOT_EXISTS	8038	The directory does not exist
39	HBI_UNREGISTER_CALLBACK	8039	Unregistered callback function
40	HBI_ERR_IMAGE_IEMPTY	8040	EMMC no image data (wireless)
41	HBI_ERR_ABNORMALMAL_IMAGE	8041	Abnormal image data
42	HBI_PREPARATION_COMPLETE	8042	Preparation completed
43	HBI_ERR_FILE_EXCEPTION	8043	File data exception, such as size mismatch or empty
44	HBI_ERR_COMM_TYPE	8044	Wrong communication type
45	HBI_ERR_INVALID_MODE	8045	Invalid Mode entry
46	HBI_ERR_FIRM_CONFIG_DATA	8046	The firmware parameters contain incorrect data
47	HBI_ERR_IS_NULL	8047	Pointer is empty
48	HBI_ERR_ROM_PARAMETER	8048	Incorrect ROM parameters
49	HBI_ERR_FILE_TYPE	8049	File type error
50	HBI_END	8050	Main monitoring thread exits

Error Message List:

Struct CodeStringTable

```
{
    int num;
```

```
int HBIRETCODE;

Const char * errString;

};

Static const CodeStringTable CrErrStrList{

    {0, HBI_SUCCSS, "Success"},

    {1, HBI-ERR-ININTFAILED, "Init DLL failed"},

    {2, HBI-ERR-OPEN-DETECTOR-FAILED, "Open device driver failed"},

    {3, HBI-ERR-INVALID6ARAMS, "Parameter error"},

    {4, HBI-ERR-CONNECT-FAILED, "Connect failed"},

    {5, HBI-ERR-MALLOC-FAILED, "Malloc memory failed"},

    {6, HBI-ERR-RELIMGMEMFAILED, "Releaseimagemem fail"},

    {7, HBI-ERR-RETIMGMEMFAILED, "ReturnImageMemo fail"},

    {8, HBI-ERR-NODEVICE, "No Init DLL Instance"},

    {9, HBI-ERR-DISCONNECT, "Is Disconnect"},

    {10, HBI-ERR-DEVICE-BUSY, "FPD is busy"},

    {11, HBI-ERR-SENDDATA-FAILED, "SendData failed"},

    {12, HBI-ERR-RECEIVE-DATA-FAILED, "Receive Data failed"},

    {13, HBI-ERR-COMMAND-DISMATCH, "Command mismatch"},

    {14, HBI-ERR-NO-IMAGE-RAW, "No Image raw"},

    {15, HBI-ERR-PTHREAD-ACTIVE-FAILED, "Pthread active failed"},

    {16, HBI-ERR-STOP-ACQUISITION, "Pthread stop data acquisition failed"},

    {17, HBI-ERR-INSERT-FAILED, "insert calibrate mode failed"},

    {18, HBI-ERR_Get-CFG-FAILED, "get Fcd config failed"},

    {19, HBI-NOT-SUPPORT, "not sharp yet"},

    {20, HBI-REGISTER-CALLBACK-FAILED, "failed to register callback function"},

    {21, HBI_SEND-MESSAGE-FAILD, "send message failed"},

    {22, HBI-ERR-WORKMODE, "switch work mode failed"},

    {23, HBI-FAILED, "operation failed"},
```



```
{24, HBI-FILE-NOT-EXISTS, "file does not exist"},
{25, HBI-COMM-TYPE.ERR, "communication is not existing"},
{ 26, HBI_TYPE_IS_NOT_EXISTS, "this type is not exists"},
{27, HBI-SAVE-FILE-FAILED, "save file failed"},
{28, HBI-INIT-PARAM-FAILED, "Init dll param failed"},
{29, HBI-INIT-FILE-NOT-EXIST, "Init dll config file is not existing"},
{30, HBI-INVALID-FLAT-PANEL, "Invalid flat panel detector"},
{31, HBI-INVALID-DLL_HANDLE, "Invalid DLL instance handle"},
{32, HBI-FPD-IS-DISCONNECT, "FPD is disconnected"},
{33, HBI-ERR-DETECTOR-NUMBER, "Detector number error"},
{34, HBI-ERR-DATA-CHECK-FAILED, "Data Check failed"},
{ 35, HBI_ERR_CFG_ISEMPY, "The configuration variable is empty"},
{36, HBI-ERR-OPEN-WIZARD-FAILED, "Open template wizard failed"},
{37, HBI-ERR-WIZARD-ALREADY-EXIST, "template wizard already exists"},
{38, HBI-DIRECTORY-NOT-EXISTS, "File directory is not existing"},
{39, HBI_UNREGISTER-CALLBACK, "Unregistered callback function"},
{40, HBI-ERR-IMAGE-ISEMPY, "Image is empty"},
{41, HBI-ERR-ABNORMMAL-IMAGE, "Abnormal image"},
{42, HBI-PREPARATION-COMPLETE, "Preparation complete"},
{43, HBI-ERR-FILE-EXCEPTION, "File exception"},
{44, HBI-ERR-COMM-TYPE, "Wrong communication type"},
{ 45, HBI_ERR_INVALID_MODE, "This mode contains incorrect parameters"},
{46, HBI-ERR-FIRM-CONFIG-DATA, "Firmware parameter data is abnormal"},
{47, HBI-ERR-IS-NULL, "The pointer is NULL"},
{48, HBI-ERR-ROM-PARAMETER, "ROM includes correct parameters"},
{49, HBI-ERR-FILE-TYPE, "File type exception"},
{50, HBI-END, "Exit monitoring"};
```

Please refer to the attachment "HbiError. h" for details.

/******

* Description: Get the error information by return code

* Function: GetHbiErrorString

* Parameter Descriptions: CodeStringTable* inTable // Error code table

* int count // Return error code number

* int recode // Return error code

* Return Value: const char * // Return string

* Remarks:

*****/

const char * GetErrString (const CodeStringTable * inTable, int count, int recode)

{

 const char * retString="Unknown";

 for (int i=0; i<count; i++){

 if (inTable [i]. HBIRETCODE==recode){

 retString=inTable [i]. errString;

 break;

 }

 }

 return retString;

}

int main()

{

 static const CodeStringTable CrErrStrList[] = {

 { 0, HBI_SUCCSS, "Success" },

 { 1, HBI_ERR_ININT_FAILED, "Init dll failed" },

 { 2, HBI_ERR_OPEN_DETECTOR_FAILED, "Open device driver failed" },

 { 3, HBI_ERR_INVALID_PARAMS, "Parameter error" },

```
{ 4, HBI_ERR_CONNECT_FAILED, "Connect failed" },
{ 5, HBI_ERR_MALLOC_FAILED, "Malloc memory failed" },
{ 6, HBI_ERR_RELIMGMEM_FAILED, "Release image memory faild" },
{ 7, HBI_ERR_RETIMGMEM_FAILED, "Return Image memory faild" },
{ 8, HBI_ERR_NODEVICE, "No Init DLL Instance" },
{ 9, HBI_ERR_DISCONNECT, "Is Disconnect" },
{ 10, HBI_ERR_DEVICE_BUSY, "Fpd is busy" },
{ 11, HBI_ERR_SENDDATA_FAILED, "SendData failed" },
{ 12, HBI_ERR_RECEIVE_DATA_FAILED, "Receive Data failed" },
{ 13, HBI_ERR_COMMAND_DISMATCH, "Command mismatch" },
{ 14, HBI_ERR_NO_IMAGE_RAW, "No Image raw" },
{ 15, HBI_ERR_PTHREAD_ACTIVE_FAILED, "Pthread active failed" },
{ 16, HBI_ERR_STOP_ACQUISITION, "Pthread stop data acquisition failed" },
{ 17, HBI_ERR_INSERT_FAILED, "insert calibrate mode failed" },
{ 18, HBI_ERR_GET_CFG_FAILED, "get Fpd config failed" },
{ 19, HBI_NOT_SUPPORT, "not surport yet" },
{ 20, HBI_REGISTER_CALLBACK_FAILED, "failed to register callback function" },
{ 21, HBI_SEND_MESSAGE_FAILED, "send message failed" },
{ 22, HBI_ERR_WORKMODE, "switch work mode failed" },
{ 23, HBI_FAILED, "operation failed" },
{ 24, HBI_FILE_NOT_EXISTS, "file does not exist" },
{ 25, HBI_COMM_TYPE_ERR, "communication is not exist"},
{ 26, HBI_TYPE_IS_NOT_EXISTS, "this type is not exists"},
{ 27, HBI_SAVE_FILE_FAILED, "save file failed"},
{ 28, HBI_INIT_PARAM_FAILED, "Init dll param failed"},
{ 29, HBI_INIT_FILE_NOT_EXIST, "Init dll config file is not exist"},
{ 30, HBI_INVALID_FLAT_PANEL, "Invalid flat panel detector"},
{ 31, HBI_INVALID_DLL_HANDLE, "Invalid dll instance handle"},
```

```
{ 32, HBI_FPD_IS_DISCONNECT, "Fpd is disconnect"},
{ 33, HBI_ERR_DETECTOR_NUMBER, "Detector number error"},
{ 34, HBI_ERR_DATA_CHECK_FAILED, "Data Check failed"},
{ 35, HBI_ERR_CFG_IEMPTY, "The configuration variable is empty"},
{ 36, HBI_ERR_OPEN_WIZARD_FAILED, "Open template wizard failed"},
{ 37, HBI_ERR_WIZARD_ALREADY_EXIST, "template wizard already exist"},
{ 38, HBI_DIRECTORY_NOT_EXISTS, "File directory is not exists"},
{ 39, HBI_UNREGISTER_CALLBACK, "Unregistered callback function"},
{ 40, HBI_ERR_IMAGE_IEMPTY, "Image is empty"},
{ 41, HBI_ERR_ABNORMMAL_IMAGE, "Abnormal image"},
{ 42, HBI_PREPARATION_COMPLETE, "Preparation complete"},
{ 43, HBI_ERR_FILE_EXCEPTION, "File exception"},
{ 44, HBI_ERR_COMM_TYPE, "Wrong communication type"},
{ 45, HBI_ERR_INVALID_MODE, "This mode contains incorrect parameters"},
{ 46, HBI_ERR_FIRM_CONFIG_DATA, "Firmware parameter data is abnormal"},
{ 47, HBI_ERR_IS_NULL, "The pointer is NULL"},
{ 48, HBI_ERR_ROM_PARAMETER, "ROM includes incorrect parameters"},
{ 49, HBI_ERR_FILE_TYPE, "File type exception"},
{ 50, HBI_END, "Exit monitoring"}};

int nCode = 0;

int count = sizeof(CrErrStrList) / sizeof(CodeStringTable);

const char *tmpString = GetErrString(CodeStringTable, count , nCode);

printf("nCode=%d,tmpString=%s\n", nCode, tmpString);

return 0;

}
```

4. Detector calibration

4.1. About Image Calibration

Calibration type, There are 4 kinds of image calibration for flat panel detector product, offset calibration, gain calibration, defect calibration and dummy calibration, while the dummy calibration is not available now. The following contents are mainly focus on offset calibration, gain calibration and defect calibration.

Table 33.

Calibration type	Remarks
Offset Calibration	Eliminating the physical fixed pattern from dark image
Gain calibration	Flatten the image due to the cone beam X-ray
Defect calibration	Calibrate physical defects of TFT sensor

Calibration template, there is a correction template for each calibration, the template saves all calibration coefficients for each pixel, the calibration algorithm will process the image based on these templates.

Table 34.

Calibration type	Template (hbi_id0_template, where id0 represents board A)
Offset Calibration	OffsetS02B1F00250.raw (panel, binning, frame rate, zoom)
Gain calibration	Gaina.raw and gainb.raw (panel, binning, PGA, KV)
Defect calibration	Defect.raw (panel, binning, PGA)

Software calibration and hardware calibration. "Software calibration" means all image calibration calculation process will be done through SDK, the software calibration option must be enabled before using it, It is suitable for low frame rate or single frame image acquisition due to the software resource cost. "hardware calibration" means all image calibration calculation process is done by firmware, as firmware can't generate these templates (gain, defect) by itself, so the gain and defect templates need to be generated by software and downloaded to detector in advance, this can help to reduce the resource consumption, so it is recommended to use firmware calibration for high frame rate application.

Template generation, the Offset template is generated from a set of dark images, the Gain template generation requires at least one set of bright images, and the Defect template generation requires at least three sets of bright images with different dose levels. For the bright image acquisition, it must be ensured that there is no imaging object in the active imaging area. The system integrator can use the DT tool software to generate these templates, another way is to call SDK API functions by system software, this can also improve the flexibility.

For offset template, it can be generated by software, this offset template can be downloaded to firmware too if firmware offset calibration enabled, also supporting firmware generates the offset template directly by

command (this function is supported by dynamic products, it can improve the offset template generation efficiency)

Template download is designed for dynamic high frame rate detectors. In order to reaching the maximum frame rate, the computer hardware resource consumption factor must be considered, moving the image calibration calculation process to firmware can reduce the software load, it can improve the image transmission integrity. Before enabling firmware calibration option, all of these templates must be generated and downloaded.

For offset template, it can be generated and downloaded to firmware, also supporting firmware generates the offset template directly by command (this function is supported by dynamic products, it can improve the offset template generation efficiency)

Calibration enable once the calibration templates are generated or downloaded, the next step needs to enable the calibration option.

Table 35.

Calibration type	Calibration enable status
Offset calibration	0 - Disable offset calibration 1 - Software Pre-offset calibration 2 - Firmware post offset calibration 3 - Frmware Pre-offset calibration
Gain calibration	0 - Disable gain calibration 1 - Software gain calibration 2 - Firmware gain calibration
Defect calibration	0 - Disable defect calibration 1 - Software defect calibration 2 - Firmware defect calibration

There are three calibration methods: Offset, Gain, and Defect. The implementation of the calibration function can be done through software or firmware. Users can choose whether to calibrate through software or firmware. The corresponding functions can be configured through configuration options. The list of supported modes for software and firmware calibration is as follows:

Table 36.

	Pre Offset	Post Offset	Gain	Defect
Software	√	×	√	√

Firmware	√	√	√	√
----------	---	---	---	---

Offset Calibration

Offset calibration must be performed with the X-ray turned off. During the offset template generation process, a series of dark field images will be acquired, the resulting averaged image is the offset calibration template and it is used to eliminate the inherent background deviation of each pixel in the detector.

Gain Calibration

The gain calibration process involves capturing a series of exposure images without any object in image field. (flat field) This is used to correct the image non-uniformity caused by the cone beam X-ray and inconsistent pixel response of the detector. Several exposure images under different exposure doses are acquired. The average exposure image for each dose is calculated and stored. A gain calibration template is then generated with gain template generation algorithm.

Defect Calibration

The defective pixel calibration process also need to acquire a series of exposure images without any object present (flat field). Several exposure images under different exposure doses are acquired. The average exposure image for each dose is calculated and stored. the defect template will be generated, the purpose of defect calibration is replacing the bad pixels' value with normal ones which can be implemented by simple replacing or interpolating.

1. Offset Calibration

Offset calibration can be divided into software pre-offset calibration, firmware post offset calibration, and firmware pre-offset calibration.

(1) Software pre-offset calibration

The software pre-offset calibration requires to generate the pre-offset template and save in local space in advance, a series discard (n) + reserved (m) image can be acquired by configuring software offset template wizard, once finishing these dark image acquisition, the pre-offset template can be generated by averaging the dark image stack together.

(2) Firmware pre-offset calibration

The firmware can directly generate the pre-offset calibration template by command, it also supports to generate the pre-offset template by software, then download it to firmware. As the offset template will be affected by frame rate, temperature and PGA gain, so it is easy to fail once these conditions have been changed, so command updating way is recommended considering the efficiency and usability.

(3) Firmware post offset calibration

The firmware post offset calibration does not need to prepare the offset template in advance, Instead, an extra dark image will be acquired for each exposure image once post offset calibration is enabled. This dark image is taken as the offset calibration template and will be subtracted from the exposure image.

2. Gain Calibration

The gain calibration supports software and firmware gain calibration two methods. The gain calibration template must be generated by software, this template must be downloaded to firmware in advance if firmware gain calibration will be used. The generation of a gain template requires acquiring multiple sets of bright field images with different exposure doses, for the simplified way, at least one set bright field is needed.

3. Defect Calibration

The defect calibration supports software and firmware defect calibration two methods. The defect calibration template must be generated by software, this template must be downloaded to firmware in advance if firmware defect calibration will be used. The generation of a defect template requires acquiring multiple sets of bright field images with different exposure doses, at least three sets of bright field images are needed.

Bad pixel classification:

(1) Bad pixels: Pixels which do not light response.

(2) Bad pixels: Nonlinear response pixels, the light response generated by pixels deviate significantly from other pixels.

Calibration sequence

raw image -> offset calibration -> gain calibration -> defect calibration -> display.

4.2. Quick Template Generation

"Quick template generation" is designed to generate template in a simplified way. This method aims to generate templates that are not only standard-compliant but also created efficiently. It is particularly suited for users involved in integration development. By simply calling the provided interface, users can quickly generate template files without needing to understand the details of the template generation process. The basic parameters required for generation are set to default values.

This interface is specially developed to quickly integrate and generate templates, simplify the difficulty of integrated development, and meet the needs of different development users. we will provide a brief explanation of the generation process of offset, gain and defect.

Note: When acquiring bright field images, there should be no object in imaging area, it is required an "empty field". Also to be sure that the full active imaging area will be fully covered by X-ray beam.

4.2.1. Offset Template

The offset template includes software Pre-Offset template, firmware Post-Offset template, and firmware Pre-Offset template. This template depends on parameters such as flat panel type, binning type, PGA gear, frame rate, and Zoom area. Changing any of these parameters will affect the offset template.

Table 37.

Offset template	Functions and parameters	Remarks
Software Pre Offset Template	Interface function: HBI_GenerateTemplate Parameter: SOFTWARE_OFFSET_TEMPLATE	Collect a set of unprocessed dark field images and generate a Pre-Offset template locally using the Offset template algorithm. After enabling the software Pre-Offset, the image will be automatically calibrated by the software Pre-Offset before being uploaded to the user. If the firmware is downloaded, enable the firmware Pre-Offset, and complete the Pre-Offset calibration before uploading the firmware to the dynamic library;
Firmware Post Offset Template	Enabling firmware Post Offset is sufficient	Common single frame acquisition, when the user enables the firmware Post Offset, after the exposure acquisition of one frame of bright field image is completed, the high-pressure ball tube is closed, and the firmware immediately collects one frame of dark field image (Post Offset template). The bright field image is calibrated with the dark field image through Post Offset, and then the image is uploaded to the upper computer;
Firmware Pre Offset Template	Interface function: HBI_GenerateTemplate Parameter: OFFSET_TEMPLATE_TYPE	Additionally, pre offset calibration can also be completed by uploading software generated pre offset templates to the firmware;

4.2.2. Gain Template

Gain templates are usually generated by the software side. If firmware Gain calibration is required, it is necessary to first generate a Gain template and download the firmware. This template depends on parameters such as flat panel type, binning type, PGA gear, high-pressure dose, and Zoom area. Changing any of these parameters will affect the Gain template. The generation of Gain templates relies on the Offset template, so update the Offset template first.

Table 38.

Gain template	Functions and parameters	Remarks
Gain template	Interface function: HBI_GenerateTemplate Parameter: GAIN_TEMPLATE_TYPE	Adjust the high-voltage dose (high-voltage, milliampere second and other parameters, because only one group of bright field images are collected, and the dose is taken normally), open the bulb tube, call the generation template interface after the bulb tube ray is stabilized, collect a

		group of bright field images and generate the Gain template, pay attention to the "empty field", and there should be no objects between the bulb tube and the flat plate.
--	--	---

4.2.3. Defect Template

Defect templates are usually generated by the software side. If firmware Defect calibration is required, it is necessary to first generate a Defect template and download the firmware. This template depends on parameters such as flat panel type, binding type, PGA level, high-pressure dose, and Zoom area. Changing any of these parameters will affect the Defect template. The generation of the Defect template relies on the Offset template, so the Offset template is updated first. The generation process of the Defect template is similar to that of the Gain template. The Defect template needs to collect at least three sets of bright field images, and the high-pressure dose of the other three sets of bright field images is different, for example.

Table 39.

Defect template	Functions and parameters	Remarks
Defect template	Interface function: HBI_GenerateTemplate Parameter: DEFECT_TEMPLATE_GROUP1	Normal high voltage, milliampere second adjustment normal 10%
	Interface function: HBI_GenerateTemplate Parameter: DEFECT_TEMPLATE_GROUP2	Normal high voltage, milliampere second adjustment normal 50%
	Interface function: HBI_GenerateTemplate Parameter: DEFECT_TEMPLATE_GROUP3	Normal high voltage, milliampere second adjustment normal

4.2.4. Template Download

Only when firmware calibration is required.

Note: The following firmware manages template storage and loading in a Binning type manner. In the "5.4 Multi Mode Management Integration" chapter, another template storage and management method for firmware will be introduced.

Steps:

1》 Register progress status callback function, download real-time status and results will be fed back to the upper computer through the callback function;

2» Call the download interface function and pay attention to input parameters, such as the download template file type, binning type, and Mode ID. Some template download interface functions also require passing in the absolute path of the template file;

Table 40.

Number	Interface functions	Remarks
1	HBI_RegProgressCallBack	Progress status callback function, pay attention to distinguishing the main callback function, refer to SDK Demo for details.
2	HBI_DownloadTemplate HBI_DownloadTemplateByType HBI_DownloadModeTemplate HBI_DownloadModeTemplateByType HBI_DownloadModeTemplateByIdAndType	Template download, input parameters are also different. Generally, input parameters include template type, absolute path of template file, Mode Id, and Binning type. Example: 1» Binning method, key parameter is Binning type, Mode Id can be ignored; 2» Multiple Mode mode, key parameter is Mode Id, Binning type can be ignored; 3» Individual functions require the absolute path of the template file;

4.2.5. Calibration Enable

After completing the template, it is necessary to reset the calibration enable. Only when the calibration enable is enabled will the image be calibrated based on these parameters, either through software calibration or firmware calibration.

Table 41.

Interface functions	Quasi enabling parameters
HBI_UpdateCorrectEnable	0x00- No calibration done
	0x01- Software offset
	0x02- Firmware post offset
	0x03- Firmware pre offset, but a pre offset template needs to be created first
	0x00- No calibration done
	0x01- Software gain
	0x02- Firmware gain, but the gain template needs to be downloaded to the flat panel first

	0x00- No calibration done
	0x01- Software defect
	0x02- Firmware defect, but the defect template needs to be downloaded to the flat panel first

In addition, there are other interface functions that can also update calibration enablement, such as

HBI_TriggerAndCorrectDisplay

HBI_TriggerPgaBinningAcqTimeCorrect

HBI_MTriggerAndCorrectDisplay

HBI_MTriggerPgaBinningAcqTimeCorrect

4.3. Template Wizard Resources

For users who want to generate offset, gain, and defect templates using their own system software, template wizard resources have been added to the DLL for smooth integration. Users can directly call the interface to open the wizard resource window, generate templates, configure parameters, and release resources. Bright field image acquisition should be performed in conjunction with exposure operations.

4.3.1. Open Template Wizard

Code:

```
int ret=HBI_OpenTemplateWizard (m_pFPD)// Open Wizard Interface Function
```

```
If (ret!=HBI_SUCCSS)
```

```
{
    If (HBI_ERR_NODEVICE==ret)
    {
        : AfxMessageBox (_T("warning: Dll not initialized!"));
    }
    else if (HBI_ERR_NODEVICE_TRY_CONNECT==ret)
    {
        : AfxMessageBox (_T("warning: disconnect!"));
    }
    else if (HBI_ERR_INVALID_PARAMS==ret)
    {
        : AfxMessageBox (_T("err: pRomCfg is NULL!"));
    }
    else if (HBI_ERR_OPEN_WIZARD_FAILED==ret)
```

```

{
    : AfxMessageBox (_T("err: open template wizard failed!"));
}
else if (HBI_ERR_WIZARD_ALREADY_EXIST==ret)
{
    : AfxMessageBox (_T("warning: template wizard already exists!"));
}
else
{
    : AfxMessageBox (_T("err: other error!"));
}
}

```

Users only need to call the interface to pop up the template wizard dialog box, generate and download offset, gain, and defect calibration templates according to the interface prompts, and support setting some key parameters to reduce the difficulty of user integration.

For specific usage, please refer to the XDiscovery tool user manual, which is similar to the template wizard module in XDiscovery.

4.3.2. Offset Template Generation Page

This chapter includes the procedures on generating software pre-offset template and firmware pre-offset template, only dynamic product supports firmware pre-offset calibration function, while the software pre-offset calibration is widely used by static products.

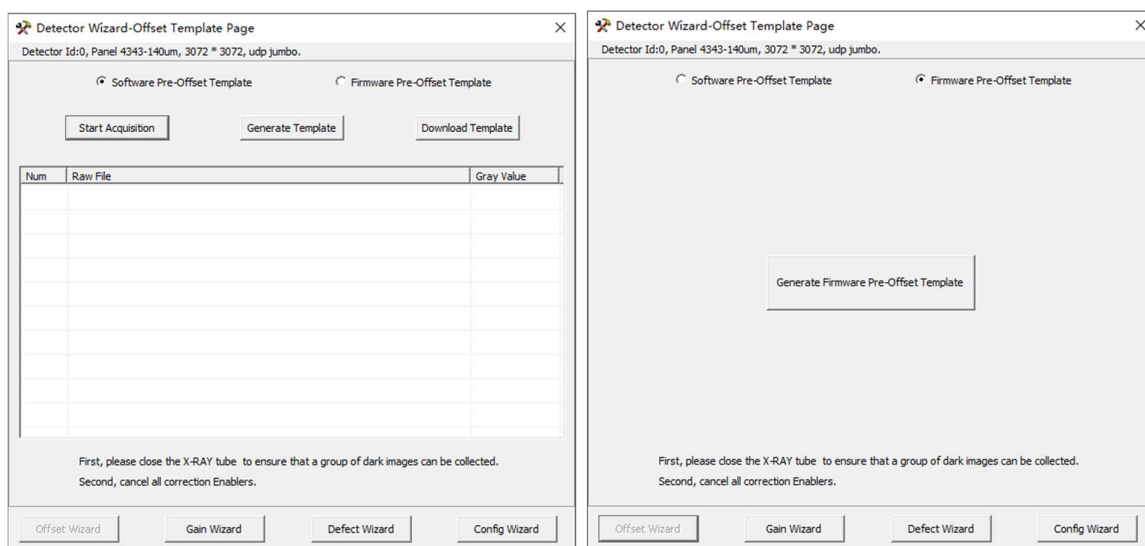


Figure 23

Start Acquisition -> Generate Template -> Download Offset Template -> End.

Firmware offset calibration also supports generating the offset template using software and downloading the template to the firmware.

4.3.3. Gain Template Generation Page

The SDK also supports generating gain calibration templates under different triggering modes. For static products, single-frame acquisition mode can be used for bright field image acquisition, while for dynamic products, continuous acquisition mode is recommended for bright field image acquisition. It is suggested to acquire images in three groups under different exposure conditions; the relationship between groups and exposure conditions for each group can be checked in the table. Typically, three images are acquired for each group, and at least one group is required.

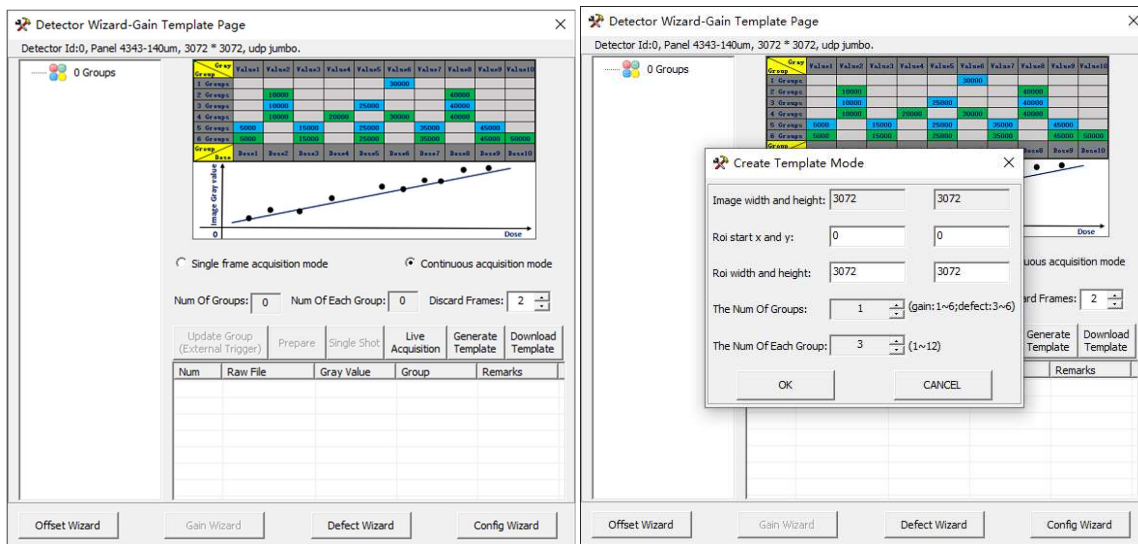


Figure 24

The tree control on the left and the list control in the small right corner use the right mouse button to open submenus for creating, deleting and clearing sub items.

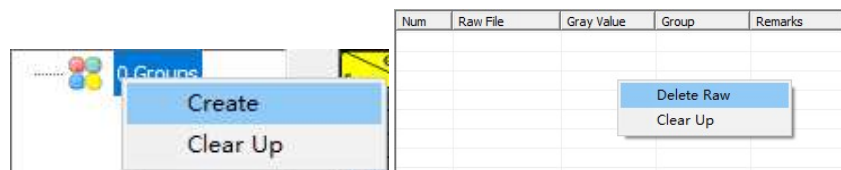


Figure 25

For example, the user has established three groups.

0 group, setting exposure condition with 1/6 saturation dose level, click left mouse button to choose 0 group item, then acquire three bright field images.

1 group, setting exposure condition with 1/2 saturation dose level, click left mouse button to choose 1 group item, then acquire three bright field images.

2 group, setting exposure condition with 4/6 saturation dose level, click left mouse button to choose 2 group item, then acquire three bright field images.

Then clicking generate template button to generate gain template, if firmware gain calibration will be used, also need to click the download template button to download the template to firmware.

4.3.4. Defect Template Generation Page

The SDK also supports generating defect calibration templates under different triggering modes. For static products, single-frame acquisition mode can be used for bright field image acquisition, while for dynamic products, continuous acquisition mode is recommended for bright field image acquisition. It is suggested to acquire images in three groups under different exposure conditions; the relationship between groups and exposure conditions for each group can be checked in the table. Typically, three images are acquired for each group, and at least three groups are needed.

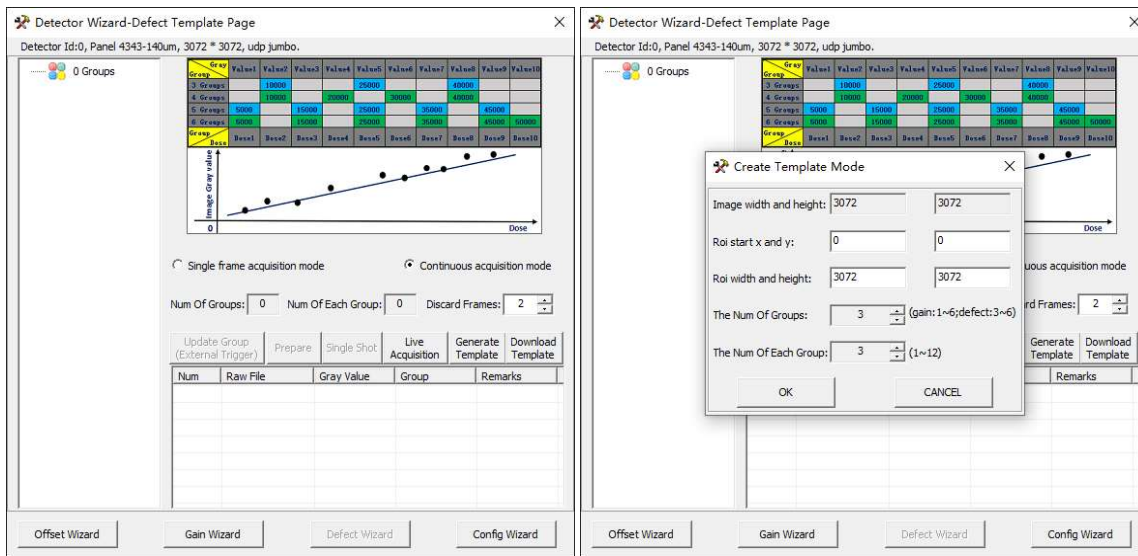


Figure 26

The tree control on the left and the list control in the small right corner use the right mouse button to open submenus for creating, deleting and clearing sub items.

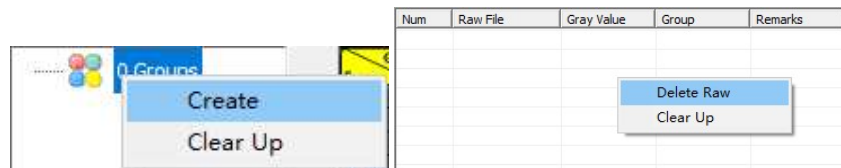


Figure 27

For example, the user has established three groups:

0 group, setting exposure condition with 1/6 saturation dose level, click left mouse button to choose 0 group item, then acquire three bright field images.

1 group, setting exposure condition with 1/2 saturation dose level, click left mouse button to choose 1 group item, then acquire three bright field images.

2 group, setting exposure condition with 4/6 saturation dose level, click left mouse button to choose 2 group item, then acquire three bright field images.

Then clicking generate template button to generate defect template, if firmware defect calibration will be used, also need to click the download template button to download the template to firmware.

4.3.5. Configuration Page

Mainly including functions:

View basic information of the detector (flat panel ID/type/resolution/communication method/product serial number/software firmware version number/current image size)

Set thumbnail mode (supported by some flat panels)

Set Binning Type

Set PGA gear

Set Prepare delay (control single frame acquisition mode)

Set Live Time (Static Flat panel Continuous Collection Time Interval)

Set Self Dumping Time (dynamic flat panel continuous acquisition time interval, i.e. frame rate)

Set trigger mode and enable calibration status

Restore factory settings, export flat panel parameters, and open SDK log directory

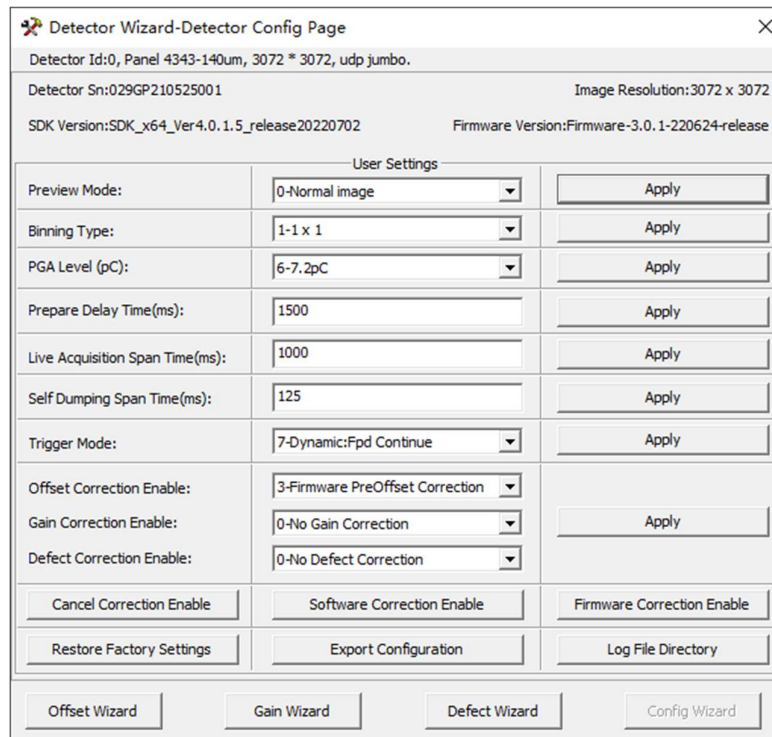


Figure 28

4.4. Template Management

If software calibration options are enabled, the software will search and load the template file on the local disk based on the parameters (provided the templates already exist) and implement software calibration calculation. While if firmware calibration options are enabled, it must to download the templates file to firmware in advance, these files will be saved in volatile and non-volatile memory two copies, all calibration calculation will be done by firmware.

4.4.1. Software Template Management

The local template path and template file name naming rules are shown in the table below. For detailed operations, please refer to the XDiscovery User Manual.

- Template root directory

Table 42.

Detector A (id: 0)	Detector B (id: 1)	Detector C (id: 2)	...	Detector N (id: n)
hbi_id0_template	hbi_id1_template	hbi_id2_template	...	hbi_idn_template

- Template directory and template file name

Taking Panel-1613 as an example							
Key letters							
S	B	P	V	Z	F	r	m
Panel size Reference tablet model information table	Binning Type 1-binning1x1 2-binning1x1 3-binning1x1 4-binning1x1	PGA Level 1-11 represents different PGA level, refer to PGA level table.	High Voltage (Dose parameters) Currently, due to the inability to obtain specific parameters, it defaults to 0.	ZOOM Rect Rect(left.top,width,height)	FPS (Affects offset templates) Dynamic:1000ms/dumping time Static:1000ms/living time Currently, for character alignment, it has been expanded by 1000 times	Rotate 1- Right Rotate 90° 2- Left Rotate 90° 0 or other default images do nothing	Mirror (1-Vertically Mirror,2-Horizontal Mirror) 1-Vertically Mirror 2-Horizontal Mirror 3-5 indicates that the firmware has completed different image processing, independent of template generation 0 or other default images are not
Offset template (16-bit raw file)				Notes			
Example 1 offsetS03B1PF620000				Eg. time interval=50ms, 20 frames represent 1000000/50=F20000, If the time interval is 2000ms, 0.5 frames represent F00500, the time interval is 100000, and 0.01 frames represent F00010.			
Example 2 offsetS03B1PF620000Z0000049612480032				Z0000049612480032 indicates the ZOOM rect parameter:left=0;top=496,width=1248,height=32			
Example 3 offsetS03B1PF620000Z0000049612480032_r1m2				r1m2 indicates that the image has been rotated right and mirrored horizontally			
Gain template (32-bit raw file)		Folder		Template file		Notes	
Example 1		S03B1P6V000		gaina.raw, gainb.raw		Firmware only updates gaina.raw Z0000049612480032 indicates the ZOOM rect parameter:left=0;top=496,width=1248,height=32	
Example 2		S03B1P6V000Z0000049612480032		gaina.raw, gainb.raw		Indicates that the image has been rotated right and horizontally mirrored and generate gain template.	
Example 3		S03B1P6V000		gaina_r1m2.raw, gainb_r1m2.raw		Indicates that the image has been rotated right and horizontally mirrored and generate gain template.	
		S03B1P6V000Z0000049612480032		gaina_r1m2.raw, gainb_r1m2.raw			
Defect template (16-bit raw file)		Folder		Template file		Notes	
Example 1		S03B1P6V000		defect.raw		The grayscale value of 65535 represents bad points, and 0 represents normal points Z0000049612480032 indicates the ZOOM rect parameter:left=0;top=496,width=1248,height=32	
Example 2		S03B1P6V000Z0000049612480032		defect.raw		Indicates that the image has been rotated right and horizontally mirrored and generate defect template.	
Example 3		S03B1P6V000		defect_r1m2.raw		Indicates that the image has been rotated right and horizontally mirrored and generate defect template.	
		S03B1P6V000Z0000049612480032		defect_r1m2.raw			
Notes							
1. The ZOOM parameter only supports binning1x1;							
2. The offset template is related to the panel size, binning type, PGA gear, frame rate, Zoom parameters, and software side rotation mirror parameters;							
3. The gain and defect templates are related to the panel size, binning type, PGA gear, Zoom parameters, and software side rotation mirror parameters;							
4. At present, the rotation mirror processing sequence is: rotate first and then mirror.							

激沃 Win

Figure 29

4.4.2. Firmware Template Management

The firmware side cannot generate gain calibration templates (Gain) and bad point calibration templates (Defect), so the template file for firmware calibration needs to be uploaded from the software side to the firmware.

At present, firmware has different ways of managing templates, as follows:

1» Binning method, key parameter is Binning type, Mode Id can be ignored.

Table 43.

Binning type	Binning 1 * 1	Binning 2 * 2	Binning 3 * 3	Binning 4 * 4
EMMC address	Address 1	Address 2	Address 3	Address 4
Template file	Offset template	Offset template	Offset template	Offset template
	Gain template	Gain template	Gain template	Gain template
	Defect template	Defect template	Defect template	Defect template

This method only uses the Binning type as a dependency parameter for storage and loading, so it is only suitable for simple applications that do not require template switching, but the integration is simple.

2» Multiple Mode mode, key parameter is Mode Id, Binning type can be ignored, $n \in [0,9]$;

Table 44.

Mode Id	Mode 0	Mode 1	Mode 2	Mode n	Mode 9
EMMC address	Address 1	Address 2	Address 3	Address n	Address 9
Template file	Offset template	Offset template	Offset template	Offset template	Offset template
	Gain template	Gain template	Gain template	Gain template	Gain template
	Defect template	Defect template	Defect template	Defect template	Defect template

At present, it supports up to 9 sets of Mode options, and users can set valid Mode options. Each Mode option contains different parameters, such as flat panel model, PGA gear, binning type, dosage, frame rate, Zoom area, and so on. This method is suitable for applications that switch templates in different scenarios. In the early stage, templates with the same Mode item need to be completed, and switching in the later stage is simple and fast.

Finally, users can choose different integration methods based on their actual business needs, and the dynamic library we provide is also compatible with these two methods.

5. Secondary Development

The SDK provides a full function access API package, user can access the detector and get the image and feedback through calling the corresponding API functions.

5.1. Interface Description

-hbi_fpd_ver *. * interface root directory

|---HBI_FPD_Tool: Tool software

| -Interface document for DT FPD Pro SDK API Programming Reference Ver *. *. pdf

| -Factory. csv factory configuration export file, used to restore factory settings

| -Win32 includes 32-bit XdDiscovery. exe, HBISDKApi. DLL, and 32-bit Opencv dependency libraries, as well as XML file bit engineering configuration files that can be created, saved, and modified.

| -X64 includes 64 bit Xdsidiscovery.exe and HBISDKApi.DLL, as well as 64 bit Opencv dependency library XML file bit engineering configuration files, which can be created, saved, and modified.

|---HBI_DLL: Interface file, including header file, dynamic library DLL, static library file, and configuration file

| -HBI_DLL \ INCLUDES: header file, please refer to the interface documentation for details.

| - 'HbiFpd.h': Interface function file, Export functions and instructions

| - 'HbiType.h': parameter, structure data type file

| - 'HbiError.h': Error List File

| -HBI_DLL \ BIN \ x86: 32bit dynamic library DLL, static library files, and configuration files

| -HBISDKApi.DLL and HBISDKApi.lib: 32-bit release dynamic and static library files

| -Opencv_*** : 32-bit dependency library

| -HBI_DLL \ BIN \ x64: 64bit dynamic library DLL, static library files, and configuration files

| -HBISDKApi.DLL and HBISDKApi.lib: 64 bit release dynamic and static library files

| -Opencv_*** : 64 bit dependency library

| -HBI_DLL \ Document: interface development document

|---SDK_Demo_Example: Interface development source code instance (C++ or C #), supported on VS2008 and above versions

| -SDK_ Demo_ SDK_ DEMO2008.7z

5.2. Software deployment

Taking C/C++ language development as an example. Copy the "HBI-DLL" file directory to the user development specified directory, configure the header file directory and library file directory, and pay attention to the 32 bit and 64 bit library files.

Taking SDK Demo as an example, the steps are as follows:

- 1》 Copy the 'HBI_DLL' folder to the "DemoDll \ DemoDll" or "Demolib \ Demolib" or "HB_SDK_DEMO2013\HB_SDK_DEMO2008" directory, where the header file directory HBI_DLL\INCLUDES, library file directory HBI_DLL \ BIN \ Win32, HBI_DLL \ BIN \ x64;
- 2》 The " HB_SDK_DEMO2013" project relies on the Opencv341 library and can directly copy "Opencv341 Unzip to D Drive. 7z" to D Drive for direct decompression. The decompression directory is D: \ opencv341 \ opencv;
- 3》 In some debugging modes, the dependency library opencv requires other libraries, such as msvcp100d. DLL or msvcr100d. DLL or msvcp120d. DLL or msvcr120d. DLL. You can find 32-bit or 64 bit library files in the " HB_SDK_DEMO2013_*\ output" directory, and pay attention to 32-bit and 64 bit, as shown in the "Dependency Library Screenshot. png". You can contact the R&D personnel to obtain them;
- 4》 The generated directories are: project directory \output\Win32 and project directory\output\x64, please refer to 'HB_ SDK_ DEMO2013' project for details;
- 5》 The 'HB_SDK_DEMO2008' project is developed based on MFC Dialogs, including display, template generation, etc., with the most comprehensive functions. DemoDll and Demolib are developed based on console console, including basic functions. DemoDll is a dynamically loaded Demo, and Demolib is a statically loaded Demo. Please refer to the project documentation for details, such as the "DT SDK Development DEMO Project Description Ver4.0. *. *. pdf";
- 6》 Users who need to integrate a "dual detectors" solution can contact after-sales or R&D personnel to request the development of DEMO engineering source code;
- 7》 In addition, if you need a C # SDK demo, please contact after-sales or R&D personnel;

5.3. Binning Integration

Please refer to the SDK Demo development program for details.

5.3.1. Steps

1. Include header files in user project

```
#Define_ DLL_ EX_ IM 0
```

```
#Include "HbiFpd.h"//Dynamic library header file
```

```
#Pragma comment (lib, "HBISDKApi. lib")
```

2. User defined devices

```
// Device handles and configurations
```

```
Void * m_pFpd; // Fpd sdk object handle

RegCfgInfo * m_pLastRegCfg ; // Firmware configuration

FPD_AQC_MODE aqc_mode; // Acquisition mode and parameters, single frame acquisition, multi frame
acquisition, dark field image and bright field image acquisition

// Image resolution

int m_imgW, m_imgH;

char m_path[MAX_PATH];

fpd_base_cfg_st *fpd_base;

IMAGE_PROPERTY img_pro;

IMAGE_CORRECT_ENABLE* pCorrect;

// Some basic data types

m_pLastRegCfg = new RegCfgInfo;

pCorrect = new IMAGE_CORRECT_ENABLE;

fpd_base = new fpd_base_cfg_st;
```

The device list structure has been defined in 'HbiType. h'.

3. Interface call process

```
// Initialize DLL resources

Void * m_pFpd =HBI_Init();

if (m_pFpd == NULL) {

printf("HBI_Init init failed!\n");

return false;

}

// Registering Callbacks

//MyCallbackFun callback function

int ret=HBI_RegEventCallBackFun (m_pFpd, myCallbackFun);

if (ret != 0) {

printf("HBI_RegEventCallBackFun failed!\n");

HBI_Destroy(m_pFpd);

m_pFpd = NULL;
```

```
        return false;

    }

    // Connect the flat panel detector

    COMM_CFG commCfg;

    Bool doOffsetTemps=false; // enable or disable firmware pre-offset template update

    //1. PCIe mode

    CommCfg_type= FPD_COMM_TYPE::PCIE_COMM_TYPE;

    CommCfg_pciid =7024; // 7024, 7024 are the PCIe card hardware ID

    #If 0

    //2. UDP mode

    //commCfg.type = FPD_COMM_TYPE:: UDP_COMM_TYPE;

    //strcpy(commCfg.localip, "192.168.10.20");

    //strcpy(commCfg.remoteip, "192.168.10.40");

    //commCfg._localPort = 32896;

    //commCfg._remotePort = 32897;

    //3. UDP JUMBO mode

    //commCfg_type = FPD_COMM_TYPE:: UDP_JUMBO_COMM_TYPE;

    //strcpy(commCfg.localip, "192.168.10.20");

    //strcpy(commCfg.remoteip, "192.168.10.40");

    //commCfg._localPort = 32896;

    //commCfg._remotePort = 32897;

    #Endif

    // DoOffsetTemplate - false: The firmware does not update offset template after a successful connection, true:
    The firmware updates an offset template after a successful connection

    ret=HBI_Connect Detector (m_pFpd, commCfg, doOffsetTemps);

    // ret=HBI_Connect Detector (m_pFpd, commCfg) // After a successful connection, the firmware does not
    update offset template

    if (ret != 0) {

        printf("Connection failed! \n");
```

```
    HBI_Destroy(m_pFpd);

    m_pFpd = NULL;

    return false;
}

// Set trigger mode and enable image correction
Int_triggerMode=7// 1-Soft trigger, 3-High voltage trigger, 4-freeAED,
..., 7-DYNAMIC_ FPD_ Continuue_ MODE
IMAGE_CORRECT_ENABLE * pCorrect=new IMAGE_CORRECT_ENABLE;
if (pCorrect!=NULL){

    Pccorrect ->bFeedbackCfg=true // True ECALLBACK_TYPE_ROM_UPLOAD Event, false
    ECALLBACK_TYPE_SET_CFG_OK Event

    Pccorrect ->ucOffsetCorrection=0x03 // 00- "Do nothing"; 01- "Prepare offset correction"; 02- "Post
    offset correction";

    Pccorrect ->ucGainCorrection=0x02 // 00- "Do nothing"; 01- "Software gain correction"; 02- "Hardware
    gain correction"

    Pccorrect ->ucDefectCorrection=0x02 // 00- "Do nothing"; 01- "Software defect correction"; 02-
    "Software defect correction"

    Pccorrect ->ucDummyCorrection=0x00 // 00- "Do nothing"; 01- "Software dummy correction"; 02-
    "Software dummy correction"

    ret=HBI_TriggerAndCorrectApplay (m_pFpd, _triggerMode, pCorrect);

    if (ret!=0){

        printf("Connection failed! \n");

    }

}

// disconnect
HBI_DisconnectDetector (m_pFpd);

// Release resources (including disconnection and resource release)
HBI_Destroy (m_pFpd);

m_pFpd =NULL;
```

Note:

1. After a successful connection, the product will automatically feedback the product ROM parameters.
2. 'HBI_Init' and 'HBI_Destroy': initializes and releases device.
3. HBI_ConnectDetector and HBI_DisconnectDetector: Connect and disconnect the flat panel detector.
4. Connect the detector using 'HBI_ConnectDetector', callback event ECALLBACK_TYPE_ROM_UPLOAD provides feedback on the current firmware parameters.
5. 'HBI_GetSystemConfig' and 'HBI_GetFpdCfgrInfo': Read all system configuration parameters.

'HBI_GetSystemConfig' is an asynchronous API function which sends parameters read command to detector.

'HBI_GetFpdCfgrInfo' is a synchronization API function which gets parameters after a successful connection or setting.

6. 'HBI_TriggerAndCorrectApply', After a successful connection, initialize the basic control parameters, such as trigger mode and calibration enable.

7. HBI_Destroy releases resources, handle is NULL, if closed directly, call 'HBI_Destroy' function, this function will call 'HBI_DisConnectDetector' internally.

8. Define callback functions.

```
#####
```

```
# Callback function and event description:
```

```
// Function description: Callback function definition, handling configuration, images, and product status.
```

```
// Parameter description:
```

```
// @pContext: Parameter 1, software object pointer, can be null
```

```
// @ ufpdId: Parameter 2, such as panel ID
```

```
// @byteEventId: Parameter 3, Event ID, refer to enum eCallbackEventCommType in 'HbiType.h'
```

```
// @ pvParam1: Parameter 4, configure structure pointer or image structure pointer
```

```
// @nParam2: Parameter 5, such as data size (image data is the length of the image buffer)
```

```
// @param3: Parameter 6, such as frame number, frame ID
```

```
// @param4 Parameter 7, such as frame rate
```

```
// Callback function return value description: The return values are integer 0 or 1, 1- Received message, 0- Not received message
```

```
#####
```

```
int CDemoDlg:: myCallbackFun (void * _context, int nfpdId, unsigned char command, void * pvParam1, int nParam2, int param3, int param4)
```

```
{
```



```
int ret=0;

if ((command == ECALLBACK_TYPE_SINGLE_IMAGE) ||

    (command == ECALLBACK_TYPE_MULTIPLE_IMAGE) ||

    (command == ECALLBACK_TYPE_ROM_UPLOAD))

{
    if (buff == NULL || len == 0){

        printf("Registration callback function parameter exception! \n");

        return ret;

    }

}

//

int status = -1;

int j = 0;

ret = 1;

switch (command)

{

    case ECALLBACK_TYPE_FPD_STATUS://Panel status: connected / disconnected / ready / busy

        printf("ECALLBACK_TYPE_FPD_STATUS, recode=% d \n", len);

    }

    break;

    case ECALLBACK_TYPE_ROM_UPLOAD:/* Update configuration*/

        printf("ECALLBACK TYPE ROM_UPLOAD: \n");

        break;

    case ECALLBACK_TYPE_SET_CFG_OK:

        printf("ECALLBACK TYPE SET CFG OK: \n");

        break;

    case ECALLBACK_TYPE_SINGLE_IMAGE: // Single frame image acquisition

    case ECALLBACK_TYPE_MULTIPLE_IMAGE: // Continuous image acquisition
```

```
        break;

    case ECALLBACK_OVERLAY_NUMBER: // Overlying frame number, such as averaging 8 frames, now acquiring
    the 5th frame.

    break;

    case ECALLBACK_OVERLAY_16BIT_IMAGE: // After image overlying, update the averaged result image (16 bits)

    break;

    .....

    case ECALLBACK_TYPE_THREAD_EVENT:

    default:

        printf("command=%02x \n", command);

        break;

    }

    return ret;
}
```

Return the image properties, length, width, number of bits, type, size information, trigger mode, firmware calibration enable, etc. through callback functions.

Return the image data address and length through a callback function..

5.3.2. Offset Template Generation and Download

Firstly, connect the flat panel detector and update the flat panel control parameters, such as binning type, PGA gear, frame rate, and Zoom area.

Collect 1 set of dark field images and generate a template until success is returned, otherwise failed is returned.

```
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE::OFFSET_TEMPLATE_TYPE;
```

```
int ret = HBI_GenerateTemplate(m_pFpd, enumTemplateType);
```

```
if (ret!=HBI_SUCCSS) {}//failed
```

```
else {}//Success
```

The offset template generated by the firmware generally does not need to be downloaded. In some scenarios, the high-pressure ball tube continuously emits X-rays, and the software needs to generate calibration in advance and download it to the firmware.

// To register a callback function, it is necessary to register and provide feedback on progress or result information

//Callback function: USER_CALLBACK_HANDLE_PROCESS handFun - Global function or static member function, refer to HbiType. h

//PContext - Pointer to the upper computer object, which can also be assigned as null if not needed by the user.

If (0!=HBI-RegProgressCallBack (m_pFPD, DownloadCallBackFun, (void *) this))

{

printf("err: HBI-RegProgressCallBack failed! \n");

Return;

}

// parameters

//if (downloadfile == NULL) downloadfile = new DOWNLOAD_FILE;

//if (downloadfile == NULL)

//{

// printf("err:malloc downloadfile failed!\n");

// return;

//}

//downloadfile->emfiletype = OFFSET_TMP;

// downloadfile->nBinningtype = 1;// eg, the binningtype of template is binning1x1

//int j=sprintf(downloadfile ->filepath, "%s \\ hbi_id0_template \ offsetS * * B * P * F ***** raw", theDemo ->d_path);

//downloadfile->filepath[j] = '\0';

// start download the template of Pre-offset

//int ret = HBI_DownloadTemplate(m_pFpd, downloadfile);

//if (ret != HBI_SUCCSS)

// ::AfxMessageBox(_T("err:Download offset-template failed!"));

//else

// ::AfxMessageBox(_T("Download offset-template success!"));

// Release resources

```
//Delete pUploadFile;

//PUUploadFile=NULL;

or

// start download the template of Pre-offset

int infiletype=2// Download file types 0-gain template, 1-defect template, 2-offset template, others - not
supported

int ret=HBI_DownloadTemplateByType (d_pFPD, infiletype)// Suggested use

If (ret!=HBI_SUCCSS)

    : AfxMessage    Box    (T ("err: Download offset template failed!"));

else

    : AfxMessage    Box    (T ("Download offset template success!"));

// Update offset calibration enable, currently enabling firmware Pre offset calibration

IMAGE_ CORRECTION_ENABLE * pccorrect=new IMAGE_ CORRECTION_ENABLE;

If (pccorrect!=NULL)

{

    pccorrect ->bFeedbackCfg=false;

    pccorrect ->ucOffsetCorrection=0x03// 0x00- no calibration, 0x01- software offset, 0x02- firmware
pose offset, 0x03- firmware pre offset offset, dynamic flat panel uses firmware pre offset, which is 0x03

    pccorrect ->ucGainCorrection=0x00// 0x00- no calibration, 0x01- software gain, 0x02- firmware gain,
but the gain template needs to be downloaded to the flat panel first

    pccorrect ->ucDefectCorrection=0x00// 0x00- No calibration, 0x01- Software defect, 0x02- Firmware
defect, but the defect template needs to be downloaded to the flat panel first

    pccorrect ->ucDummyCorrection=0x00// 0x00- No calibration, 0x01- Currently not supported, 0x02-
Firmware currently not supported, currently not supported by software firmware

    ret=HBI_UpdateCorrectEnable (theDoc ->m_pFPD, pccorrect);

    if (ret==0)

        // successful

    else

        // failed

    //Release resources

    delete correct;
```

```
pccorrect=NULL;

}
```

5.3.3. Gain Template Generation and Download

Firstly, ensure that the flat panel detector is connected and update the flat panel control parameters, such as binning type, PGA gear, high-pressure dose, and Zoom area.

```
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE::GAIN_TEMPLATE_TYPE;

int ret=HBI_GenerateTemplate (d_pFPD, enumTemplateType);

If (ret!=HBI_SUCCSS) {}//failed

else {}//Success

// register a callback function, it is necessary to register and provide feedback on progress or result
information

//Callback function: USER_CALLBACK_HANDLE_PROCESS handFun - Global function or static member function,
refer to HbiType. h

//PContext - Pointer to the upper computer object, which can also be assigned as null if not needed by the
user.

If (0!=HBI_RegProgressCallBack (m_pFPD, DownloadCallBackFun, (void *) this))

{

    printf("err: HBI_RegProgressCallBack failed! \n");

    return;

}

// Parameters

//if (downloadfile == NULL) downloadfile = new DOWNLOAD_FILE;

//if (downloadfile == NULL)

//{{

//    printf("err:malloce downloadfile failed!\n");

//    return;

//}}

//

//downloadfile->emfiletype = GAIN_TMP;

// downloadfile->nBinningtype = 1;// eg, the binningtype of template is binning1x1
```

```
//int j=sprintf(downloadfile ->filepath, "%s\\hbi_id0_template\\S**B*P*V***\\gaina.raw", theDemo
->m path);

//downloadfile->filepath[j] = '\0';

// Method 1 :start download the template of Gain

//int ret = HBI_DownloadTemplate(m_pFpd, downloadfile);

//if (ret != HBI_SUCCSS)

//      ::AfxMessageBox(_T("err:Download gain-template failed!"));

//else

//      ::AfxMessageBox(_T("Download gain-template success!"));

// free resources

//Delete pUploadFile;

//PUploadFile=NULL;

or

// Method 2:start download the template of Gain

int infiletype=0// Download file types 0-gain template, 1-defect template, 2-offset template, others - not
supported

int ret=HBI_DownloadTemplateByType (d_pFPD, infiletype)// Suggested use

if (ret!=HBI_SUCCSS)

    : AfxMessageBox (_ T ("err: Download gain template failed!"));

else

    : AfxMessageBox (_ T ("Download gain template success!"));

//Update gain calibration enable, currently enabling firmware gain calibration

IMAGE_ CORRECTION_ENABLE * pccorrect=new IMAGE_ CORRECTION_ENABLE;

if (pccorrect!=NULL)

{

    pccorrect ->bFeedbackCfg=false;

    pccorrect ->ucOffsetCorrection=0x03 // 0x00- no calibration, 0x01- software offset, 0x02- firmware
pose offset, 0x03- firmware pre offset offset, dynamic flat panel uses firmware pre offset, which is 0x03

    pccorrect ->ucGainCorrection=0x02 // 0x00- no calibration, 0x01- software gain, 0x02- firmware gain,
but the gain template needs to be downloaded to the flat panel first
```

pccorrect ->ucDefectCorrection=0x00// 0x00- No calibration, 0x01- Software defect, 0x02- Firmware defect, but the defect template needs to be downloaded to the flat panel first

pccorrect ->ucDummyCorrection=0x00// 0x00- No calibration, 0x01- Currently not supported, 0x02- Firmware currently not supported, currently not supported by software firmware

```
Ret=HBI_UpdateCorrectEnable (theDoc ->m_pFPD, pccorrect);

if (ret!=HBI_SUCCSS) {}//failed

else {}//Success

//Release resources

delete correct;

pccorrect=NULL;

}
```

5.3.4. Defect Template Generation and Download

First, make sure the flat panel detector is connected. Update the flat panel control parameters, including binning type, PGA gear, high-pressure dose, and zoom area.

Collect three sets of bright field images, which will require cooperation with a high-voltage generator. After the X-ray tube radiation stabilizes, trigger the command to complete template generation.

1》 Normal high voltage, milliampere second adjustment normal 10%

```
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE: DEFECT_ACQ_GROUP1;
```

```
int ret=HBI_GenerateTemplate (m_pFPD, enumTemplateType);
```

```
if (ret!=HBI_SUCCSS) {}//failed
```

```
else {}//Success
```

Note: As it is a firmware pre offset template, there is no need to download it to the firmware again.

2》 Normal high voltage, milliampere second adjustment normal 50%

```
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE: DEFECT_ACQ_GROUP2;
```

```
int ret=HBI_GenerateTemplate (d_pFPD, enumTemplateType);
```

```
if (ret!=HBI_SUCCSS) {}//failed
```

```
else {}//Success
```

3》 Normal high voltage, milliampere second adjustment normal

```
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE: DEFECT_ACQ_GROUP3;

int ret=HBI_GenerateTemplate (m_pFPD, enumTemplateType);

if (ret!=HBI_SUCCSS) {}//failed

else {}//Success

/*

* After the template is successfully generated, start downloading the template

*/

// register a callback function, it is necessary to register and provide feedback on progress or result
information

//Callback function: USER_CALLBACK_HANDLE_PROCESS handFun - Global function or static member function,
refer to 'HbiType. h'

//PContext - Pointer to the upper computer object, which can also be assigned as null if not needed by the
user.

If (0!=HBI_RegProgressCallBack (m_pFPD, DownloadCallBackFun, (void *) this))

{

    printf("err: HBI_RegProgressCallBack failed! \n");

    Return;

}

// parameter

//if (downloadfile == NULL) downloadfile = new DOWNLOAD_FILE;

//if (downloadfile == NULL)

//{

//    printf("err:malloce downloadfile failed!\n");

//    return;

//}

//

//if (downloadfile->emfiletype != DEFECT_TMP)downloadfile->emfiletype = DEFECT_TMP;

// downloadfile->nBinningtype = 1;// eg, the binningtype of template is binning1*1

//int j = sprintf(downloadfile->filepath, "%s\\hbi_id0_template\\S**B*P*V***\\defec.raw",m_path);

//downloadfile->filepath[j] = '\0';
```



```
// method 1: start download the template of defect

//int ret = HBI_DownloadTemplate(m_pFpd, downloadfile);

//if (ret != HBI_SUCCSS) {

//    ::AfxMessageBox(_T("err:Download defect template failed!"));

//}

//else

//    ::AfxMessageBox(_T("Download defect template success!"));

// free resource

//Delete pUploadFile;

//PUploadFile=NULL;

or

// method 2: start download the template of defect

int infiletype=1// Download file types 0-gain template, 1-defect template, 2-offset template, others - not
supported

int ret=HBI_DownloadTemplateByType (d_pFPD, infiletype)// Suggested use

If (ret!=HBI_SUCCSS)

    : AfxMessage    Box    (_T ("err: Download defect template failed!"));

else

    : AfxMessage    Box    (_T ("Download defect template success!"));

//Update defect calibration enable, currently enabling firmware defect calibration

IMAGE_CORRECTION_ENABLE * pccorrect=new IMAGE_CORRECTION_ENABLE;

If (pccorrect!=NULL)

{

    pccorrect ->bFeedbackCfg=false;

    pccorrect ->ucOffsetCorrection=0x03// 0x00- no calibration, 0x01- software offset, 0x02- firmware
pose offset, 0x03- firmware pre offset offset, dynamic flat panel uses firmware pre offset, which is 0x03

    pccorrect ->ucGainCorrection=0x02// 0x00- no calibration, 0x01- software gain, 0x02- firmware gain,
but the gain template needs to be downloaded to the flat panel first

    pccorrect ->ucDefectCorrection=0x02// 0x00- No calibration, 0x01- Software defect, 0x02- Firmware
defect, but the defect template needs to be downloaded to the flat panel first
```

pccorrect ->ucDummyCorrection=0x00// 0x00- No calibration, 0x01- Currently not supported, 0x02- Firmware currently not supported, currently not supported by software firmware

```
ret=HBI_UpdateCorrectEnable (theDoc ->m_pFPD, pcorrect);

if (ret==0)

    // successful

else

    // failed

//Release resources

delete correct;

pccorrect=NULL;

}
```

5.3.5. Progress Status Callback Function

Before downloading the template, be sure to register the progress and result callback functions, which can provide basic status and results during the template generation process.

// Download template callback function: static member function or global function

//InContext: Upper computer object pointer, can be NULL.

Static int DownloadCallBackFun (unsigned char command, int code, void * inContext)

```
{

    ///CTemplateTool * ptr=(CTemplateTool *) inContext;

    ///if (ptr==NULL){

    ///printf("err: inContext is NULL! \n");

    ///return 0;

    ///}

    ///if (TheDemo==NULL){

    ///printf("err: TheDemo is NULL! \n");

    ///return 0;

    ///}

    int len=0;

    switch (command)
```

```
{
case ECALLBACK_UPDATE_STATUS_START://Download begins

    printf("DownloadCallBackFun: ECALLBACK UPDATE-STATUS-START: Download start \n");

    break;

case ECALLBACK_UPDATE_STATUS_PROGRESS://Download progress. Code as percentage (0-100)

    printf("DownloadCallBackFun: ECALLBACK UPDATE-STATUS-PROGRESS:%d \n", code);

    break;

case ECALLBACK_UPDATE_STATUS_RESULT://Feedback results and errors

    if (0<=code)&&(code<=6)//Success

    {

        if (code==0) {//Download the Offset template

            printf("DownloadOffsetCBFun:%s \n", "Download Offset template! \n");

        }

        else if (code==1) {//Download the Gain template

            printf("DownloadOffsetCBFun:%s \n", "Download Gain template! \n");

        }

        else if (code==2) {//Download the Defect template

            printf("DownloadOffsetCBFun:%s \n", "Download Defect template! \n");

        }

        else if (code==3) {//Offset template download completed

            printf("DownloadOffsetCBFun:%s \n", "Offset template download completed! \n");

        }

        else if (code==4) {//Gain template download completed

            printf("DownloadOffsetCBFun:%s \n", "Download gain finish! \n");

        }

        else if (code==5) {//Defect template download completed

            printf("DownloadOffsetCBFun:%s \n", "Defect template download completed! \n");

        }

        else/* if (code==6) */{//Download successful and exit thread
```

```
        printf("DownloadOffsetCBFun:% s \ n", "Download success and exit thread! \ n");
    }
}
else//Failed
{
    if (code==1){
        printf("err: DownloadOffsetCBFun:% s \ n", "wait event other error! \ n");
    }
else if (code==2){
        printf("err: DownloadOffsetCBFun:% s \ n", "timeout! \ n");
    }
    else if (code==3){
        printf("err: DownloadOffsetCBFun:% s \ n", "downloaded offset failed! \ n");
    }
    else if (code==4){
        printf("err: DownloadOffsetCBFun:% s \ n", "downloaded gain failed! \ n");
    }
    else if (code==5){
        printf("err: DownloadOffsetCBFun:% s \ n", "downloaded defect failed! \ n");
    }
    else if (code==6){
        printf("err: DownloadOffsetCBFun:% s \ n", "Download failed and exit thread! \
n");
    }
    else if (code==7){
        printf("err: DownloadOffsetCBFun:% s \ n", "read offset failed! \ n");
    }
    else if (code==8){
        printf("err: DownloadOffsetCBFun:% s \ n", "read gain failed! \ n");
    }
}
```

```

    }
    else if (code=-9){
        printf("err: DownloadOffsetCBFun:% s \ n", "read defect failed! \ n");
    }
    else{
        printf("err: DownloadOffsetCBFun: unknown error,% d \ n", code);
    }
}
break;
default://unusual
    printf("DownloadCallBackFun: unusual, retcode=% d \ n", code);
    break;
}
return 1;
}

```

5.3.6. Update Calibration Setting

After completing the calibration template, in order to prevent the automatic cancellation of calibration enable during the template generation process, please update the calibration enable again after the template generation process is completed.

```
IMAGE_CORRECTION_ENABLE * pccorrect=new IMAGE_CORRECTION_ENABLE;
```

```
if (pccorrect!=NULL)
```

```
{
```

```
    pccorrect ->bFeedbackCfg=true// Feedback firmware parameters to the upper computer, false only
    returns confirmation information
```

```
    pccorrect ->ucOffsetCorrection=0x03// 0x00- no calibration, 0x01- software offset, 0x02- firmware
    pose offset, 0x03- firmware pre offset offset, dynamic flat panel uses firmware pre offset, which is 0x03
```

```
    pccorrect ->ucGainCorrection=0x02// 0x00- no calibration, 0x01- software gain, 0x02- firmware gain,
    but the gain template needs to be downloaded to the flat panel first
```

```
    pccorrect ->ucDefectCorrection=0x02// 0x00- No calibration, 0x01- Software defect, 0x02- Firmware
    defect, but the defect template needs to be downloaded to the flat panel first
```

pccorrect ->ucDummyCorrection=0x00// 0x00- No calibration, 0x01- Currently not supported, 0x02- Firmware currently not supported, currently not supported by software firmware

```
ret=HBI_UpdateCorrectEnable (theDoc ->m_pFPD, pcorrect);

If (ret==0)

    // successful

else

    // failed

//Release resources

Delete correct;

Pccorrect=NULL;

}
```

In addition, some parameters can affect the calibration template, such as Binning type, PGA level, frame rate, high pressure dose, and ZOOM parameters, all of which can cause the template to fail. Therefore, if the above parameters are changed, it is best to update the calibration template to prevent loss of calibration.

5.4. Multi-template Management and Integration

Users often experience "anomalies" in the current image calibration due to switching between several (1 or more) flat panel control parameters. The specific reason is that the calibration template does not match (i.e., the phenomenon of image misalignment). The most direct way to solve this problem is to regenerate the calibration template file under the current flat panel control parameter conditions. However, the process of generating templates often requires certain skills and relevant professional knowledge,

In order to cooperate with the multi template management of firmware calibration (or hardware calibration), the latest DLL supports user multi mode management, making it convenient for users to quickly switch between different mode items. Mode items synchronize and control detector image acquisition in real-time based on parameters. In addition, the correction and parameter setting process is simplified. For details, please refer to the "DT SDK Image Calibration User Mode and Multi Template Calibration Management Integration Explanations. pdf".

What are the main contents of multi template management?

1>> Enable multiple Mode management status parameters, which can be distinguished and compatible with Binning storage methods;

2>> The maximum number of Mode list items is 10, which can support 10 different templates. Often, customers do not need to manage many templates. The number of Mode items can be dynamically managed, and interface functions for modification are provided;

3>> The data of the Mode item can be edited and updated. The Mode item includes parameters such as Mode ID, Binning type, PGA level, dose value, ZOOM area parameters, and template status (all valid, all invalid, dark and bright field template status);

4>> Mode item switching, template downloading, etc.

The following mainly describes the process of multi template calibration management and integration from two steps: user mode update and user mode switching. User mode update is a prerequisite for user mode switching, which includes modifying mode item parameters, switching mode items, generating mode item templates, and downloading mode item templates. In short, switching the Mode option is only effective after the user Mode update is completed. As shown in Figure 30:

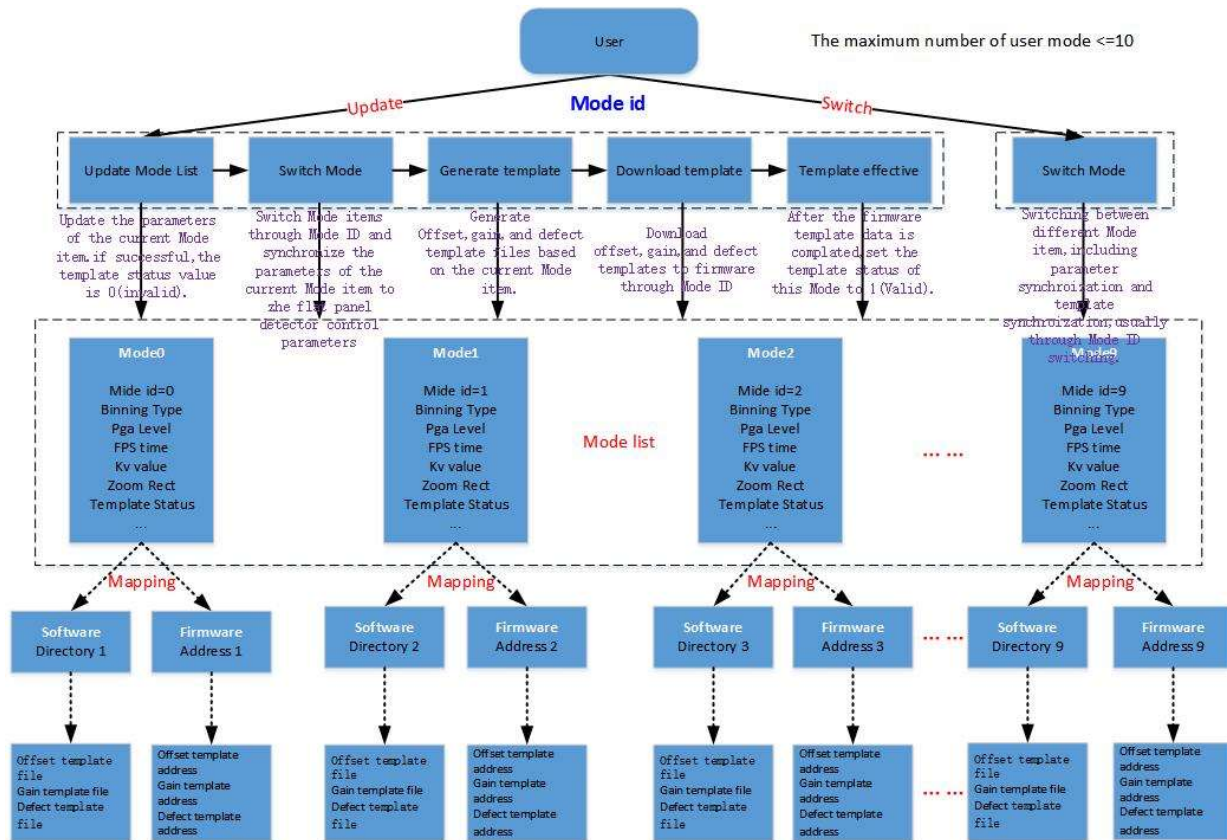


Figure 30

5.4.1. Integration

1. Include header files in user project

```
#Define_DLL_EX_IM 0

#include "HbiFpd.h" // Dynamic library header file

#pragma comment(lib, "HBISDKApi.lib")
```

2. User defined devices

```
// Device handles and configurations
```

```

Void * m_pFPD // Fpd sdk object handle

RegCfgInfo * m_pLastRegCfg // Firmware configuration

FPD_AQC_MODE aqc_mode // Acquisition mode and parameters, single frame acquisition, multi
frame acquisition, dark field image and bright field image acquisition

// Image resolution

int m_imgW, m_imgH;

char m_path[MAX_PATH];

fpd_base_cfg_st *fpd_base;

IMAGE_PROPERTY img_pro;

IMAGE_CORRECT_ENABLE* pCorrect;

// Some basic data types

m_pLastRegCfg = new RegCfgInfo;

pCorrect = new IMAGE_CORRECT_ENABLE;

fpd_base = new fpd_base_cfg_st;

```

The device list structure has been defined in HbiType. h

3. Interface call process

```

// Initialize DLL resources

Void * m_pFPD=HBI_Init(0);

If (m_pFPD==NULL){

printf("HBI_Init init failed! \n");

Return false;

}

// Registering Callbacks function

//MyCallbackFun callback function

int ret=HBI_RegEventCallBackFun (m_pFPD, myCallbackFun);

If (ret!=0){

printf("HBI_RegEventCallBackFun failed! \n");

```



```
HBI_ Destroy (m_pFPD);

M_ PFPD=NULL;

Return false;

}

// Connect the flat panel detector

COMM_ CFG commCfg;

Bool doOffsetTemps=false; // If not use firmware offset calibration

//1. PCIe mode

CommCfg._type=FPD_ COMM_ TYPE: PCIE_ COMM_ TYPE;

CommCfg._pciId =7024; // 7024,7024 is the PCIe card hardware ID

#ifdef

//2. UDP mode

//commCfg._type = FPD_COMM_TYPE:: UDP_COMM_TYPE;

//strcpy(commCfg._localip, "192.168.10.20");

//strcpy(commCfg._remoteip, "192.168.10.40");

//commCfg._localPort = 32896;

//commCfg._remotePort = 32897;

//3. UDP Jumbo mode

//commCfg._type = FPD_COMM_TYPE:: UDP_JUMBO_COMM_TYPE;

//strcpy(commCfg._localip, "192.168.10.20");

//strcpy(commCfg._remoteip, "192.168.10.40");

//commCfg._localPort = 32896;

//commCfg._remotePort = 32897;

#endif

//DoOffsetTemplate - false: The firmware will not update offset template after a successful connection,
true: The firmware will update the offset template after a successful connection

ret=HBI_ConnectDetector (m_pFPD, commCfg, doOffsetTemps);
```

// ret=HBI_Connect Detector (m_pFPD, commCfg) // After successful connection, the firmware does not update the offset template

```
if (ret!=0){
    printf("Connection failed! \n");
    HBI_Destroy (m_pFPD);
    m_pFPD =NULL;
    return false;
}
```

// Switch Mode option, modify trigger mode and enable image calibration

DETECTOR_INIT_PARAMS param;

param._ucmodeid=0 // Mode id [0,9];

para.uctriggerType=7 // 1-Software trigger mode, 3-High voltage trigger mode, 4-AED mode, 5-Dynamic: External trigger mode, 6-Dynamic: External trigger / FPD master mode, 7-Dynamic: Internal / SW trigger mode, 8-SAEC mode;

// Only valid in binning template mangement mode, invalid in multi-template management mode, because the mode item already contains trigger mode parameters to be compatible with different versions.

para.m_ucOffsetCorrection=0x03; // 0-No offset correction; 1-Software pre-offset correction; 2-Firmware post offset correction; 3-Firmware pre-offset correction.

param._ucGainCorrection=0x02; // 0-No gain correction; 1-Software gain correction; 2-Firmware gain correction.

param._ucDefectCorrection=0x02; // 0-No defect correction; 1-Software defect correction; 2-Firmware defect correction.

param._ucDummyCorrection=0x00; // Not supported.

int ret=HBI_InitDetectorParams (m_pFPD, inparam);

if (ret != HBI_SUCCSS)

{

if (ret == HBI_INVALID_DLL_HANDLE) printf("error:m_pFpd is NULL!");

else if (ret == HBI_ERR_DISCONNECT) printf("error:disconnect!\n");

else if (ret == HBI_ERR_CFG_ISEMPY) printf("error:m_pRomCfg is NULL!\n");

else if (ret == HBI_ERR_INVALID_PARAMS) printf("error:invalid params!\n");

else if (ret == HBI_NOT_SUPPORT) printf("warnning:not support mutil-mode!");

```

else if (ret == HBI_ERR_ROM_PARAMETER) printf("warning:mode number or mode id!\n");
else if (ret == HBI_ERR_INVALID_MODE)printf("error:Mode prammeter is abnormality!\n");
else if (ret == HBI_ERR_MALLOC_FAILED) printf("error:malloc failed!\n");
else if (ret == HBI_ERR_SENDDATA_FAILED) printf("error:send failed!\n");
else printf("err:Apply the mode failed!\n");
}
else
{
    printf("HBI_InitDetectorParams success!\n");
}
// disconnect
HBI_DisconnectDetector (m_pFPD);
// free resource (includedisconnect and free resource)
HBI_Destroy (m_pFPD);
m_pFPD =NULL;

#####
# Callback function and event description:
// Function description: Callback function definition, handling configuration, images, and panel return status.
// Parameter description:
// @pContext: Parameter 1, software object pointer, can be null
// @ ufpdId: Parameter 2, such as panel ID
// @byteEventid: Parameter 3, Event ID, refer to enum eCallbackEventCommType in 'HbiType. h'
//@ pvParam1:: Parameter 4, configure structure pointer or image structure pointer
// @nParam2: Parameter 5, such as data size. (image data is the length of the image buffer)
// @param3: Parameter 6, such as frame number, frame ID
// @param4 Parameter 7, such as frame rate.
// Callback function return value description: The return values are integer 0 or 1, 1- Received message, 0- Not
received message

```

#####

int CDemoDlg:: myCallbackFun (void * _context, int nfpdId, unsigned char command, void * pvParam1, int nParam2, int param3, int param4)

```
{
    int ret=0;

    if ((command == ECALLBACK_TYPE_SINGLE_IMAGE) ||
        (command == ECALLBACK_TYPE_MULTIPLE_IMAGE) ||
        (command == ECALLBACK_TYPE_ROM_UPLOAD))
    {
        if (pvParam1== NULL || nParam2== 0)
        {
            printf("Registration callback function parameter exception! \n");
            return ret;
        }
    }

    //
    int status=-1;
    int j=0;
    ret=1;
    switch (command)
    {
        case ECALLBACK_TYPE_FPD_STATUS: // Panel status: connected / disconnected / ready / busy
            printf("ECALLBACK TYPE NET-ERR-MSG, recode=%d \n", len);
            break;

        case ECALLBACK_TYPE_ROM_UPLOAD: /* Update configuration*/
            printf("ECALLBACK TYPE ROM_UPLOAD: \n");
            break;

        case ECALLBACK_TYPE_SET_CFG_OK:
            printf("ECALLBACK_TYPE_SET_CFG_OK:\n");
    }
}
```

```

break;

case ECALLBACK_TYPE_SINGLE_IMAGE: // Single frame image acquisition

case ECALLBACK_TYPE_MULTIPLE_IMAGE: // Continuous image acquisition

break;

case ECALLBACK_OVERLAY_NUMBER: // Overlying image number, such as total stacking 8 frames, the
current stacking number is 5 frames.

break;

case ECALLBACK_OVERLAY_16BIT_IMAGE: // Overlying finished, update the stacking result image (16 bits)

break;

.....

case ECALLBACK_TYPE_THREAD_EVENT:

default:

printf("command=%02x\n", command);

break;

}

return ret;

}

```

Return the image properties, length, width, number of bits, type, and size information, trigger mode, firmware calibration setting, etc. through callback functions.

Return the image data address and length through a callback function.

5.4.2. Multi-template Management

The combination of detector parameters shown in the following table is known to significantly affect the image calibration template, as described below.

Table 45.

Calibration Type	Size	Binning	PGA Level	FPS	kV	Zoom
Offset	√	√	√	√	×	√
Gain	√	√	√	×	√	√
Defect	√	√	√	×	√	√

Offset template: Binning, PGA, frame rate parameters can affect the offset template.

Gain template: Binning, PGA, KV and zoom can affect the gain template.

Defect template: Binning, PGA, KV and zoom can affect the defect template.

5.4.2.1. Multi-template Setting

Enable or disable multiple modes

unsigned int uSelect=1// Enable

//unsigned int uSelect = 0; // 0 or other means disabled, default to using binning method

```
int ret=HBI_FirmwareManagementTempalteType (m_pFPD, (unsigned int) select1);
```

```
if (ret!=HBI_SUCCSS)
```

```
{
```

```
    AfxMessageBox (_T ("error: HBI_FirmwareManagementTempalteType failed!"));
```

```
}
```

```
else
```

```
{
```

```
    AfxMessageBox (_T ("HBI_FirmwareManagementTempalteType success!"));
```

```
}
```

Set the number of valid Mode items

int nModeNumber=4// For example, setting 4 sets of Mode items must be greater than 0

```
int ret=HBI_SetValidModeltemNumber (m_pFPD, nModeNumber);
```

```
If (ret!=HBI_SUCCSS)
```

```
{
```

```
    AfxMessageBox (_T ("error: HBI_SetValidModeltemNumber failed!"));
```

```
}
```

```
else
```

```
{
```

```
    AfxMessageBox (_T ("HBI_SetValidModeltemNumber success!"));
```

```
}
```

Note : The dynamic library is compatible with both the firmware binning template management mode and the multi-template management mode. The dynamic library automatically determines the template management mode based on firmware parameters.

5.4.2.2. Updating Mode List

To prevent asynchronization between software and firmware, a mode list mechanism is designed to manage these multi templates. For each mode, the item member contains all related parameters, such as mode ID, binning, PGA, frame rate, KV, zoom and template validity indicator. The mode list information is part of full system configuration parameters which will be sent from software to firmware. After a successful connection, the system parameters will be synchronized to the DLL software. The mode list data can also be modified through DLL API. Users can switch different modes through the mode ID. The parameters under current mode will be synchronized to firmware after mode switch.

Note: it can only support at most 10 mode items due to the internal volatile memory capacity limitation.

Mode Id	Binning Type	PGA Level	FPS Time (ms)	Prepare Time (ms)	Voltage (Kv)	Zoom Rect(l,t,w,h)	Trigger Mode	Template Status(Valid)
Mode - 0	1x1	7.2pC	50	1000	0	0,0,0,0	7-Dynamic:Fpd Continue	Yes
Mode - 1	2x2	1.2pC	50	1000	0	0,0,0,0	7-Dynamic:Fpd Continue	Yes
Mode - 2	1x1	2.4pC	50	1000	0	0,0,0,0	7-Dynamic:Fpd Continue	Yes
Mode - 3	1x1	7.2pC	100	1000	0	0,0,0,0	7-Dynamic:Fpd Continue	No
Mode - 4	2x2	3.6pC	33	0	0	0,0,0,0	7-Dynamic:Fpd Continue	No
Mode - 5	1x1	7.2pC	1500	1500	0	0,0,0,0	7-Dynamic:Fpd Continue	No
Mode - 6	1x1	7.2pC	1000	2000	0	0,0,0,0	7-Dynamic:Fpd Continue	No
Mode - 7	1x1	4.8pC	1000	1200	0	0,0,0,0	7-Dynamic:Fpd Continue	No
Mode - 8	1x1	9.6pC	1000	0	0	0,0,0,0	7-Dynamic:Fpd Continue	No
Mode - 9	2x2	4.8pC	500	500	0	0,0,0,0	7-Dynamic:Fpd Continue	No

Figure 31

The mode list information can be exported through the XDiscovery tool in the following format:

###hbi_id0_mode_list. ini file###

[MODE-LIST] # Mode list

Panel_Size=1 # detector model

Mode_ Number=4 # The default total number of mode items is 10, and the current valid mode items number is 4.

Mode_ Curid=0 # The current mode ID is 0

[MODE-0] # mode ID 0 Information

Binning_Type=1 # binning

Fps_Time=50 # frame rate

PGA_Level=6 # PGA

Do_Kv=0 # KV

Zoom_Left=0 # zoom rect left

Zoom_Top=0 # zoom rect top

Zoom_Width=0 # zoom rect width

Zoom_Height=0 # zoom rect height

Trigger_Mode=7 # trigger mode

Prepare_Time=1000 # prepare delay time

Enbale_Status=1 # template validity. 0- all templates are invalid, 1- Only offset template is valid, 2- Only gain and defect templates are valid, 3- The offset, gain and defect template are valid, others – all templates are invalid

[MODE_1] # mode ID 1 Information

Binning_Type=2

Fps_Time=50

PGA_Level=2

Do_Kv=0

Zoom_Left=0

Zoom_Top=0

Zoom_Width=0

Zoom_Height=0

Trigger_Mode=7

Prepare_Time=1000

Enbale_Status=1

...

It is recommended to update the mode list through the XDiscovery tool, and users can also integrate the corresponding API to update by system software. You can find all detailed instruction in XDiscovery software user manual.

Introduction on interface functions to update the mode list.

Table 46.

Function	Parameter	Remarks
HBI_UpdateModeList	FPD_MODE_LIST* modelist;	Update mode list interface function, customer can set the number of valid mode items to n,

	<pre>typedef struct fpd_mode_list_st { unsigned char m_byCurModelId; FPD_MODE_DATA m_stModeArray[MODE_LIST_TOTAL]; }FPD_MODE_LIST;</pre>	each mode item parameter must be legal, otherwise it will fail to update due to invalid parameters.
HBI_UpdateCurModeData	<pre>FPD_MODE_DATA* mode; typedef struct fpd_mode_data_st { unsigned char m_byBinningType; unsigned int m_unFpsTime; unsigned char m_byPgaLevel; unsigned short m_usKvValue; unsigned short m_sZoomRectLeft; unsigned short m_sZoomRectTop; unsigned short m_sZoomRectWidth; unsigned short m_sZoomRectHeight; unsigned char m_byFpdTrigger; unsigned int m_unPrepareTime; unsigned char m_byTempStatus; }FPD_MODE_DATA;</pre>	Update the current Mode item data, including binning, PGA, KV, zoom area parameters, trigger mode and template status (dark field valid, bright field valid, all valid, all invalid)
HBI_UpdateModeData	<pre>FPD_MODE_DATA* mode; unsigned int id = 0;</pre>	Update the mode item data through the mode ID. This interface function is similar to HBI_UpdateCurModeData, but the Mode ID requires input parameters and can be used to update any Mode item data.

// Update Mode List

FPD_MODE_LIST modelist;

```
modelist.m_byCurModelId = usermode.m_byCurModelId;

memcpy(modelist.m_stModeArray, usermode.m_stModeList, sizeof(FPD_MODE_DATA) * MODE_LIST_TOTAL);

// Get the number of valid mode items

unsigned int usize = HBI_GetModeListSize(m_pFpd);

if(usize > MODE_LIST_TOTAL)

{

    ::AfxMessageBox(_T("err:HBI_GetModeListSize failed!"));

    return;

}

// Default parameter update

if (modelist.m_byCurModelId < 0 || modelist.m_byCurModelId >= usize )

{

    modelist.m_byCurModelId = 0;

}

//

BYTE bytePanelSize = theDoc->m_pRegRomCfg->m_SysBaseInfo.m_byPanelSize;

// Automatically update other parameters to default values

for (int i = 0; i < usize; i++)

{

    if (modelist.m_stModeArray[i].m_byFpdTrigger <= 0 || modelist.m_stModeArray[i].m_byFpdTrigger > 8)

        modelist.m_stModeArray[i].m_byFpdTrigger = 7;

    //

    if (modelist.m_stModeArray[i].m_byBinningType < 1 || modelist.m_stModeArray[i].m_byBinningType >

4)

        modelist.m_stModeArray[i].m_byBinningType = 1;

    //Pga Level

    if (bytePanelSize == 0x0a ||

        bytePanelSize == 0x0b ||
```

```

        bytePanelSize == 0x12 ||
        bytePanelSize == 0x13 ||
        bytePanelSize == 0x14 ||
        bytePanelSize == 0x15)
    {
        if (modelist.m_stModeArray[i].m_byPgaLevel != 8 &&
modelist.m_stModeArray[i].m_byPgaLevel != 9)

            modelist.m_stModeArray[i].m_byPgaLevel = 8;
    }
    else
    {
11)        if (modelist.m_stModeArray[i].m_byPgaLevel < 1 || modelist.m_stModeArray[i].m_byPgaLevel >

        {

            modelist.m_stModeArray[i].m_byPgaLevel = 6;

        }
        else
        {

            if (modelist.m_stModeArray[i].m_byPgaLevel == 8 ||
modelist.m_stModeArray[i].m_byPgaLevel == 9)

                modelist.m_stModeArray[i].m_byPgaLevel = 8;

        }
    }
    //
    if (modelist.m_stModeArray[i].m_unFpsTime <= 0)

        modelist.m_stModeArray[i].m_unFpsTime = 1000;

    //
    if (modelist.m_stModeArray[i].m_sZoomRectWidth == 0)
    {

        if (modelist.m_stModeArray[i].m_sZoomRectLeft != 0)

```

```
        modelist.m_stModeArray[i].m_sZoomRectLeft = 0;

    }

    //

    if (modelist.m_stModeArray[i].m_sZoomRectHeight == 0)

    {

        if (modelist.m_stModeArray[i].m_sZoomRectTop != 0)

            modelist.m_stModeArray[i].m_sZoomRectTop = 0;

    }

}

//

int ret = HBI_UpdateModeList(m_pFpd, &modelist);

if (HBI_SUCCSS != ret)

{

    ::AfxMessageBox(_T("err:HBI_UpdateModeList failed!"));

}

else

{

    ::AfxMessageBox(_T("HBI_UpdateModeList success!"));

}
```

//Update current mode list items

```
FPD_MODE_DATA user_mode;

user_mode.m_byBinningType = 1;

user_mode.m_unFpsTime = 50;

user_mode.m_unPrepareTime = 1500;

user_mode.m_byPgaLevel = 6;

user_mode.m_sZoomRectLeft = 0;

user_mode.m_sZoomRectTop = 0;
```

```
user_mode.m_sZoomRectWidth = 0;
user_mode.m_sZoomRectHeight = 0;
user_mode.m_usKvValue = 0;
user_mode.m_byFpdTrigger = 7;
user_mode.m_byTempStatus = 0;
//
int ret = HBI_UpdateCurModeData(theDoc->m_pFpd, &user_mode);
if (ret != HBI_SUCCSS)
{
    ::AfxMessageBox(_T("err:HBI_UpdateCurModeData failed!"));
}
else
{
    ::AfxMessageBox(_T("err:HBI_UpdateCurModeData success!"));
}
```

// Update mode list items through mode ID

```
unsigned int nModelItem=1; // Mode ID range from 0 to 9
FPD_MODE_DATA user_mode;
user_mode.m_byBinningType = 1;
user_mode.m_unFpsTime = 50;
user_mode.m_unPrepareTime = 1500;
user_mode.m_byPgaLevel = 6;
user_mode.m_sZoomRectLeft = 0;
user_mode.m_sZoomRectTop = 0;
user_mode.m_sZoomRectWidth = 0;
user_mode.m_sZoomRectHeight = 0;
user_mode.m_usKvValue = 0;
```

```

user_mode.m_byFpdTrigger = 7;

user_mode.m_byTempStatus = 0;

//

int ret = HBI_UpdateModeData(theDoc->m_pFpd, &user_mode, nModelItem);

if (ret != HBI_SUCCSS)

{

    ::AfxMessageBox(_T("err:HBI_UpdateModeData failed!"));

}

else

{

    ::AfxMessageBox(_T("err:HBI_UpdateModeData success!"));

}

```

5.4.2.3. Switching Mode Items

After the user finishes editing and updating the mode item, call the mode item through the mode ID, synchronize the mode item parameters to the product image acquisition control parameters.

Table 47.

Function	Interface Function	Parameter Description
Switch Mode	HBI_SwitchUserMode(void* handle, unsigned int modeid = 0, FPD_MODE_DATA *PMODEDATA=NULL);	Unsigned int modeid identifies the specified mode item. FPD_MODE_DATA * * PMODEDATA, updates the parameters of current mode item. PMODEDATA can be null (mode item parameters are not updated, only switch). If it is not null, update the mode item parameters and switch to the mode item.

Example:

```

// switch to mode 2

unsigned int umodeid=2;

int ret=HBI_SwitchUserMode (m_pFPD, umodeid, NULL/*&modedata */);

If (ret!=HBI_SUCCSS)

```

```

{
    AfxMessageBox (_T ("err: HBI_SwitchUserMode failed!"));
}

else

{
    AfxMessageBox (_T ("HBI_SwitchUserMode success!"));
}

```

5.4.2.4. Calibration Template Generation and Download

This multi-template management mechanism establishes a flexible relationship between parameters and image calibration templates, as illustrated in Figure 32:

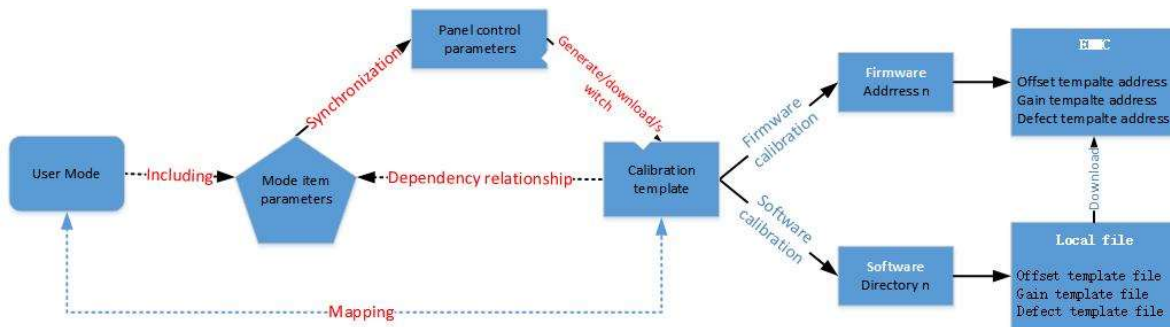


Figure 32

Step 1: After the user has edited and updated the mode item, call the mode item through mode ID, synchronize the mode item parameters to the system control parameters and complete the mode item switch.

Step 2: After switching the mode option, generate different types of templates (offset, gain, defect) under different parameter combinations, and associate the calibration template with the mode items.

Table 48.

Functions	Interface Function	Parameter Description
Quick template generation	<pre> HBI_GenerateTemplate(void *handle, EnumIMAGE_ACQ_CMD _mode, int bpreview); </pre>	EnumIMAGE_ACQ_CMD _mode, continuous image acquisition command, enumeration class, select the type of generated template, as follows // One-key quick generation mode

		OFFSET_TEMPLATE_TYPE, // Quick generation process of offset template - firmware generates offset template GAIN_TEMPLATE_TYPE, // Quick generation process of gain template with 1 group image DEFECT_TEMPLATE_GROUP1, // Quick generation process of defect template - group1. DEFECT_TEMPLATE_GROUP2, // Quick generation process of defect template - group2. DEFECT_TEMPLATE_GROUP3, // Quick generation process of defect template - group3. SOFTWARE_OFFSET_TEMPLATE, // Quick generation process of offset template - software generates offset template int bpreview - Whether to generate a preview template, 1-generated, 0-not generated.
--	--	--

Note: After generating the template, it is better to update the image correction enable setting, as the image correction enable status needs to be cancelled during the template generation process (except for the firmware offset correction enable state)

Step 3: For firmware calibration, the template file needs to be downloaded to the firmware in advance. This template mapping relationship relies on the mode parameter (mode ID), and the firmware will write the calibration template to the specified address based on the parameters.

Table 49.

Function	Interface Functions	Parameter Description
Download template based on Mode parameter	HBI_DownloadModeTemplate(void* handle, DOWNLOAD_MODE_FILE downloadfile);	DOWNLOAD_MODE_FILE downloadfile, download template file
Download correction template files according to template type (default to current mode Id)	HBI_DownloadModeTemplateByType(void* handle, int intemptype)	int intemptype, template type
Download correction template files according to mode Id and template type	HBI_DownloadModeTemplateByIdAndType(void* handle, unsigned int modeid, int intemptype)	int modeid, template id int intemptype, template type

Download correction template (default to current mode Id)	HBI_DownloadTemplate(void *handle, DOWNLOAD_FILE *downloadfile);	DOWNLOAD_FILE * downloadfile, download the template file to the current mode, and the mode ID is the current default mode ID.
Download the specified correction template file by default according to the type (default to the current mode ID)	HBI_DownloadTemplateByType(void *handle, int infiletype);	int intemptype, template type

Data type [DOWNLOAD_MODE_FILE](#):

//Template file attributes

typedef struct download_mode_file_st

```
{
    download_mode_file_st()
    {
        m_byModelId = 0;
        m_byBinningType = 1;
        m_byPgaLevel = 6;
        m_unFpsTime = 0;
        m_usKvValue = 0;
        m_sZoomRectLeft = 0;
        m_sZoomRectTop = 0;
        m_sZoomRectWidth = 0;
        m_sZoomRectHeight = 0;
        emtemplatetype = GAIN_TEMPLATE;
    };
    unsigned char m_byModelId;
    unsigned char m_byBinningType;
    unsigned char m_byPgaLevel;
    unsigned int m_unFpsTime;
    unsigned short m_usKvValue;
```

```
unsigned short m_sZoomRectLeft;

unsigned short m_sZoomRectTop;

unsigned short m_sZoomRectWidth;

unsigned short m_sZoomRectHeight;

emTEMPLATE_TYPE emtemplatetype;

}DOWNLOAD_MODE_FILE;
```

Step 4: After generating and downloading templates for all mode items, users can enable the image calibration in different modes by switching mode items.

5.4.3. Offset Template Generation and Download

// eg, Generate a template for mode 2 and download the template

// Connect the panel detector and switch the mode option. Once successful, the template can be generated in that mode.

```
unsigned int umodeid = 2;

int ret = HBI_SwitchUserMode(m_pFpd, umodeid, NULL/*&modedata*/);

if (ret != HBI_SUCCSS)

{

    AfxMessageBox(_T("err:HBI_SwitchUserMode failed!"));

}

else

{

    AfxMessageBox(_T("HBI_SwitchUserMode success!"));

}
```

// Acquire 1 set of dark field image and generate offset template.

```
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE:: OFFSET_TEMPLATE_TYPE;
```

```
int ret = HBI_GenerateTemplate(m_pFpd, enumTemplateType);
```

```
if (ret != HBI_SUCCSS) {}// failed
```

```
else {} // success
```

```
/*
```

The offset template generated by the firmware does not need to be downloaded. In some terminal applications, the detector will be assembled under continuous x-ray exposure conditions, and the software must generate the calibration in advance and download it to the firmware in such cases.

*/

// Registering Callbacks

// Callback function: USER_CALLBACK_HANDLE_PROCESS handFun - Global function or static member function, refer to HbiType. h

//pContext - Pointer to the software object, which can also be assigned as null if not needed by the user.

if (0 != HBI_RegProgressCallBack(m_pFpd, DownloadCallBackFun, (void *)this))

{

 printf("err:HBI_RegProgressCallBack failed!\n");

 return;

}

// parameters

//DOWNLOAD_MODE_FILE *downloadfile =NULL;

//if (downloadfile == NULL) downloadfile = new DOWNLOAD_MODE_FILE;

//if (downloadfile == NULL)

//{

// printf("err:malloc downloadfile failed!\n");

// return;

//}

//Example of Mode Item Data

//downloadfile->m_byModelId = (BYTE)umodeid;

//downloadfile->m_byBinningType = 1;

//downloadfile->m_byPgaLevel = 7;

//downloadfile->m_unFpsTime = 20;

//downloadfile->m_usKvValue = 0;

//downloadfile->m_sZoomRectLeft = 0;

//downloadfile->m_sZoomRectTop = 0;

//downloadfile->m_sZoomRectWidth = 0;

```
//downloadfile->m_sZoomRectHeight = 0;

//downloadfile->emtemplatetype = OFFSET_TEMPLATE;

// method 1: start download pre-offset template

//int ret = HBI_DownloadModeTemplate(m_pFpd, downloadfile);

//if (ret != HBI_SUCCSS)

//    ::AfxMessageBox(_T("err:Download offset-template failed!"));

//else

//    ::AfxMessageBox(_T("Download offset-template success!"));

// free resource

//Delete pUploadFile;

//PUploadFile=NULL;

or

// method 2: start download pre-offset template

int infiletype=2// Download file types 0-gain template, 1-defect template, 2-offset template, others - not
supported

int ret=HBI_DownloadModeTemplateByType (d_pFPD, infilename)

If (ret!=HBI_SUCCSS)

    : AfxMessage Box (_ T ("err: Download offset template failed!"));

else

    : AfxMessage Box (_ T ("Download offset template success!"));


// Update offset calibration setting and enable firmware pre-offset calibration

IMAGE_CORRECT_ENABLE *pcorrect = new IMAGE_CORRECT_ENABLE;

if (pcorrect != NULL)

{

    pcorrect->bFeedbackCfg = false;

    pccorrect ->ucOffsetCorrection=0x03 // 0x00- no calibration, 0x01- software pre-offset, 0x02- firmware
post offset, 0x03- firmware pre-offset offset.

    pccorrect ->ucGainCorrection=0x00 // 0x00- no calibration, 0x01- software gain, 0x02- firmware gain.
```

pccorrect ->ucDefectCorrection=**0x00** // 0x00- No calibration, 0x01- software defect, 0x02- firmware defect.

Pccorrect ->ucDummyCorrection=**0x00** // 0x00- Not supported

```
ret=HBI_UpdateCorrectEnable (theDoc -> m_pFPD, pcorrect);

if (ret==0)

    // success

else

    // failed

//Release resources

delete correct;

pccorrect=NULL;

}
```

5.4.4. Gain Template Generation and Download

// Generate a template for Mode 2 and download the template

Connect the panel detector and switch the mode option. Once successful, gain template can be generated under this mode. If it has already been switched, there is no need to repeat the switch.

```
unsigned int umodeid=2;

int ret = HBI_SwitchUserMode(m_pFpd, umodeid, NULL/*&modedata*/);

if (ret != HBI_SUCCSS)

{

    AfxMessageBox(_T("err:HBI_SwitchUserMode failed!"));

}

else

{

    AfxMessageBox(_T("HBI_SwitchUserMode success!"));

}
```

Acquire 1 set of bright field images with a fixed KV value.

EnumIMAGE_ACQ_MODE enumTemplateType = EnumIMAGE_ACQ_MODE::GAIN_TEMPLATE_TYPE;

```
int ret = HBI_GenerateTemplate(m_pFpd, enumTemplateType);

if (ret!=HBI_SUCCSS) {} // failed

else {}//Success


// Registering Callbacks

//Callback function: USER_CALLBACK_HANDLE_PROCESS handFun - Global function or static member function,
refer to HbiType. h

// pContext- Software object pointer, if not needed by the user, can also be assigned as null.

if (0 != HBI_RegProgressCallBack(m_pFpd, DownloadCallBackFun, (void *)this))
{
    printf("err:HBI_RegProgressCallBack failed!\n");
    return;
}

// parameter

//DOWNLOAD_MODE_FILE *downloadfile =NULL;

//if (downloadfile == NULL) downloadfile = new DOWNLOAD_MODE_FILE;

//{
//    printf("err:malloce downloadfile failed!\n");
//    return;
//}

//Example of mode Item data

//downloadfile->m_byModelId = (BYTE)umodeid;

//downloadfile->m_byBinningType = 1;

//downloadfile->m_byPgaLevel = 7;

//downloadfile->m_unFpsTime = 20;

//downloadfile->m_usKvValue = 0;

//downloadfile->m_sZoomRectLeft = 0;

//downloadfile->m_sZoomRectTop = 0;

//downloadfile->m_sZoomRectWidth = 0;
```

```
//downloadfile->m_sZoomRectHeight = 0;
```

```
//downloadfile->emtemplatetype = GAIN_TEMPLATE;
```

```
// method 1: start download the gain template
```

```
//int ret = HBI_DownloadModeTemplate(m_pFpd, downloadfile);
```

```
//if (ret != HBI_SUCCSS)
```

```
//    ::AfxMessageBox(_T("err:Download gain-template failed!"));
```

```
//else
```

```
//    ::AfxMessageBox(_T("Download gain-template success!"));
```

```
or
```

```
// method 2: start download the gain template
```

```
int infiletype=0 // Download file types 0-gain template, 1-defect template, 2-offset template, others - not supported
```

```
int ret=HBI_DownloadModeTemplateByType (d_pFPD, infiletype)
```

```
If (ret!=HBI_SUCCSS)
```

```
    : AfxMessageBox (_ T ("err: Download gain template failed!"));
```

```
else
```

```
    : AfxMessageBox (_ T ("Download gain template success!"));
```

```
// Update gain calibration setting and enable firmware gain calibration
```

```
IMAGE_CORRECT_ENABLE *pcorrect = new IMAGE_CORRECT_ENABLE;
```

```
if (pcorrect != NULL)
```

```
{
```

```
    pcorrect->bFeedbackCfg = false;
```

```
    pccorrect ->ucOffsetCorrection=0x03 // 0x00- no calibration, 0x01- software offset, 0x02- firmware post offset, 0x03- firmware pre-offset.
```

```
    pccorrect ->ucGainCorrection=0x02 // 0x00- no calibration, 0x01- software gain, 0x02- firmware gain.
```

```
    pccorrect ->ucDefectCorrection=0x00 // 0x00- No calibration, 0x01- software defect, 0x02- firmware defect.
```

```
    pccorrect ->ucDummyCorrection=0x00 // Not supported.
```

```
    ret=HBI_UpdateCorrectEnable (theDoc -> m_pFPD, pcorrect);
```

```
if (ret==0)
    // success

else
    // faild

//Release resources

delete correct;

pccorrect=NULL;
```

5.4.5. Defect Template Generation and Download

// Generate a template for Mode 2 and download the template

Connect the panel detector and switch the mode option. Once successful, a template can be generated under this mode. If it has already been switched, there is no need to repeat the switch.

```
unsigned int umodeid = 2;

int ret = HBI_SwitchUserMode(m_pFpd, umodeid, NULL/*&modedata*/);

if (ret != HBI_SUCCSS)
{
    AfxMessageBox(_T("err:HBI_SwitchUserMode failed!"));
}

else
{
    AfxMessageBox(_T("HBI_SwitchUserMode success!"));
}
```

1. Adjust the exposure condition to 1/5 saturation dose level, acquire 1st group bright field image.

```
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE: DEFECT_ACQ_GROUP1;
```

```
int ret=HBI_GenerateTemplate (m_pFPD, enumTemplateType);
```

```
if (ret!=HBI_SUCCSS) {}//failed
```

```
else {}//Success
```

Note: As current setting is firmware pre-offset calibration, there is no need to download offset template to the firmware.

2. Adjust the exposure condition to 1/2 saturation dose level, acquire 2nd group bright field image.

```
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE: DEFECT_ACQ_GROUP2;
```

```
int ret=HBI_GenerateTemplate (d_pFPD, enumTemplateType);
```

```
if (ret!=HBI_SUCCSS) {}//failed
```

```
else {}//Success
```

3. Adjust the exposure condition to 4/5 saturation dose level, acquire 3th group bright field image.

```
EnumIMAGE_ACQ_MODE enumTemplateType=EnumIMAGE_ACQ_MODE: DEFECT_ACQ_GROUP3;
```

```
int ret=HBI_GenerateTemplate (m_pFPD, enumTemplateType);
```

```
if (ret!=HBI_SUCCSS) {}//failed
```

```
else {}//Success
```

```
/*
```

*** After the template is successfully generated, start downloading the template**

```
*/
```

```
// Registering Callbacks
```

```
//Callback function: USER_CALLBACK_HANDLE_PROCESS handFun - Global function or static member function,  
refer to HbiType. h
```

```
//PContext - Pointer to software object, which can also be assigned as null if not needed by the user.
```

```
if (0 != HBI_RegProgressCallBack(m_pFpd, DownloadCallBackFun, (void *)this))
```

```
{
```

```
    printf("err:HBI_RegProgressCallBack failed!\n");
```

```
    return;
```

```
}
```

```
//DOWNLOAD_MODE_FILE *downloadfile =NULL;
```

```
//if (downloadfile == NULL) downloadfile = new DOWNLOAD_MODE_FILE;
```

```
//if (downloadfile == NULL)
```

```
//{
```

```
//    printf("err:malloce downloadfile failed!\n");
```

```
//      return;

//}

//Example of Mode Item Data

//downloadfile->m_byModelId = (BYTE)umodeid;

//downloadfile->m_byBinningType = 1;

//downloadfile->m_byPgaLevel = 7;

//downloadfile->m_unFpsTime = 20;

//downloadfile->m_usKvValue = 0;

//downloadfile->m_sZoomRectLeft = 0;

//downloadfile->m_sZoomRectTop = 0;

//downloadfile->m_sZoomRectWidth = 0;

//downloadfile->m_sZoomRectHeight = 0;

//downloadfile->emtemplatetype = DEFECT_TEMPLATE;

// method 1: start download the defect template

//int ret = HBI_DownloadModeTemplate(m_pFpd, downloadfile);

//if (ret != HBI_SUCCSS) {

//  ::AfxMessageBox(_T("err: Download defect template failed!"));

//}

//else

//  ::AfxMessageBox(_T("Download defect template success!"));

//Delete pUploadFile;

//PUploadFile=NULL;

or

// method 2: start download the defect template

int infiletype=1// Download file types 0-gain template, 1-defect template, 2-offset template, others - not
supported

int ret=HBI_DownloadModeTemplateByType (m_pFPD, infiletype)

if (ret != HBI_SUCCSS)

    ::AfxMessageBox(_T("err:Download defect-template failed!"));
```

else

```
::AfxMessageBox(_T("Download defect-template success!"));
```

```
// Update defect calibration setting and enable firmware defect calibration
```

```
IMAGE_CORRECT_ENABLE *pcorrect = new IMAGE_CORRECT_ENABLE;
```

```
if (pcorrect != NULL)
```

```
{
```

```
    pcorrect->bFeedbackCfg = false;
```

```
    pccorrect ->ucOffsetCorrection=0x03 // 0x00- no calibration, 0x01- software pre-offset, 0x02- firmware  
post offset, 0x03- firmware pre-offset offset.
```

```
    pccorrect ->ucGainCorrection=0x02 // 0x00- no calibration, 0x01- software gain, 0x02- firmware gain.
```

```
    pccorrect ->ucDefectCorrection=0x02 // 0x00- no calibration, 0x01- Software defect, 0x02- firmware  
defect.
```

```
    pccorrect ->ucDummyCorrection=0x00 // 0x00- not supported
```

```
    ret=HBI_UpdateCorrectEnable (theDoc ->m_pFPD, pcorrect);
```

```
    if (ret==0)
```

```
        // success
```

```
    else
```

```
        // failed
```

```
    //Release resources
```

```
    delete correct;
```

```
    pccorrect=NULL;
```

```
}
```

5.4.6. Progress Status Callback Function

```
// Download template callback function: static member function or global function
```

```
// InContext: Software object pointer, can be NULL.
```

```
Static int DownloadCallBackFun (unsigned char command, int code, void * inContext)
```

```
{
```

```
    ///CTemplateTool *ptr = (CTemplateTool *)inContext;
```

```
    ///if (ptr == NULL) {
```

```
///  
    printf("err:inContext is NULL!\n");  
  
///  
    return 0;  
  
///  
  
///  
if (theDemo == NULL) {  
  
    printf("err:theDemo is NULL!\n");  
  
    return 0;  
  
}  
  
int len = 0;  
  
switch (command)  
{  
  
case ECALLBACK_UPDATE_STATUS_START: // Download starts  
  
    printf("DownloadCallBackFun: ECALLBACK_UPDATE_STATUS_START: Download starts \n");  
  
    break;  
  
case ECALLBACK_UPDATE_STATUS_PROGRESS: // Download progress. Code as percentage (0-100)  
  
    printf("DownloadCallBackFun: ECALLBACK_UPDATE_STATUS_PROGRESS: %% d \n", code);  
  
    break;  
  
case ECALLBACK_UPDATE_STATUS_RESULT://Feedback results and errors  
  
    if (0<=code)&&(code<=6)) // Success  
  
    {  
  
        if (code==0) {// Download the Offset template  
  
            printf("DownloadOffsetCBFun:% s \n", "Download Offset template! \n");  
  
        }  
  
        else if (code==1) {// Download the Gain template  
  
            printf("DownloadOffsetCBFun:% s \n", "Download Gain template! \n");  
  
        }  
  
        else if (code==2) {// Download the Defect template  
  
            printf("DownloadOffsetCBFun:% s \n", "Download Defect template! \n");  
  
        }  
  
    }  
  
}
```

```
        else if (code==3) { // Offset template download completed

            printf("DownloadOffsetCBFun:%s \n", "Offset template download completed! \n");

        }

        else if (code==4) { // Gain template download completed
printf("DownloadOffsetCBFun:%s \n", "Download gain finish! \n");

        }

        else if (code==5) { // Defect template download completed
printf("DownloadOffsetCBFun:%s \n", "Defect template download completed! \n");

        }

        else /* if (code==6) */ { // Download successful and exit thread

            printf("DownloadOffsetCBFun:%s \n", "Download success and exit thread! \n");

        }

    }

    else // Failed

    {

        if (code==1){

            printf("err: DownloadOffsetCBFun:%s \n", "wait event other error! \n");

        }

        else if (code==2){

            printf("err: DownloadOffsetCBFun:%s \n", "timeout! \n");

        }

        else if (code==3){

            printf("err: DownloadOffsetCBFun:%s \n", "downloaded offset failed! \n");

        }

        else if (code==4){

            printf("err: DownloadOffsetCBFun:%s \n", "downloaded gain failed! \n");

        }

        else if (code==5){
```

```

        printf("err: DownloadOffsetCBFun:%s \n", "downloaded defect failed! \n");
    }
    else if (code=-6){
        printf("err: DownloadOffsetCBFun:%s \n", "Download failed and exit thread! \n");
n");
    }
    else if (code=-7){
        printf("err: DownloadOffsetCBFun:%s \n", "read offset failed! \n");
    }
    else if (code=-8){
        printf("err: DownloadOffsetCBFun:%s \n", "read gain failed! \n");
    }
    else if (code=-9){
        printf("err: DownloadOffsetCBFun:%s \n", "read defect failed! \n");
    }
    else{
        printf("err: DownloadOffsetCBFun: unknown error,%d \n", code);
    }
}
break;
default://unusual
    printf("DownloadCallBackFun: unusual, retcode=%d \n", code);
    break;
}
return 1;
}

```

5.4.7. Status of Calibration Template

The offset, gain and defect template status (valid or invalid) are stored in the firmware for each mode, as follows:

- When the Binning, PGA, frame rate, and zoom rect change, the dark field (Offset) template will be invalid. The solution is to update the current offset template and download to firmware.
- When the Binning, PGA, KV and zoom rec change, the bright field (gain and defect) templates will be invalid. The solution is to update the current gain and defect templates and download to firmware.
- Set template related parameters or download templates, the template status will be synchronized.

Data type `FPD_MODE_DATA`:

//Notice: fpd mode data information: 22

```
typedef struct fpd_mode_data_st
```

```
{
```

```
... ..
```

unsigned char m_byTempStatus; // 0-Both dark and bright field templates are invalid, //1-Only dark field templates are valid, 2-Only the bright field template is valid, 3-Both dark and bright field templates are valid, others-Both dark and bright field templates are invalid.

```
}FPD_MODE_DATA;
```

Table 50.

Status value	Description
0	Both dark field (offset) and bright field (gain and defect) are invalid
1	The dark field (offset) is valid, while the bright field (gain and defect) is invalid
2	Dark field (offset) invalid, bright field (gain and defect) valid
3	Both the dark field (offset) and the bright field (gain and defect) are valid

Add template status will be feedback to the main callback event:

```
int handleCommandEventA (void * pvContext, int nDevId, unsigned char byteEvnetid, void * pvParam1, int nParam2, int nParam3, int nParam4)
```

```
{
```

```
...
```

```
case ECALLBACK_FIRMWARE_TEMPLATE_STATUS:
```

{//nParam2 represents the current Mode ID, and nParam3 represents the template status information of the Mode, as shown in the Template Status Table above.

```
printf("ECALLBACK_FIRMWARE_TEMPLATE_STATUS:CurModelId=%d,nTempStatus=%d",
nParam2, nParam3); }
```

```
        break;

...

}
```

5.4.8. Update Calibration

To prevent the automatic cancellation of calibration enable during the template generation process, please update the calibration enable again after the template generation process is completed.

```
IMAGE_CORRECT_ENABLE *pcorrect = new IMAGE_CORRECT_ENABLE;

if (pcorrect != NULL)
{
    Pccorrect ->bFeedbackCfg=true// Feedback firmware parameters to software, false only returns
confirmation information

    pccorrect->ucOffsetCorrection=0x03 // 0x00- no calibration, 0x01- software pre-offset, 0x02- firmware
post offset, 0x03- firmware pre-offset offset.

    pccorrect->ucGainCorrection=0x02 // 0x00- no calibration, 0x01- software gain, 0x02- firmware gain.

    pccorrect->ucDefectCorrection=0x02 // 0x00- no calibration, 0x01- Software defect, 0x02- firmware
defect.

    pccorrect->ucDummyCorrection=0x00 // 0x00- not supported.

    ret=HBI_UpdateCorrectEnable (theDoc ->m_pFPD, pcorrect);

    if (ret==0)
        // success

    else
        // failed

    //release resources

    delete correct;

    pcorrect=NULL;

}
```

There are several situations where the calibration template needs to be updated.

1. Updating the Binning, PGA, frame rate and zoom rect parameters affect the offset template.
2. Updating the Binning, PGA, KV and zoom rect parameters affect the gain and defect templates.

3. It may be necessary to cancel the image correction enable setting during the template generation process, except for the firmware pre-offset correction enable setting. Therefore, it is best to update the template correction enable setting parameters after generating these templates.

4. It is better for users to pre-set the mode list parameters and generate templates based on each mode item. They need to download and update firmware calibration templates to avoid mismatch. Users can modify the mode item by switching between the mode items and controlling parameters real-time.

5.5. Problems and Suggestions

1. Socket exception with error codes 10049 and 100048.

Reason:

a. Port is occupied or IP address is incorrect. For example, if a program has already been opened, attempting to open a second program connection will result in an error code of 10049.

b. Closing the socket resource without fully releasing resource, resulting in an error code of 10048.

Solution:

Avoid opening multiple programs that occupy the same port.

Confirm the disconnection and resource release before connecting again.

2. Socket exception with error code 10040.

Reason:

Receive buffer overflow or insufficient space, the reason is the mismatch between firmware Jumbo packet and software standard packet.

Solution:

Align the communication mechanism between software and firmware.

3. Socket exception, sending data failed with error code 10051.

Reason:

The local network address is incorrect.

Solution:

Correct the local network IP address.

5. Call HBI_TriggerAndCorrectApply function, then wait for the panel parameters to be updated before proceeding with HBI_GetFpdCfgInfo, otherwise the modified parameters will not be synchronized, as firmware data parsing still needs a specific time to modify all parameters to the latest updated value.

6. Image Overlay

The image overlay function supports both software and firmware overlay mechanisms. All acquired images are averaged into a single image once the configured frame count is reached. The averaged result image can be output in either 16-bit or 32-bit format. The overlay number must be greater than 0 .

6.1. Software Overlay

For software image overlay, the accumulation and averaging of images are performed within the software. The acquisition frame number parameter in the HBI_LiveAcquisition function is used to control how many frames are acquired and averaged. Refer to the implementation of the continuous acquisition function and set the parameters as follows.

For example, software overlays 8 frames.

```
aqc_mode.eAqcCmd = LIVE_ACQ_DEFAULT_TYPE;

aqc_mode.eLivetype = OVERLAY_16BIT_IMG; // overlap result is 16-bit image
//aqc_mode.eLivetype = OVERLAY_32BIT_IMG; // overlap result is 32-bit image

aqc_mode.ndiscard = 0;

aqc_mode.nframeid = 0;

aqc_mode.nAcqnumber = 8; // overlay number

aqc_mode.ngroupno = 0;

int ret = HBI_LiveAcquisition(m_pFpd, aqc_mode);

if (ret != HBI_SUCCSS)
{
    AfxMessageBox(_T("error:HBI_LiveAcquisition failed!"));
}

else
{
    AfxMessageBox(_T("HBI_LiveAcquisition success!"));
}

// The stacking process and result will be output through the main callback function.

case ECALLBACK_OVERLAY_NUMBER: // Overlay frame number, such as overlaying 8 frames, now stacking up
to 5 frames

...

break;
```

```
case ECALLBACK_OVERLAY_16BIT_IMAGE: // When finishing the image overlay, update the stacking result
image (16-bit)
```

```
...
```

```
break;
```

```
case ECALLBACK_OVERLAY_32BIT_IMAGE: // When finishing the image overlay, update the stacking result
graph (32-bit)
```

```
...
```

```
break;
```

6.2. Firmware Overlay

For firmware image overlay, the process of image accumulation and average calculation is performed within the firmware. An acquisition frame number parameter is included in the "HBI_SetFirmwareOverlay" function. This parameter can be used to control how many frames will be acquired and averaged. Refer to the implementation of the firmware overlay function and set the parameters as follows.

For example, firmware overlays 8 frames.

```
unsigned int noverlaynum = 8;

int ret = HBI_SetFirmwareOverlay(m_pFpd, noverlaynum);

if (ret != HBI_SUCCSS)
{
    AfxMessageBox(_T("error:HBI_SetFirmwareOverlay failed!"));
}

else
{
    AfxMessageBox(_T("HBI_SetFirmwareOverlay success!"));
}

// Cancel image overlay

noverlaynum = 0;

ret = HBI_SetFirmwareOverlay(m_pFpd, noverlaynum);

if (ret != HBI_SUCCSS)
{
    AfxMessageBox(_T("error:HBI_SetFirmwareOverlay failed!"));
}
```

```
}  
else  
{  
    AfxMessageBox(_T("HBI_SetFirmwareOverlay success!"));  
}
```

7. Rotation and Mirroring

Image rotation & mirroring features can be implemented through both software and firmware. Currently, software supports both rotation and mirroring functions, while the firmware only supports mirroring, but not rotation due to the high implementation difficulty. If the image needs to be both rotated and mirrored, the default sequence is to rotate first, then mirror the image.

Note: It is better to use firmware mirror function instead of software, considering the resource cost and efficiency.

Table 51.

Rotate/Mirror	Rotate 90 ° to the left	Rotate 90 ° to the right	Mirror Horizontally	Mirror vertically	Mirror horizontally first and then vertically
Software	√	√	√	√	×
Firmware	×	×	√	√	√

Rotation and mirroring can be achieved by setting different parameters, as shown in the table below.

Table 52.

Rotate	0 - Original direction	Original direction
	1 - Software Right rotate 90°	Software right rotate 90°
	2 - Software Left rotate 90°	Software left rotation 90°
Mirror	0 - Original direction	Original direction
	1 - Software Vertical mirror	Software vertical mirror
	2 - Software Horizontal mirror	Software horizontal mirror
	3 - Firmware vertically mirror	Firmware vertical mirror
	4 - Firmware horizontal mirror	Firmware horizontal mirror
	5 - Firmware vertically horizontal mirror	Firmware horizontally & vertically mirror

The rotation and mirror functions will also affect the image calibration templates, all of these features needs to be configured before the image calibration generation, so that the generated templates will includes these rotation and mirror features.

7.1. Software Rotation and Mirroring

For example, the image is right rotated 90° and vertically mirrored through software.

```
int nRotate=1;

int nFlip=2;

int ret = HBI_SetImageRotateAndFlip(m_pFpd, nRotate, nFlip);

if (ret != HBI_SUCCSS)

{

    AfxMessageBox(_T("error:HBI_SetImageRotateAndFlip failed!"));

}

else

{

    AfxMessageBox(_T("HBI_SetImageRotateAndFlip success!"));

}
```

7.2. Firmware Mirroring

```
int nRotate=0;

int nFlip=3;

int ret = HBI_SetImageRotateAndFlip(m_pFpd, nRotate, nFlip);

if (ret != HBI_SUCCSS)

{

    AfxMessageBox(_T("error:HBI_SetImageRotateAndFlip failed!"));

}

else

{

    AfxMessageBox(_T("HBI_SetImageRotateAndFlip success!"));

}
```

8. Preview Mode

For wireless products, a preview image is required before the final image. This preview image can be used to provide an overview of the final image, especially when the final image requires a long transmission time in wireless mode.

The preview image can be generated as a thumbnail by extracting one pixel from each 16x16 pixel matrix of the full-resolution image. This creates a new pixel matrix, making the preview image one-sixteenth the size of the original full-resolution image (width / 4, height / 4).

The preview setting function is `HBI_SetPreviewMode(void *handle, int inMode)`, where `inMode` is used to set the current image mode: 0 for normal image, 1 for preview image, and 2 for preview image plus normal image (only supports single frame acquisition).

Application 1:

Upload the preview and normal images separately. Upload the preview image in preview mode (`inMode = 1`), and upload the normal image in normal mode (`inMode = 0`).

1. Set `inMode = 1`, call `HBI_SetPreviewMode` to set preview mode, and call the "ECALLBACK_TYPE_ROM_UPLOAD" event feedback to update parameters after successful setting. "m_slmageHeight" and "m_slmageWidth" will be updated to the latest size, which is one-fourth of the original value.
2. A software image acquisition command, high voltage trigger signal, or exposure under AED mode will initiate one frame image acquisition. The image data will be uploaded by "ECALLBACK_TYPE_PREVIEW_IMAGE," supporting both single frame and continuous image acquisition.
3. After acquiring the preview image, cancel preview mode (set `inMode = 0`) and call `HBI_SetPreviewMode` again. "m_slmageHeight" and "m_slmageWidth" will be updated to the full resolution size.
4. Acquire the normal image.
5. The preview image is fed back by the "ECALLBACK_TYPE_PREVIEW_IMAGE" event (event no. 0x53), while the normal image is fed back by "ECALLBACK_TYPE_SINGLE_IMAGE" (event no. 0x51) or "ECALLBACK_TYPE_MULTIPLE_IMAGE" (event no. 0x52).

Application 2:

Upload the preview and normal images consecutively; upload one frame of preview image and normal image at the same time.

1. Set `inMode = 2`, call `HBI_SetPreviewMode` to set preview image + normal image mode, and call the `ECALLBACK_TYPE_ROM_UPLOAD` event to update parameters. "m_slmageHeight" and "m_slmageWidth" will be updated to the latest size, which is one-fourth of the original value.
2. A software image acquisition command, high voltage trigger signal, or exposure under AED mode will initiate one frame image acquisition. The image data will be uploaded by "ECALLBACK_TYPE_PREVIEW_IMAGE," supporting both single frame and continuous image acquisition.
3. Acquire the preview image and normal image together.

4. Cancel preview image + normal image mode (set `inMode = 0`) and call `HBI_SetPreviewMode` again. "m_sImageHeight" and "m_sImageWidth" parameters will be updated to the full resolution size.

5. Preview image data is fed back by the `ECALLBACK_TYPE_PREVIEW_IMAGE` event (event no. 0x53), while the normal image data is fed back through `ECALLBACK_TYPE_SINGLE_IMAGE` (event no. 0x51).

Note:

The preview image feature only supports panel-2530-85um and panel-3543-140um products.

Preview mode is mutually exclusive with binning 2x2, binning 3x3, and binning 4x4 modes. If preview image is enabled, the binning type must be binning 1x1.

9. SAEC Mode

In SAEC mode, a pre-exposure is needed before the final exposure and image acquisition, this pre-expose image will be taken as a exposure condition evaluation, the final exposure condition will be calculated according to pre-exposure image. This is a good way to control the exposure condition for the examination image.

Note:

1. SAEC mode is only supported by panel-2530-85um detector.
2. The SAEC mode is work mode = 0x08.

Calling Steps:

a. Call the "HBI_SAecModeApply" function, where 1 means uploading preview image, 0 means uploading normal image, m_nPerReadyTm, m_nPostReadyTm are the preview ready delay time and post ready delay time.

b. Call the "HBI_SAecModeApply" function, where 0 means uploading normal images.

c. Call "HBI SAecAcq" function to acquire image.

d. ID of the event in the callback function:

case ECALLBACK_TYPE_SINGLE_IMAGE: normal image, for example, panel-2530-85um, image width=2816, image height=3584.

case ECALLBACK_TYPE_PREVIEW_IMAGE: preview image, for example, panel-2530-85um, image width = 2816/4 = 704, image height = 3584/4 = 896.

```
void CSAecModeTester::OnBnClickedBtnSaecAutoTest()
```

```
{
    // TODO: Add control notification handler code here

    this->UpdateData(TRUE);

    //

    if (!IsOpen) {
        AfxMessageBox(_T("err: Product is disconnected!"));
        return;
    }

    //
```

```
if (m_pRegRomCfg == NULL) {
    AfxMessageBox(_T("err: m_pRegRomCfg!"));
    return;
}
//
if (m_pRegRomCfg -> m_SysBaseInfo.m_byPanelSize != 0x05) {
    AfxMessageBox(_T("err: Product is not 2530-85um!"));
    return;
}
//
if (m_nPerReadyTm * m_nPostReadyTm <= 0) {
    AfxMessageBox(_T("err: Ready time is error!"));
    return;
}
// STEP1
theDoc -> ReportError(_T("STEP1: SAEC preview-mode starts!"));
int preview = 0x01; // 01-review image; 00-normal image;
int ret = HBI_SAecModeApply(m_pFpd, preview, m_nPerReadyTm, m_nPostReadyTm);
if (ret != HBI_SUCCSS) {
    AfxMessageBox(_T("err: Set SAEC preview-mode failed!"));
    return;
}
ReportError(_T("STEP1: SAEC preview-mode finished!"));

// Waiting to upload preview images
ReportError(_T("STEP1: SAEC preview image!"));
// .....
ReportError(_T("STEP1: SAEC update exposure condition!"));
```

```
// Image change dose

// STEP2
ReportError(_T("STEP2: SAEC normal mode starts!"));

preview = 0x00; // 01-preview image; 00-normal image;

ret = HBI_SAecModeApply(m_pFpd, preview, m_nPerReadyTm, m_nPostReadyTm);

if (ret != HBI_SUCCSS) {

    AfxMessageBox(_T("err: Set SAEC normal mode failed!"));

    return;

}

ReportError(_T("STEP2: SAEC normal mode finished!"));

// STEP3
ReportError(_T("STEP3: SAEC acquisition image starts!"));

ret = HBI_SAecAcq(m_pFpd);

if (ret != HBI_SUCCSS) {

    AfxMessageBox(_T("err: Send SAEC acquisition image!"));

    return;

}

ReportError(_T("STEP3: SAEC acquisition image finished!"));

}
```

10. IP Configuration

The following description is an introduction to how to modify panel IP and MAC addresses.

1. The default product IP address is 192.168.10.40, and the local NIC card IP address is suggested to be set with 192.168.10.20 as default.
2. Modify the local network card IP address, such as "192.168.20.20".
3. Open the XDsiccovery tool, the main menu "Detector" -> "Communication Configuration" -> "Detector Base Configuration" -> "Communication".

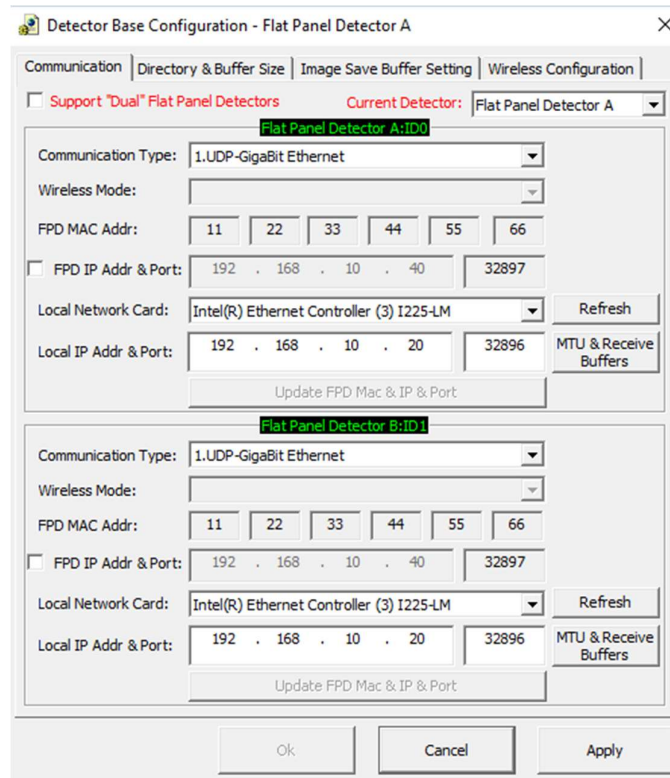


Figure 33

4. Modify the IP and MAC address, the product IP and MAC address generally does not need to be modified in single product connection case, but for a "dual panel" or "multi-panel" solution, it needs to be modified to prevent IP conflicts.

Detector Base Configuration - Flat Panel Detector A

Communication | Directory & Buffer Size | Image Save Buffer Setting | Wireless Configuration

1. Cancel Dual Flat Panel Detectors and select Detector A

☐ Support Dual Flat Panel Detectors

Current Detector: Flat Panel Detector A

Flat Panel Detector A:ID0

Communication Type: 1.UDP-GigaBit Ethernet

Wireless Mode:

2. Modify the IP MAC address and port

FPD MAC Addr: 11 22 33 44 55 66

☐ FPD IP Addr & Port: 192 . 168 . 10 . 40 32897

3. Modify the local IP Mac address and port

Local Network Card: Intel(R) Ethernet Controller (3) I225-LM

Local IP Addr & Port: 192 . 168 . 10 . 20 32896

4. Update the detector MAC IP and port

Update FPD Mac & IP & Port

Refresh

MTU & Receive Buffers

Flat Panel Detector B:ID1

Communication Type: 1.UDP-GigaBit Ethernet

Wireless Mode:

FPD MAC Addr: 11 22 33 44 55 66

☐ FPD IP Addr & Port: 192 . 168 . 10 . 40 32897

Local Network Card: Intel(R) Ethernet Controller (3) I225-LM

Local IP Addr & Port: 192 . 168 . 10 . 20 32896

Update FPD Mac & IP & Port

Refresh

MTU & Receive Buffers

5. Save to the profile

Ok Cancel Apply

Figure 34

5. Restore to factory settings, the product IP address will be restored to the "192.168.10.40". If you forget the IP address you have modified, the product will fail to connect, a broadcasting packet monitoring tool can be used to check the details. "

11. Dual Panel Integration

The SDK package has integrated Gigabit Ethernet, 2.5G Ethernet, 10G Ethernet and optical fiber interface communication mechanisms together, also including “dual panel” or “multi-panel” solutions.

Dual panel or multi-panel connect solution needs software to distinguish the connected devices by IP address or PCIe card ID.

1. Initialize DLL: HBI_InitDual or HBI_InitEx;

After initialization, two instance handles are established, all subsequent operations are based on these two instance handles, also they are used for distinguishing different products.

HBI_Init (0) and HBI_Init (1)

2. Register callback function: HBI_RegEventCallBackFun, register the callback functions for panel A and panel B separately.
3. Connection: HBI_ConnectDetector, connecting the callback functions of panel A and panel B separately.
4. Setting trigger mode and image calibration options: HBI_TriggerAndCorrectDisplay.
5. Other settings, parameters setting, etc.
6. Disconnect: HBI_DisconnectDetector.
7. Release DLL resources: HBI_DestroyEx.

Taking 1G Ethernet interface dual panel connection as an example

1. Panel A and B must have different MAC and IP addresses, it is suggested to test the A and B separately before integration.
2. Network card A is connected to product A, while network card B is connected to product B, and the network card A and B have different IP address, which can be in the same network segment or in different network segment.
3. The default IP and MAC address for panel are 192.168.10.40 and 11-22-33-44-55-66. User can modify the IP and MAC addresses as requirement.

The SDK package integrates communication mechanisms for Gigabit Ethernet, 2.5G Ethernet, 10G Ethernet, and optical fiber interfaces. It also supports “dual panel” and “multi-panel” solutions.

For dual panel or multi-panel connection solutions, the software must distinguish connected devices by their IP address or PCIe card ID.

1. Initialize DLL: Use HBI_InitDual or HBI_InitEx.

After initialization, two instance handles are created. All subsequent operations are based on these two handles, which are used to distinguish between different products.

HBI_Init(0) and HBI_Init(1)

2. Register callback function: Use HBI_RegEventCallBackFun to register callback functions for panels A and B separately.

3. Connection: Use HBI_ConnectDetector to connect the callback functions for panels A and B separately.
4. Set trigger mode and image calibration options: Use HBI_TriggerAndCorrectDisplay.
5. Other settings, such as parameter configuration, etc.
6. Disconnect: Use HBI_DisconnectDetector.
7. Release DLL resources: Use HBI_DestroyEx.

Taking the 1G Ethernet interface dual panel connection as an example:

1. Panels A and B must have different MAC and IP addresses. It is recommended to test panels A and B separately before integration.
2. Network card A should be connected to product A, while network card B should be connected to product B. Network cards A and B must have different IP addresses, which can be on the same network segment or on different segments.
3. The default IP and MAC addresses for a panel are 192.168.10.40 and 11-22-33-44-55-66, respectively. Users can modify the IP and MAC addresses as needed.

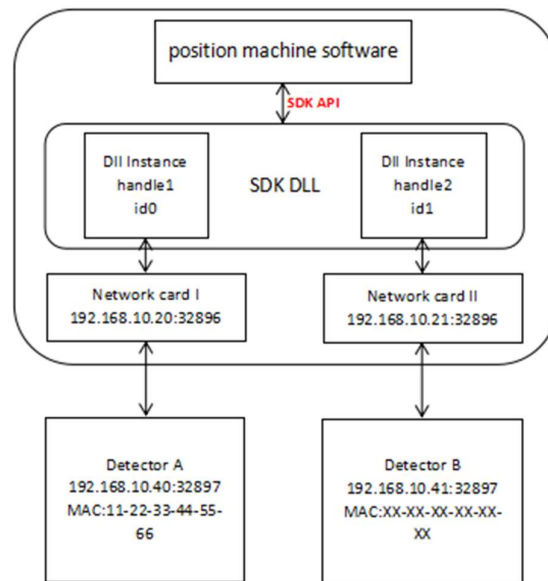


Figure 35

A direct point-to-point connection between the computer and the product is recommended. Using a router or switch may cause transmission instability in dual or multi-panel connection solutions.

The multi-panel solution is similar to the dual panel solution.

12. Appendix

Appendix I: Open-source code example

To enhance the convenience of system integration, an open-source code example project has been developed for user reference.

Table 53.

No	SDK Demo	Remarks
1	HB_SDK_DEMO2013	Developed based on C++ / VS MFC, data matrix process relies on Opencv341, open-source code package for customer integration.
2	HBI_FPDCSharpdemo	Developed based on C# language, including interface calling, image display, parameter configuration, open-source code package.
3	DemoDll	Developed based on VS console, dynamic library.
4	DemoLib	Developed based on VS console, static library.
5	HB_SDK_DEMO2013_Two	Developed based on C++ / VS MFC, an example of dual panel solution, including interface calling, image display, parameter configuration, open-source code package.
6	HB_SDK_DEMO2013_SIX	Developed based on C++ / VS MFC, an example of multi board solution, including interface calling, image display, parameter configuration, open-source code package. It can connect up to 6 panels.
7	HBI_FPDCSharpdemo*_en_six	Developed based on C# language, an example of multi board solution, including interface calling, image display, parameter configuration, open-source code package. It can connect up to 6 panels.
8	hbi_python_demo	Developed based on Python language, including interface calling, image display, parameter configuration, open-source code package.
9	HB_SDK_QTDemo	Developed based on QT platform, including interface calling, image display, parameter configuration, open-source code package.

Appendix II: Table of panel types and resolutions

Table 54.

No	Panel size	Pixel pitch	Resolution (width x height)	Type	Interface	Comments
1	4343	140um	3072 x 3072	Static	Gigabit UDP	
2	3543	140um	2560 x 3072	Static	Gigabit UDP	

3	1613	125um	1248 x 1024	Static	Gigabit UDP	
4	3030	140um	2048 x 2048	Static	Gigabit UDP	
5	2530	85um	2816 x 3584	Static / Dynamic	Gigabit Ethernet UDP / Optical Fiber	
6	3025	140um	2048 x 1792	Static	Gigabit UDP	
7	4343	100um	4288 x 4288	Static	Gigabit Ethernet UDP / Optical Fiber	
8	2530	75um	3072 x 3840	Static	Gigabit UDP	
9	2121	200um	1024 x 1024	Static	Gigabit UDP	
10	1412	50um	2784 x 2400	Dynamic	Optical Fiber	
11	0606	50um	1056 x 1200	Dynamic	Optical Fiber	
12	2323	148um	1536 x 3072	Dynamic	Optical Fiber	
13	2530	100um	2432 x 3072	Dynamic / Wireless	Gigabit Ethernet UDP / Optical Fiber	
14	3030	148um	2048 x 2048	Dynamic	Optical Fiber	
15	3543	100um	3534 x 4302	Dynamic	Gigabit Ethernet UDP / Optical Fiber	
16	1616	105um	1536 x 1536	Dynamic	Optical Fiber	
17	43100	140um	7680 x 3072	Static / Dynamic	Gigabit Ethernet UDP / Optical Fiber	
18	1515	100um	1070 x 1200	Dynamic	Optical Fiber	
19	2121	100um	2063 x 2049	Dynamic	Optical Fiber	
20	3131	100um	3095 x 3073	Dynamic	Optical Fiber	
21	2301	100um	2331 x 76	Dynamic	Gigabit UDP	
22	1917	120um	1536, 1386	Dynamic	Gigabit UDP	
23	3030	99um	3072, 3072	Dynamic	Gigabit UDP	
24	1515	75um	2048, 2048	Dynamic	Gigabit UDP	
25	2530	70um	3488, 4356	Dynamic	Gigabit UDP	
26	2530	85um	3584, 3072	Dynamic	Gigabit UDP	

27	4315	140um		Dynamic	Gigabit UDP	
28	1313	85um	1536, 1536	Dynamic	Gigabit UDP	
29	3030WOA	140um	2048, 2048	Dynamic	Gigabit UDP	
30	4343WOA	140um	3072, 3072	Dynamic	Gigabit UDP	
31	1818	84um	2048, 2048	Dynamic	Gigabit UDP	
32	1717	111um	1536, 1536	Dynamic	Gigabit UDP	
33	4343	100um	4302, 4302	Dynamic	Gigabit UDP	

PGA gain information.

PGA	Value
1	0.6pC
2	1.2pC
3	2.4pC
4	3.6pC
5	4.8pC
6	7.2pC
7	9.6pC
8	LFW (COMS)
9	HFW (COMS)
10	0.3pC
11	0.15pC

Figure 36

Appendix III: Configuration Files

Connect the configuration file hbi_fpd_init.ini

[DLL_CONFIG]

COMM_TYPE=2 # Communication mechanism, 1-UDP jumbo packet, 2-PCIe optical fiber, other - UDP standard packet.

FPD_IP=192.168.10.40 # Panel IP address, for Ethernet communication.

FPD_PORT=32897 # Panel Port Number.

LOCAL_IP=192.168.10.20 # Local network card IP address.

LOCAL_PORT=32896 # Local network card port.

TRIGGER_MODE=7 # Trigger mode.

OFFSET_ENABEL=3 # offset correction enable.

GAIN_ENABLE=2 # gain correction enable.

DEFECT_ENABLE=2 # defect correction enable.

LIVE_ACQ_TIME=1000 # Continuous acquisition interval time (static panel).

SELF_DUMPING_TIME=500 # Continuous acquisition interval time (dynamic panel).

Prepare_TIME=1500 # Prepare delay time.

DO_OFFSET=1 # Firmware pre-offset template generation setting

Appendix IV: Callback Function Parameters and States

Callback function parameter table

// Notice: call back function

//@ USER_CALLBACK_HANDLE_ENVENT

//@ pContext: Parameter 1, software object pointer, can be null

//@ ufpdId: Parameter 2, panel ID

//@ byteEventId: Parameter 3, Event ID, refer to enum eCallbackEventCommType in HbiType. h

//@ PVEventParam1: Parameter 4, configure structure pointer or image structure pointer

//@ nEventParam2: Parameter 5, such as data size

//@ nEventParam3: Parameter 6, such as frame number, frame ID

//@ nEventParam4: Parameter 7, such as frame rate

Typedef int (* USER_CALLBACK_HANDLE_ENVENT) (void * pContext, int ufpdId, unsigned char byteEventId, void * PVEventParam1, int nEventParam2, int nEventParam3, int nEventParam4);

PContext: The object pointer passed in from the upper computer, which can be NULL.

Table 55.

Notes	UfpdId	ByteEventId	PVEvent Param1	NEvent Param2	NEvent Param3	NEvent Param4
Product parameter	Instance ID	ECALLBACK_TYPE_ROM_UPLOAD	Null pointer	Configuratio n parameter data length	Invalid	Invalid
Panel status	Instance ID	ECALLBACK_TYPE_FPD_STATUS	Null pointer	Status	Invalid	Invalid
Parameters configuration	Instance ID	ECALLBACK_TYPE_SET_CFG_OK	Null pointer	Invalid	Invalid	Invalid

Single frame acquisition	Instance ID	ECALLBACK_TYPE_Single_IMAGE	Image structure pointer	Image buffer length	Frame number	Frame rate
Continuous image acquisition	Instance ID	ECALLBACK_TYPE_MULTIPLE_IMAGE	Image structure pointer	Image buffer length	Frame number	Frame rate
Offset template data	Instance ID	ECALLBACK_TYPE_OFFSET_TMP	Image structure pointer	Image buffer length	Frame number	Frame rate
	Instance ID	ECALLBACK_TYPE_PREVIEW_IMAGE	Image structure pointer	Image buffer length	Frame number	Frame rate
Number of image overlay	Instance ID	ECALLBACK_OVERLAY_NUMBER	Null pointer	Stacking counter	Frame number	Frame rate
16 bit overlay data	Instance ID	ECALLBACK_OVERLAY_16BIT_IMAGE	Image structure pointer	Image buffer length	Frame number	Frame rate
32-bit overlay data	Instance ID	ECALLBACK_OVERLAY_32BIT_IMAGE	Image structure pointer	Image buffer length	Frame number	Frame rate
Generate Template	Instance ID	ECALLBACK_TYPE_GENERATE_TEMPLATE	Null pointer	Image template generation result	Template type	Invalid
...	...	...	...	...	...	...

Appendix V: Switching from old version to the latest version

The ultimate goal of the comprehensive version is to standardize interface functions, parameters, etc., simplify the integration process.

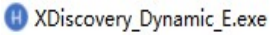
The current version supports: Ethernet standard UDP, Ethernet Jumbo packet UDP, PCIe optical fiber, and wireless connections. This unique SDK package is compatible with all types of firmware.

Over time, the SDK has undergone multiple version iterations. We will provide a detailed description of the similarities and differences between these versions.

(1) What versions have been released so far

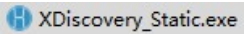
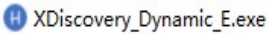
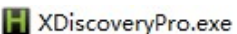
Table 56.

Name	Library	Demo UI Software	Remarks
------	---------	------------------	---------

Ethernet interface for static version	HBI_FPD.DLL HBI_FPD.lib	XDiscovery_Static.exe 	The basic draft version in the early age, Ethernet interface with standard UDP packet, just for static type product integration.
Ethernet interface with Jumbo packet for static version	HBI_FPD.DLL HBI_FPD.lib	XDiscovery_Static.exe 	The 2nd version, Ethernet interface with Jumbo UDP packet, designed for static type product which supports firmware pre-offset calibration.
Ethernet interface with Jumbo packet for dynamic version	HBI_FPD_E.DLL HBI_FPD_E.Lib	XDiscovery_Dynamic_E.Exe 	The 3rd version, Ethernet interface with Jumbo UDP packet, designed for dynamic type product which supports firmware image calibration.
Merged version	HBISDKApi.dll HBISDKApi.lib	XDiscovery.exe 	The uniform unique version, it is compatible with all kinds of interface and products.

(2) Header files, library files and dependency libraries

Table 57.

Version	Header File	Library Files	Dependency Library	Demo UI Software
Network port static version	HbDLError.h	HBI_FPD.DLL	Opencv_Core249.9ll	XDiscovery_Static.exe
Ethernet static Jumbo frame version	HbDIIType.h HbFpdDll.h	HBI_FPD.lib	Opencv_Highgui249.9ll Opencv_Imgproc249.9ll	
Network port dynamic version	HbDLError.h HbDIIType.h HbFpdDll.h	HBI_FPD_E.DLL HBI_FPD_E.Lib	Opencv_Core249.9ll Opencv_Highgui249.9ll Opencv_Imgproc249.9ll	XDiscovery_Dynamic_E.Exe 
Merged version (Ethernet, optical, wireless; static, dynamic)	HbiError.h HbiFpd.h HbiType.h	HBISDKApi.dll HBISDKApi.lib	Opencv_World341.dll	XDiscovery.exe 

(3) Change in callback function

Note: Image events, note that the buffer is a pointer to the structure

Table 58.

Version	Callback Function	Remarks
Network static version	HandleCommandEvent (unsigned char command, void * buff, int len, int nid)	ByteEventid: Parameter 1, Event ID, refer to enum eCallbackEventCommType in HbDllType. h
Ethernet static Jumbo frame version		Buff: Parameter 2, configure structure pointer or image buffer pointer
Network dynamic version		Len: Parameter 5, data size or status
Merged version (Ethernet, optical, wireless; static, dynamic)	HandleCommandEventA (void * pvContext, int nDevId, unsigned char byteEventid, void * pvParam1, int nParam2, int nParam3, int nParam4)	PContext: Parameter 1, software object pointer, can be null. UfpdId: Parameter 2, panel ID ByteEventid: Parameter 3, Event ID, refer to enum eCallbackEventCommType in HbiType. h PVEventParam1: Parameter 4, configure structure pointer or image structure pointer NEventParam2: Parameter 5, data size or status NEventParam3: Parameter 6, frame number, frame ID NEventParam4: Parameter 7, frame rate

(4) Similarities and Differences in Parameter Types

Table 59.

Interface	Interface and Parameters	Network interface static version	Network interface dynamic version	Merged Version
-----------	--------------------------	----------------------------------	-----------------------------------	----------------

Single frame image acquisition Continuous image acquisition	HBI_LiveAcquisition (void * handle, FPD_AQC_MODE_mode); HBI_SingleAcquisition (void * handle, FPD_AQC_MODE_mode); Note: The number and name of parameters are the same, but the members are different.	<pre>typedef struct FPD_AQC_MODE_St{ EnumLIVE_ ACQUISITION eLivetype; int ngrouppno; int nframesum; int ndiscard; int nframeid; }FPD_AQC_MODE;</pre>	<pre>typedef struct FPD_AQC_MODE_St{ EnumIMAGE_ ACQ_MODE aqc_mode; int ngrouppno; int nframesum; int ndiscard; int nframeid; int nLiveMode; BOOL isOverLap; BOOL bSimpleGT; EGRAY_MODE nGrayBit; }FPD_AQC_MODE;</pre>	<pre>typedef struct FPD_AQC_MODE_ St{ EnumIMAGE_ACQ_CMD eAqccmd; EnumLIVE_ ACQUISITION eLivetype; int ngrouppno; int nAcqnumber; int ndiscard; int nframeid; }FPD_AQC_MODE;</pre>
Generate template	The interface functions and parameters are different	HBI_ImmediateGenerateTemplate (void * handle, EnumIMAGE_ACQ_MODE_mode, int bpreview);	HBI_GenerateTemplateEx (void * handle, EnumGENERATE_TEMPLAT E_TYPE templateType);	HBI_GenerateTemplate (void * handle, EnumIMAGE_ACQ_CMD_mode, int bpreview);
Download template	The interface functions and parameters are different	HBI_DownloadTemplate (void * handle, DOWNLOAD_FILE * downloadfile); HBI_DownloadTemplateEx (void * handle, DOWNLOAD_MODE * downloadmode); HBI_DownloadTemplateBy Type (void * handle, int filetype); HBI_DownloadTemplate (void * handle, DOWNLOAD_FILE * downloadfile);	HBI_DownloadTemplate (void * handle, HBI_DOWNLOAD_FILE * downloadfile); HBI_DownloadOffsetTemplate (void * handle); HBI_DownloadFileToFirew are (void * handle, DOWNLOAD_TOILE&dparam);	HBI_DownloadTemplate (void * handle, DOWNLOAD_FILE * downloadfile); HBI_DownloadTemplate ByType (void * handle, int filetype);

Single frame acquisition	Interface function names are different	HBI_Prepare (void * handle);	HBI_Prepare (void * handle);	HBI_SinglePrepare (void * handle);
Prepare delay time setting	The interface functions and parameters are different	HBI_SetPreAcqTm	HBI_SetSinglePrepareTime;	HBI_SetSinglePrepareTime
Continuous image acquisition interval time setting	The interface functions and parameters are different	Ethernet static Jumbo packet: HBI_SetSelfDumpingTime; Network port static/wireless: HBI_SetLiveAcquisitionTime.	HBI_SetAcqSpanTm;	Static Jumbo packet at Ethernet Port/Dynamic Ethernet Port/Optical Port: HBI_SetSelfDumpingTime Network port static/wireless: HBI_SetLiveAcquisitionTime;

(5) "Multi-panel" solutions

The **latest version** supports the "dual panel" and "multi-panel" solution.

(6) Support for multi template management

The image calibration template file and log file are automatically generated based on the dynamic library DLL path, they will be generated and saved in the same directory as the dynamic library DLL file.

Table 60.

Version	Template path	Naming rules
Network interface static version Ethernet static Jumbo packet version	DLL file path\ template\	Offset template: offset.raw, Gain templates: gaina.raw, gainb.raw Defect template: defect. raw
Network port dynamic version	DLL file path\ template\	Divide into multiple templates S01B1F01P5L0000T0000W0000H0000 S01-panel size is 1, B1-Binning1x1,

		P5-PGA Level is 5, L0000T0000W0000H0000-ZOOM parameters. Template file storage path: DLL file path\template\S01B1F01P5L0000T0000W0000H0000\ Offset template:offset.raw, Gain templates: gaina.raw, gainb.raw Defect template: defect. raw
Merged version (Ethernet, optical, wireless; static, dynamic)	DLL file path\ hbi_id0_template\ Among them, "Panel A" log hbi_id0_template, on the contrary, "Plate B" means HBI_Id1_TEMPLATE.	Offset template path: DLL file path\hbi_id0_template, id0 represents panel A, panel ID is 0, id0 represents panel B, panel ID is 1; Offset template: offsetS02B1F00250.raw represents panel size is 2, and binning1x1, 00250 is a continuous acquisition interval time, unit is ms. Gain and defect template paths: DLL file path \ hbi_id0_template \ S02B1P5V000 \. S02B1P5V000, S02-panel size is 2, B1=binning1x1, P5-PGA Level 5, V000- not used yet. Gain templates: gaina.raw, gainb.raw Defect template: defect. raw

Example:

Taking Panel-1613 as an example							
Key letters							
S	B	P	V	Z	F	r	m
Panel size Reference tablet model information table	Binning Type 1-binning1x1 2-binning1x1 3-binning1x1 4-binning1x1	PGA Level 1-11 represents different PGA level, refer to PGA level table.	High Voltage (Dose parameters) Currently, due to the inability to obtain specific parameters, it defaults to 0.	ZOOM Rect Rect(left.top,width,height)	FPS (Affects offset templates) Dynamic 1000ms/dumping time Static 1000ms/living time Currently, for character alignment, it has been expanded by 1000 times	Rotate 1- Right Rotate 90° 2- Left Rotate 90° 0 or other default images do nothing	Mirror (1-Vertically Mirror;2- Horizontal Mirror) 1- Vertically Mirror 2- Horizontal Mirror 3- 5 indicates that the firmware has completed different image processing, independent of template generation. 0 or other default images are not
Offset template (16-bit raw file)		Template file		Notes			
Example 1	offsetS03B1P6F20000			Eg, time interval50ms, 20 frames represent 1000000/50=F20000, If the time interval is 2000ms, 0.5 frames represent F00500, the time interval is 100000, and 0.01 frames represent F000010			
Example 2	offsetS03B1P6F20000Z0000049612480032			Z0000049612480032;Indicates the ZOOM rect parameter:left=0;top=496,width=1248,height=32			
Example 3	offsetS03B1P6F20000Z0000049612480032_r1m2			r1m2;Indicates that the image has been rotated right and mirrored horizontally			
Gain template (32-bit raw file)		Folder		Template file		Notes	
Example 1		S03B1P6V000		gaina.raw, gainb.raw		Firmware only updates gaina.raw Z0000049612480032;Indicates the ZOOM rect parameter:left=0;top=496,width=1248,height=32	
Example 2		S03B1P6V000Z0000049612480032		gaina.raw, gainb.raw		Indicates that the image has been rotated right and horizontally mirrored and generate gain template.	
Example 3		S03B1P6V000		gaina_r1m2.raw, gainb_r1m2.raw		Indicates that the image has been rotated right and horizontally mirrored and generate gain template.	
		S03B1P6V000Z0000049612480032		gaina_r1m2.raw, gainb_r1m2.raw		Indicates that the image has been rotated right and horizontally mirrored and generate gain template.	
Defect template (16-bit raw file)		Folder		Template file		Notes	
Example 1		S03B1P6V000		defect.raw		The grayscale value of 65535 represents bad points, and 0 represents normal points Z0000049612480032;Indicates the ZOOM rect parameter:left=0;top=496,width=1248,height=32	
Example 2		S03B1P6V000Z0000049612480032		defect.raw		Indicates that the image has been rotated right and horizontally mirrored and generate defect template.	
Example 3		S03B1P6V000		defect_r1m2.raw		Indicates that the image has been rotated right and horizontally mirrored and generate defect template.	
		S03B1P6V000Z0000049612480032		defect_r1m2.raw			
Notes							
1. The ZOOM parameter only supports binning1x1.							
2. The offset template is related to the panel size, binning type, PGA gear, frame rate, Zoom parameters, and software side rotation mirror parameters.							
3. The gain and defect templates are related to the panel size, binning type, PGA gear, Zoom parameters, and software side rotation mirror parameters.							
4. At present, the rotation mirror processing sequence is: rotate first and then mirror.							
						测试 Win	

Figure 37

Appendix VI: Window width and window level adjustment

```
double dFactor, min, max;
int nPixelVal;
```

```
min = (2*window_center - window_width)/2.0 + 0.5;
max = (2*window_center + window_width)/2.0 + 0.5;
dFactor = 255.0/(double)(max - min);
```

```
for (i = 0; i < nNumPixels; i++)
```

```
{
    if (pixel_val < min)
    {
        disp_pixel_val = 0;
        continue;
    }
```

```
    if (pixel_val > max)
    {
        disp_pixel_val = 255;
        continue;
    }
```

```
    nPixelVal = (int)((pixel_val - min)*dFactor);
```

```
    if (nPixelVal < 0)
        disp_pixel_val = 0;
    else if (nPixelVal > 255)
        disp_pixel_val = 255;
    else
        disp_pixel_val = nPixelVal;
```

```
}
```